



DATA STRUCTURES AND ITS APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

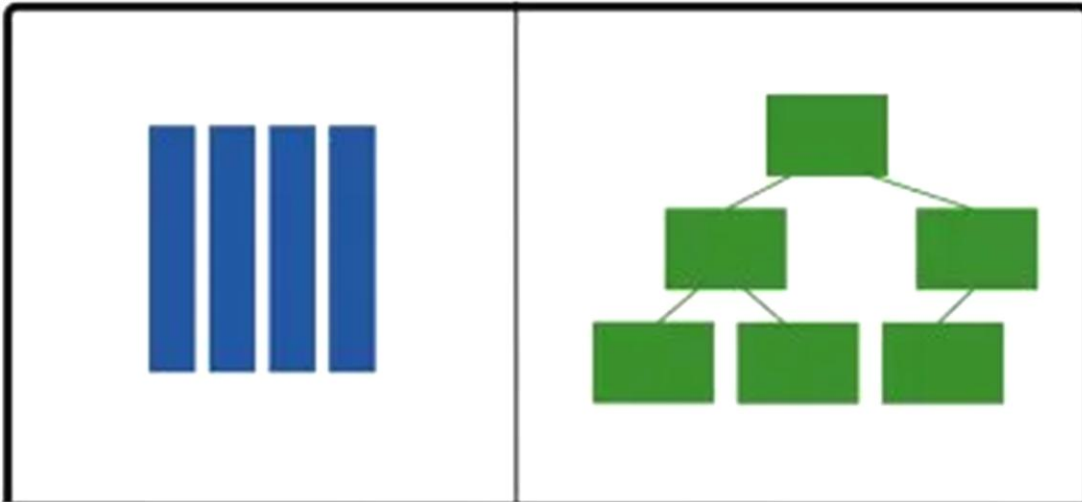
DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Data Structures

Vandana M L

Department of Computer Science and Engineering

Data Structure is a scheme of organizing data in the memory of the computer in such a way that various operations can be performed efficiently on this data



DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Data Structures

Why Data Structure?

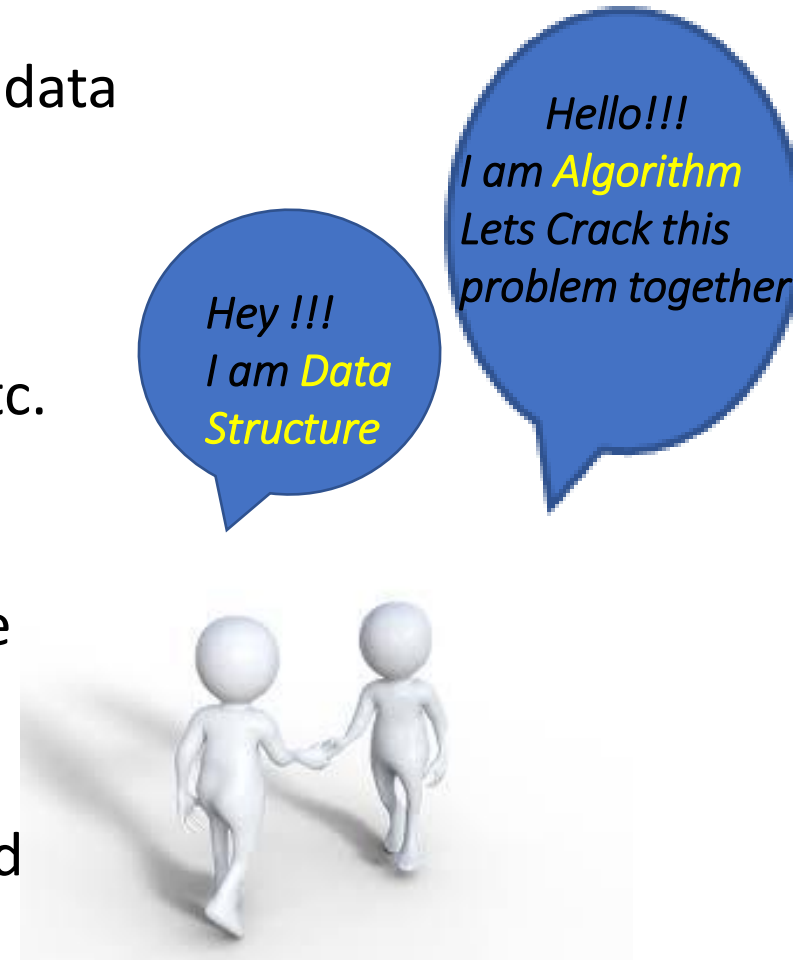


DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Data Structures

Why Data Structures?

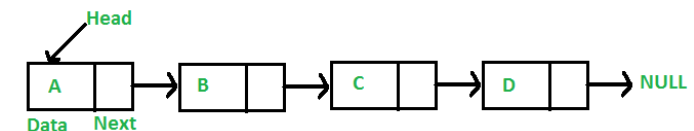
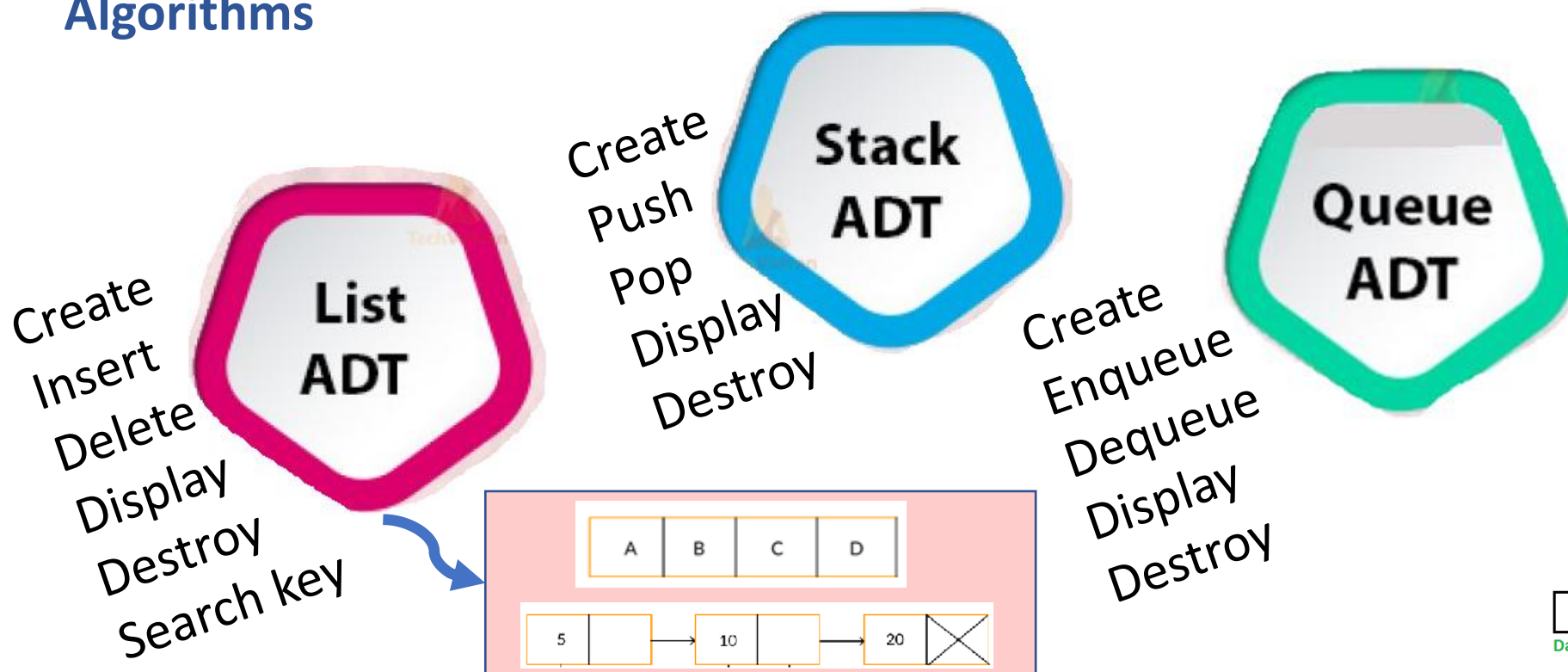
- Computer systems deal with large amount of data (text ,image, relational data etc.)
- Data is just the raw material for information, analytics, business intelligence, advertising, etc.
- The way data is organized in memory plays a key role in deciding the time complexity of the algorithms designed for solving the problems
- Data Structures and algorithm go hand in hand



Importance of Data Structures

- Data Structures is most fundamental and building block concept in computer science
- Good knowledge of Data Structures is required to build efficient software systems

- Abstract Data Type is used to represent data and operations associated with an entity from the point of view of user **irrespective of implementation**
- ADT can be implemented using one or more **Data Structures** and **Algorithms**



➤ Linear Data Structures

Stack, Queue, Linked List

➤ Non Linear Data Structures

Tree , Graph



Linear Organisation

Stack

Queue

Linked List

Linear List using Array



Non Linear Organisation

*Tree
Graph*

DATA STRUCTURES AND ITS APPLICATIONS

Few Applications of Linear Data Structures



➤ Array

- To implement other data structures
- To store files in memory

➤ Linked Lists

- To implement other data structures
- To manipulate large numbers

➤ Stacks

- Recursion
- Infix to postfix conversion

➤ Queues

- Process Scheduling
- Event handling

➤ Tree

- Auto complete features (Trie)
- Used by operating systems to maintain the structure of a file system

➤ Heaps

- Priority Queue implementation
- Heap Sort

➤ Graphs

- Computer Networks
- Shortest Path Problems

Unit -1 : Linked List and Stacks

Review of C , Static and Dynamic Memory Allocation. Linked List: Doubly Linked List, Circular Linked List – Single and Double, Multilist: Introduction to sparse matrix (structure). Skip list Case study: Dictionary implementation using skip list Stacks: Basic structure of a Stack, Implementation of a Stack using Arrays & Linked list. Applications of Stack: Function execution, Nested functions, Recursion: Tower of Hanoi. Conversion & Evaluation of an expression: Infix to postfix, Infix to prefix, Evaluation of an Expression, Matching of Parenthesis.

Unit -2 : Queues and Trees

Queues & Dequeue: Basic Structure of a Simple Queue, Circular Queue, Priority Queue, Dequeue and its implementation using Arrays and Linked List. Applications of Queue: Case Study – Josephus problem, CPU scheduling- Implementation using queue (simple /circular). General: N-ary trees, Binary Trees, Binary Search Trees (BST) and Forest: definition, properties, conversion of an N-ary tree and a Forest to a binary tree. Traversal of trees: Preorder, Inorder and Postorder.

Unit -3 : Application of Trees and Introduction to Graphs

Implementation of BST using arrays and dynamic allocation: Insertion and deletion operations, Implementation of binary expression tree., Threaded binary search tree and its implementation. Heap: Implementation using arrays. Implementation of Priority Queue using heap - min and max heap. Applications of Trees and Heaps: Implementation of a dictionary / decision tree (Words with their meanings). Balanced Trees: definition, AVL Trees, Rotation, Splay Tree, Graphs: Introduction, Properties, Representation of graphs: Adjacency matrix, Adjacency list. Implementation of graphs using adjacency matrix and lists. Graph traversal methods: Depth first search, Breadth first search techniques. Application: Graph representation: Representation of computer network topology.

Unit -4 : Applications of Graphs , B-Trees, Suffix Tree and Hashing

Application of BFS and DFS: Connectivity of graph, finding path in a network. Suffix Trees: Definition, Introduction of Trie Trees, Suffix trees. Implementations of TRIE trees, insert, delete and search operations. Hashing: Simple mapping / Hashing: hash function, hash table, Collision Handling: Separate Chaining & Open Addressing, Double Hashing, and Rehashing. Applications: URLs decoding, Word prediction using TRIE trees / Suffix Trees.

Text Book :

"Data Structures using C / C++", Langsum Yedidiah, Moshe J Augenstein, Aaron M Tenenbaum Pearson Education Inc, 2nd edition, 2015.

Reference Book:

"Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019

Data Structures and its Applications

Evaluation Policy



Pattern:	ISA: 50 marks ESA: 50 marks
ISA 1 / ISA 2:	Hybrid / CBT, conducted for 40 marks, scaled down to 20 marks (each) (16 * 1) mark MCQs + (4 * 2) marks MCQs + (4 * 4) marks descriptive questions. Total: 20 + 20 marks.
Lab Component/FSA:	10 Labs (Implementation based): 10 marks Jackfruit problem (Mini): 10 marks Total : 20 Marks
Assignment / Experiential Learning	Banana problem: 5 marks (pen and paper) (classnotes , HW) Orange problem: 5 marks (Implementation based in assessment center) Total: 10 marks
ESA:	25 marks from each unit, conducted for 100 marks, reduced to 50 marks
ISA + ESA + Lab + Experiential Learning	120 reduced to 100 marks



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Memory Allocation

Vandana M L

Department of Computer Science and Engineering

- Static Memory Allocation
- Dynamic Memory Allocation

- allocated by the compiler.
- Exact size and type of memory must be known at compile time
- Memory is allocated in stack area

```
int b;
```

```
int c[10] ;
```

- Memory allocated can not be altered during run time as it is allocated during compile time
- This may lead to under utilization or over utilization of memory
- Memory can not be deleted explicitly only contents can be overwritten
- Useful only when data size is fixed and known before processing

- Dynamic memory allocation is used to obtain and release memory during program execution.
- It operates at a low-level
- Memory Management functions are used for allocating and deallocating memory during execution of program
- These functions are defined in “stdlib.h”

Dynamic Memory Allocation Functions:

- Allocate memory - malloc(), calloc(), and realloc()
- Free memory - free()

To allocate memory use

```
void *malloc(size_t size);
```

- Takes number of bytes to allocate as argument.
- Use sizeof to determine the size of a type.
- Returns pointer of type void *. A void pointer may be assigned to any pointer.
- If no memory available, returns NULL.

To allocate space for 100 integers:

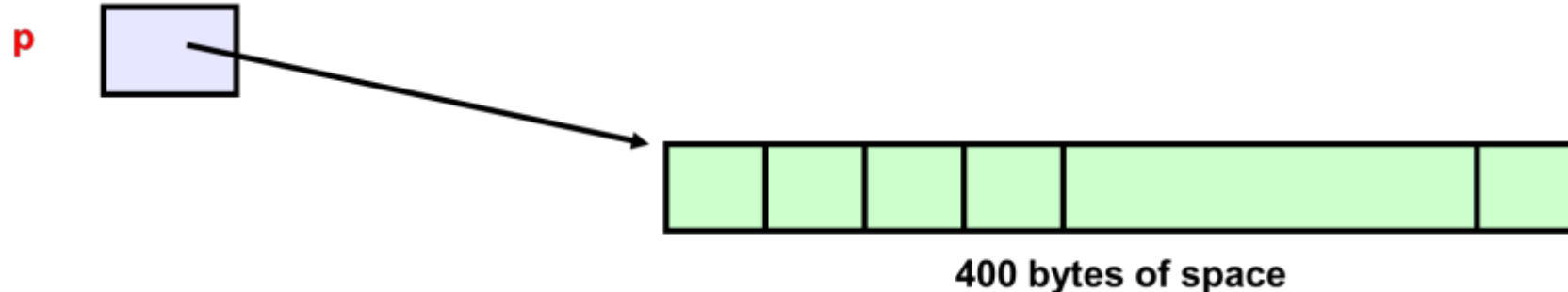
```
int *p;
```

```
if ((p = (int*)malloc(100 * sizeof(int))) == NULL){
```

```
    printf("out of  
memory\n");  
    exit();
```

```
}
```

- Note we cast the return value to int*.
- Note we also check if the function returns NULL.



Dynamic Memory Allocation Functions: malloc()

- `cptr = (char *) malloc (20);`

Allocates 20 bytes of space for the pointer `cptr` of type `char`

- `sptr = (struct stud *) malloc(10*sizeof(struct stud));`

Allocates space for a structure array of 10 elements. `sptr` points to a structure element of type `struct stud`

Always use sizeof operator to find number of bytes for a data type, as it can vary from machine to machine

Dynamic Memory Allocation Functions: malloc()

- **malloc** always allocates a block of contiguous bytes
 - The allocation can fail if sufficient contiguous memory space is not available
 - If it fails, **malloc** returns **NULL**

```
if ((p = (int *) malloc(100 * sizeof(int))) == NULL)
{
    printf ("\n Memory cannot be allocated");
    exit();
}
```

The n integers allocated can be accessed as ***p, *(p+1), *(p+2),..., *(p+n-1)** or just as **p[0], p[1], p[2], ...,p[n-1]**

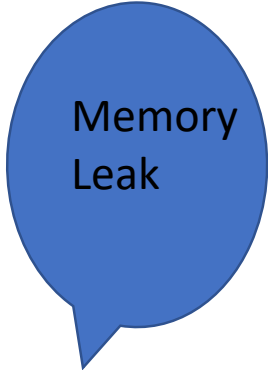
To release allocated memory use

`free(ptrvariable)`

- Deallocates memory allocated by malloc().
- Takes a pointer as an argument.

e.g

. `free(newPtr);`

A blue speech bubble with a white outline, containing the text "Memory Leak".

Memory
Leak

A blue speech bubble with a white outline, containing the text "Dangling pointer".

Dangling
pointer

Similar to malloc(),

But allocated memory space are zero by default..

calloc() requires two arguments –

```
void *calloc(size_t nitem, size_t size);
```

Example

```
int *p;
```

```
p=(int*)calloc(100,sizeof(int));
```

returns a void pointer if the memory allocation is successful,
else it'll return a NULL pointer.

Reallocate a block

Two arguments

- Pointer to the already allocated block
- Size of new block

```
int *ip;
```

```
ip = (int*)malloc(100 * sizeof(int));
```

```
...
```

```
/* need twice as much space */
```

```
ip = (int*)realloc(ip, 200 * sizeof(int));
```


Memory Allocation

- Static Memory allocation
- Dynamic memory allocation

Apply the concepts to implement C program for the following problem statement

- Multiply two matrices . Allocate the memory for the matrices dynamically

1. Which of the following is true about static memory allocation?

- a) Memory is allocated during runtime.
- b) The size of memory block can be altered during execution.
- c) Memory is allocated during compile time and remains fixed.
- d) It uses malloc() and free() functions.

1. Which of the following is true about static memory allocation?

- a) Memory is allocated during runtime.
- b) The size of memory block can be altered during execution.
- c) Memory is allocated during compile time and remains fixed.
- d) It uses malloc() and free() functions.

2. In C, which of the following dynamic allocation functions returns memory that is not initialized?

- a) malloc()
- b) calloc()
- c) realloc()
- d) free()

2. In C, which of the following dynamic allocation functions returns memory that is not initialized?

- a) `malloc()`
- b) `calloc()`
- c) `realloc()`
- d) `free()`

3. Which of the following statements is false regarding dynamic memory allocation?

- a) It is performed at runtime.
- b) It uses heap memory.
- c) The size of memory block cannot be changed once allocated.
- d) `realloc()` can resize an already allocated block.

3. Which of the following statements is false regarding dynamic memory allocation?

- a) It is performed at runtime.
- b) It uses heap memory.
- c) The size of memory block cannot be changed once allocated.
- d) realloc() can resize an already allocated block.

4. What will happen if free() is not called after using dynamically allocated memory?

- a) The program will not compile.
- b) Memory leak will occur.
- c) The stack will overflow.
- d) The data will be automatically deleted.

4. What will happen if free() is not called after using dynamically allocated memory?

- a) The program will not compile.
- b) Memory leak will occur.
- c) The stack will overflow.
- d) The data will be automatically deleted.

5. Which of the following is incorrect?

- a) Static allocation uses stack, dynamic uses heap.
- b) Static allocation is determined at compile time, dynamic at runtime.
- c) Static memory can be resized using realloc().
- d) Dynamic memory requires explicit allocation using malloc/calloc.

5. Which of the following is incorrect?

- a) Static allocation uses stack, dynamic uses heap.
- b) Static allocation is determined at compile time, dynamic at runtime.
- c) Static memory can be resized using realloc().
- d) Dynamic memory requires explicit allocation using malloc/calloc.



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

+91 7411716615



PES
UNIVERSITY

DATA STRUCTURES AND ITS APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

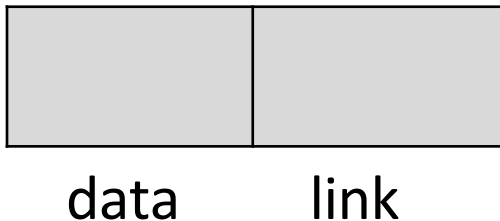
Singly Linked List: Node Structure



Defining node structure

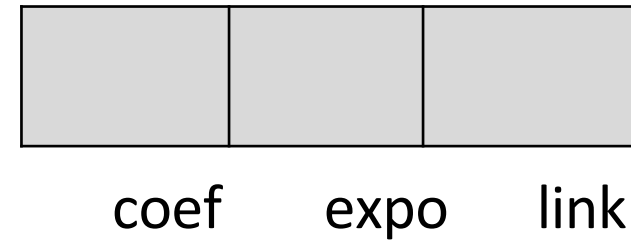
```
struct node
{
int data;
struct node *link;
};
```

```
typedef struct node NODE;
```



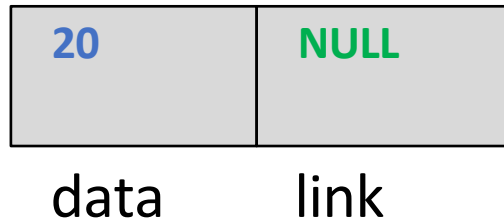
```
struct polydata
{
int coeff;
int expo;
};
```

```
typedef struct node
{
polydata data;
struct node *link;
}polynode;
```



Creating a node

- Allocate memory for the node dynamically
- If the memory is allocated successfully
 - set the data part to user defined value
 - set the link part to NULL





Inserting a node

There are 3 cases

- Insertion at the beginning
- Insertion at the end
- Insertion at a given position



Insertion at the beginning

- Create a node

If the list is empty

- make the head pointer point towards the new node;

Else

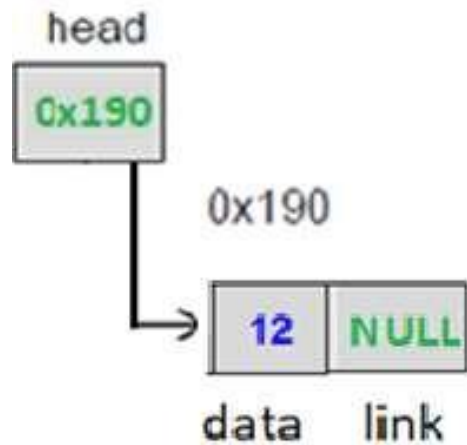
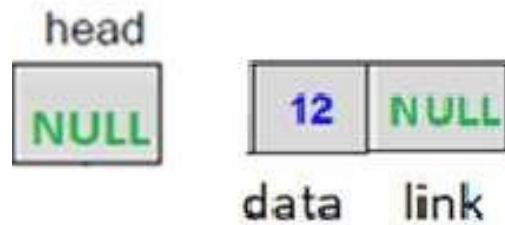
- Make the next pointer of the node point towards the first node of the list
- Make the head pointer point towards this new node

DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List operations



Insertion at the beginning



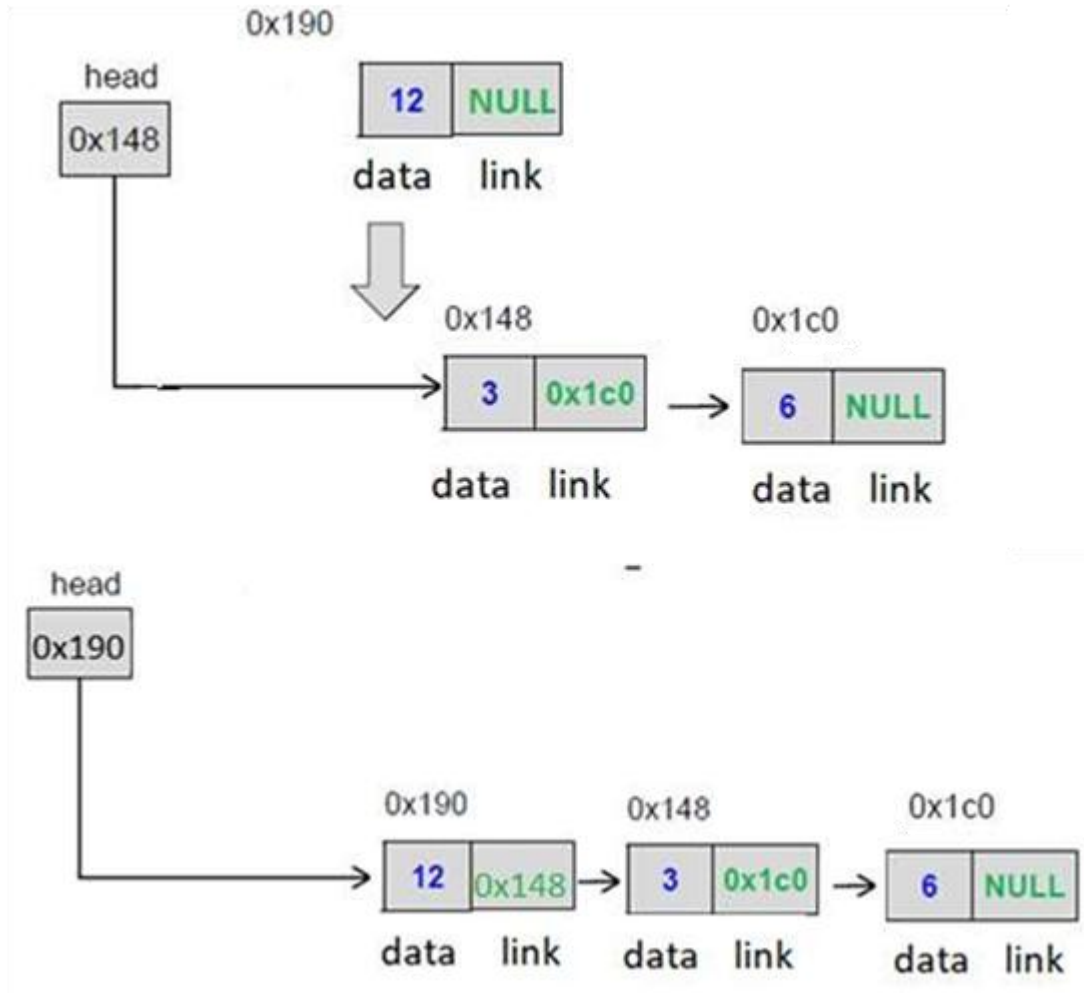
CASE1

DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List operations



Insertion at the beginning



CASE2



Insertion at the end of the

- Create a node

If the list is empty

- make the head pointer point towards the new node;

Else

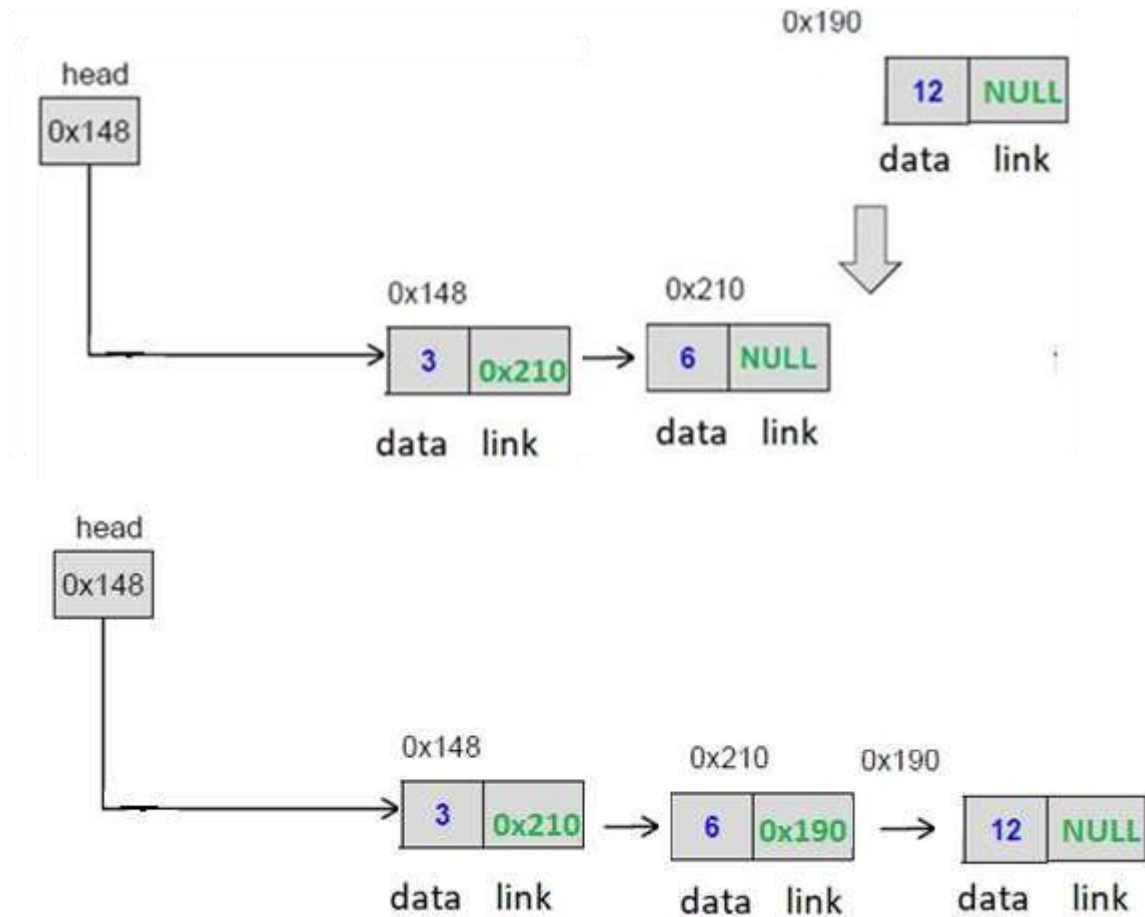
- Traverse the linked list to find out the last node
- Make the link pointer of the last node to point to the new node

DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List operations



Insertion at the end of the



Singly Linked List operations

Insertion at the given position

- Create a node

If the list is empty or if insertion is to be done at first position

- Same steps as insert front

Else

- Traverse the linked list to reach given position

- Keep track of the previous node

If it is an intermediate position

- Change previous node link to point to the newnode

- Newnode to point to the next node

Else

If it is last position

- Same steps as insert at end

Else

Print "Invalid position"

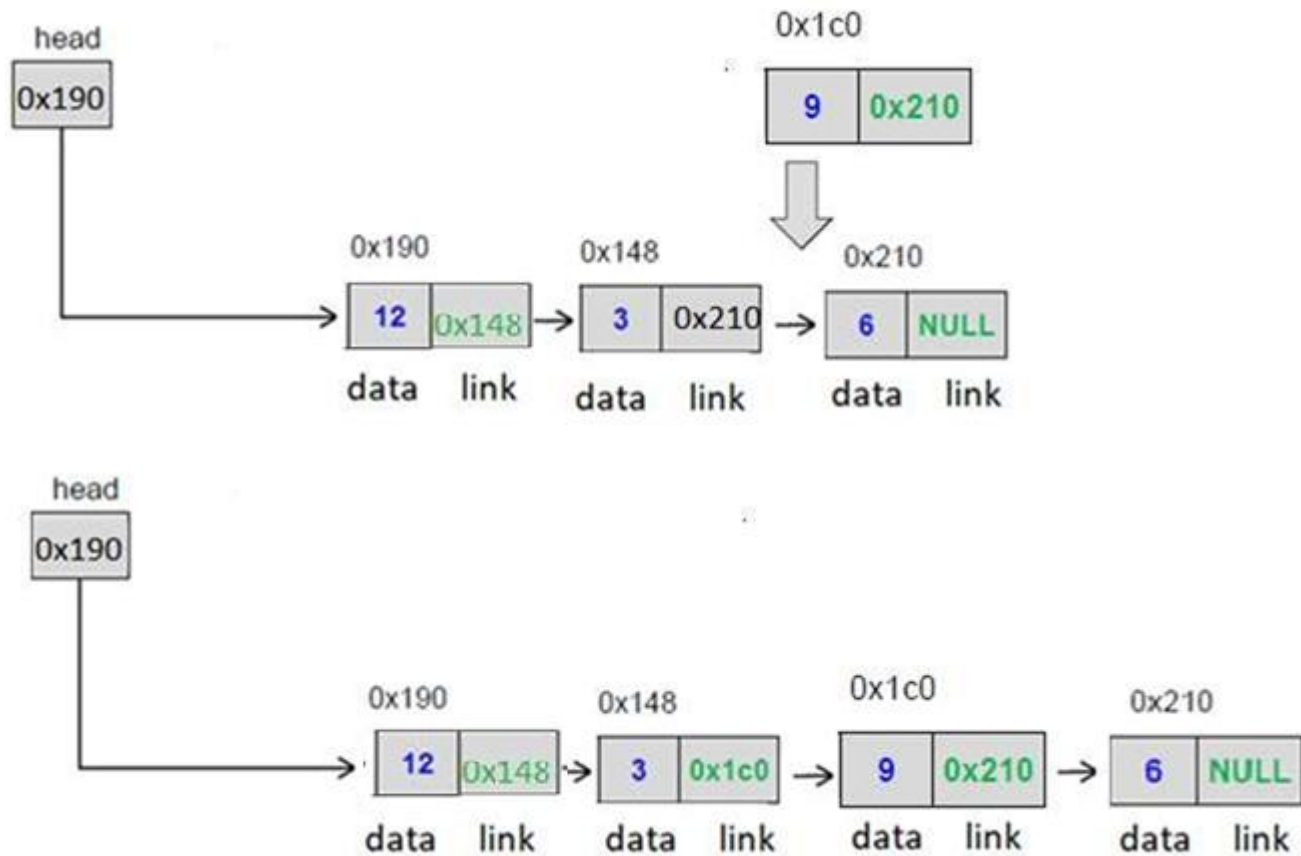


DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List operations



Insertion at the given position





Singly Linked List insert operation

Apply the concepts to implement following operations on a circular singly linked list

- Count number of nodes
- Concatenate two lists
- Sum of all the node values in the list

DATA STRUCTURES AND ITS APPLICATIONS

Multiple-Choice-Questions (MCQ's)



- 1. What is the correct sequence of steps to create a new node and insert it at the beginning of an SLL?**
- a) Allocate memory → Set first node to new node → Set data → Update head
 - b) Set data → Allocate memory → Set new node's next to head → Update head
 - c) Allocate memory → Set data → Set new node's next to head → Update head
 - d) Update head → Set data → Allocate memory

DATA STRUCTURES AND ITS APPLICATIONS

Multiple-Choice-Questions (MCQ's)



- 1. What is the correct sequence of steps to create a new node and insert it at the beginning of an SLL?**
- a) Allocate memory → Set first node to new node → Set data → Update head
 - b) Set data → Allocate memory → Set new node's next to head → Update head
 - c) Allocate memory → Set data → Set new node's next to head → Update head
 - d) Update head → Set data → Allocate memory



2. When inserting a new node at the head of a singly linked list, what is the correct sequence of operations?

- a) `new->next = head; head = new;`
- b) `head = new; new->next = head;`
- c) `new->next = NULL; head = new;`
- d) `new->next = head->next; head = new;`



2. When inserting a new node at the head of a singly linked list, what is the correct sequence of operations?

- a) `new->next = head; head = new;`
- b) `head = new; new->next = head;`
- c) `new->next = NULL; head = new;`
- d) `new->next = head->next; head = new;`



3. To insert a new node after a given node p in an SLL, which step is incorrect?

- a) Allocate memory for the new node.
- b) `new->next = p->next;`
- c) `p->next = new;`
- d) `new->next = p;`



3. To insert a new node after a given node p in an SLL, which step is incorrect?

- a) Allocate memory for the new node.
- b) `new->next = p->next;`
- c) `p->next = new;`
- d) `new->next = p;`



4. What happens when you insert a node at the end of an empty SLL without checking if head == NULL?

- a) Segmentation fault due to NULL->next access.
- b) Node is automatically added as head.
- c) Compiler error occurs.
- d) It works if malloc is used.



4. What happens when you insert a node at the end of an empty SLL without checking if head == NULL?

- a) Segmentation fault due to NULL->next access.
- b) Node is automatically added as head.
- c) Compiler error occurs.
- d) It works if malloc is used.



5. Which of the following correctly inserts a node at position n (1-based indexing) in an SLL?

- a) Traverse $n-1$ nodes and then insert.
- b) Traverse n nodes and then insert.
- c) Always insert at head if $n > 1$.
- d) Insert by swapping data of nodes.



5. Which of the following correctly inserts a node at position n (1-based indexing) in an SLL?

- a) Traverse $n-1$ nodes and then insert.
- b) Traverse n nodes and then insert.
- c) Always insert at head if $n > 1$.
- d) Insert by swapping data of nodes.



6. In a singly linked list, inserting a new node before a given node p (not head) requires:

- a) Directly linking $\text{new} \rightarrow \text{next} = p$ and updating $\text{prev} \rightarrow \text{next} = \text{new}$, where prev is the previous node of p.
- b) Simply $\text{new} \rightarrow \text{next} = p$ without finding prev.
- c) Swapping data between p and the new node (without using prev).
- d) It is not possible to insert before a given node in an SLL.



6. In a singly linked list, inserting a new node before a given node p (not head) requires:

- a) Directly linking $\text{new} \rightarrow \text{next} = p$ and updating $\text{prev} \rightarrow \text{next} = \text{new}$, where prev is the previous node of p.
- b) Simply $\text{new} \rightarrow \text{next} = p$ without finding prev.
- c) Swapping data between p and the new node (without using prev).
- d) It is not possible to insert before a given node in an SLL.

7. If an SLL has n nodes, how many pointer changes are required to insert a new node after the k -th node ($1 \leq k \leq n$)?

- a) 1
- b) 2
- c) k
- d) n

7. If an SLL has n nodes, how many pointer changes are required to insert a new node after the k -th node ($1 \leq k \leq n$)?

a) 1

b) 2

c) k

d) n



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu



PES
UNIVERSITY

DATA STRUCTURES AND ITS APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Singly Linked List

Vandana M L

Department of Computer Science and Engineering

List Data Structure

List

- Dynamic data structure consists of a collection of elements
- Can be implemented in two ways
 - ❑ By contiguous memory allocation : ArrayList
 - ❑ By Linked Allocation : Linked List

List Data Structure: Operations

- Creating a List
- Inserting an element in a list
- Deleting an element from a list
- Searching a list
- Reversing a list
- Concatenating two lists
- Traversing a list

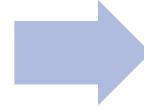
DATA STRUCTURES AND ITS APPLICATIONS

Understanding Array List (Linear List using Arrays)



Placement

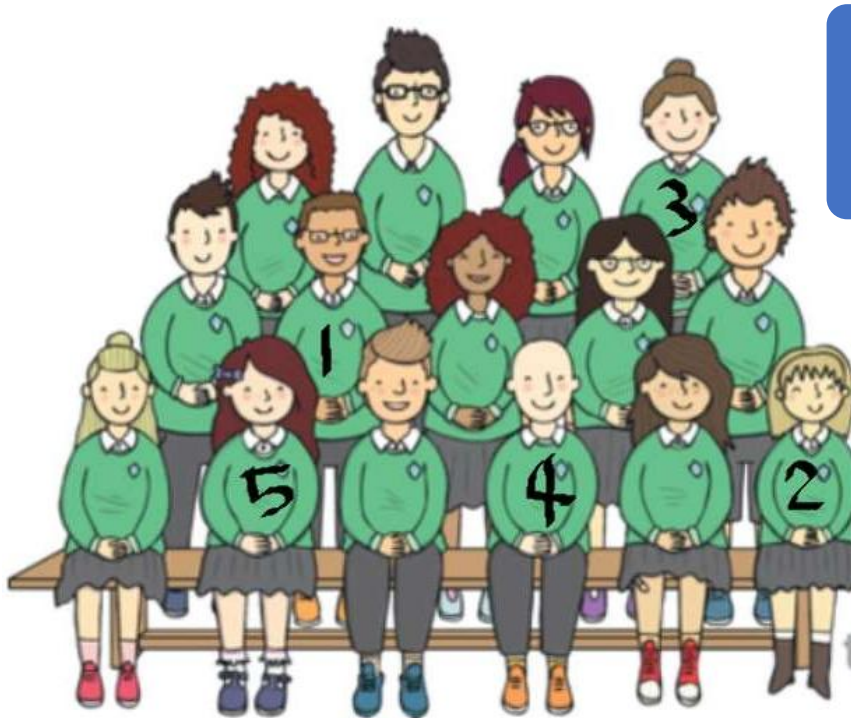
- Sequential



Access

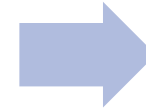
- Sequential

Array List



Placement

- Random



Access

- Sequential

Linked List

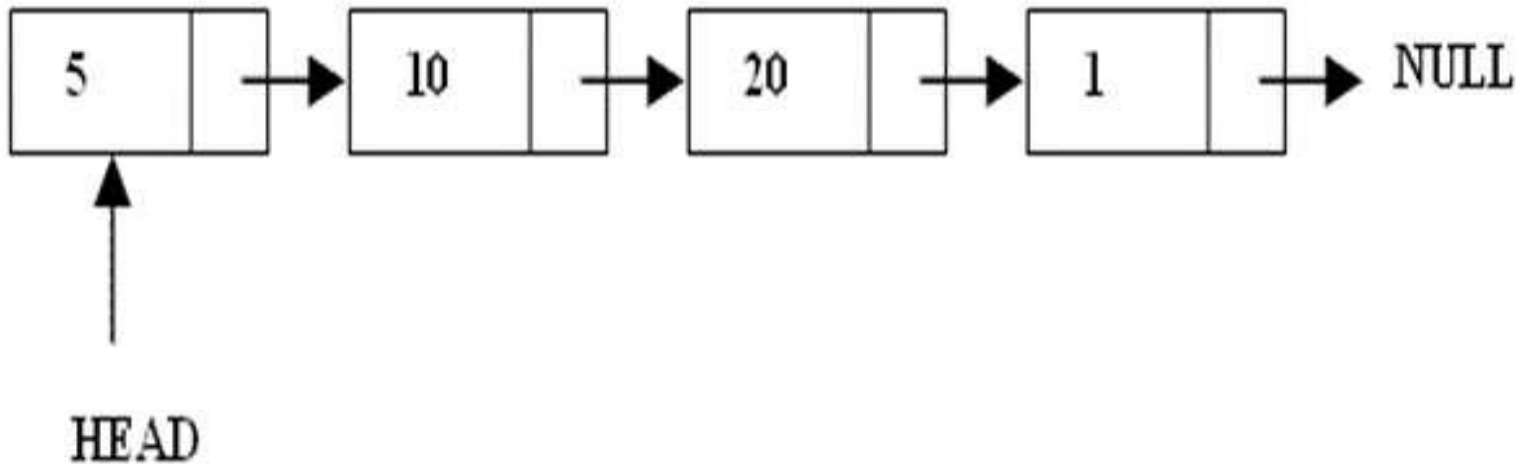
Array List Vs Linked List

ArrayList	Linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient	Insertions and Deletions are efficient
Elements in contiguous memory locations	Elements not in contiguous memory locations
May result in memory wastage if all the allocated space is not used	Since memory is allocated dynamically(as per requirement) there is no wastage of memory.
Sequential and random access is faster	Sequential and random access is slow

- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Multi Linked List

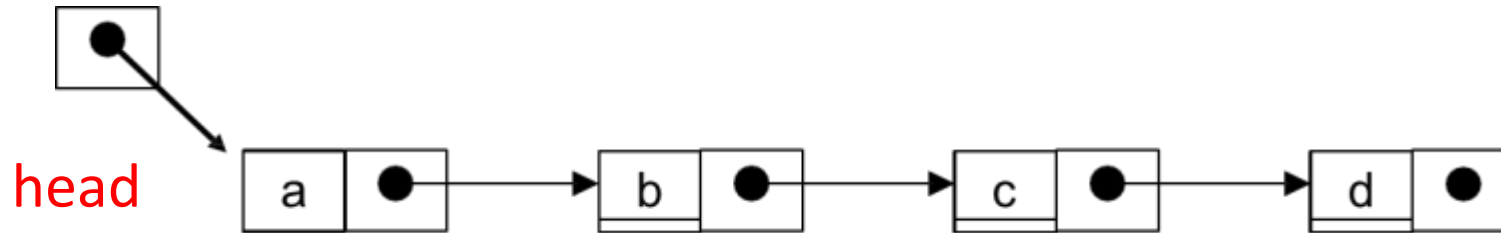
Singly Linked List

- A linked list is a linear data structure.
- Nodes make up linked lists.
- Nodes are structures made up of data and a pointer to another node.
- Usually the pointer is called as link.



- Each node has only one link part
- Each link part contains the address of the next node in the list
- Link part of the last node contains NULL value which signifies the end of the node





- Each node contains a value(data) and a pointer to the next node in the list
- **Head/start** is a pointer which points at the first node in the list

Singly Linked List

Apply the concepts to answer the following questions

- Give structure definition for node of singly linked list used to store employee data (employee no , name, salary ,designation)

1. Which of the following best describes the memory allocation for nodes in an SLL?

- a) Continuous memory blocks are used like arrays.
- b) Nodes are allocated dynamically and may be scattered in memory.
- c) Memory allocation is fixed during compilation.
- d) Memory blocks are always allocated sequentially in heap.

1. Which of the following best describes the memory allocation for nodes in an SLL?

- a) Continuous memory blocks are used like arrays.
- b) Nodes are allocated dynamically and may be scattered in memory.
- c) Memory allocation is fixed during compilation.
- d) Memory blocks are always allocated sequentially in heap.

2. Which of the following is the main drawback of an SLL compared to a doubly linked list (DLL)?

- a) SLL requires more memory per node.
- b) Traversal in reverse direction is not possible.
- c) SLL insertion is slower.
- d) SLL uses more dynamic memory allocation.

2. Which of the following is the main drawback of an SLL compared to a doubly linked list (DLL)?

- a) SLL requires more memory per node.
- b) Traversal in reverse direction is not possible.
- c) SLL insertion is slower.
- d) SLL uses more dynamic memory allocation.

3. In which scenario is it preferable to use an SLL over an array?

- a) When random access is required frequently.
- b) When frequent insertions and deletions occur at unknown positions.
- c) When memory must be allocated statically.
- d) When reverse traversal is required.

3. In which scenario is it preferable to use an SLL over an array?

- a) When random access is required frequently.
- b) When frequent insertions and deletions occur at unknown positions.
- c) When memory must be allocated statically.
- d) When reverse traversal is required.



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List

Vandana M L

Department of Computer Science and Engineering



Deleting a node

There are 3 cases

- Deleting first node
- Deleting last node
- Deleting a node at a given position

Deleting first node

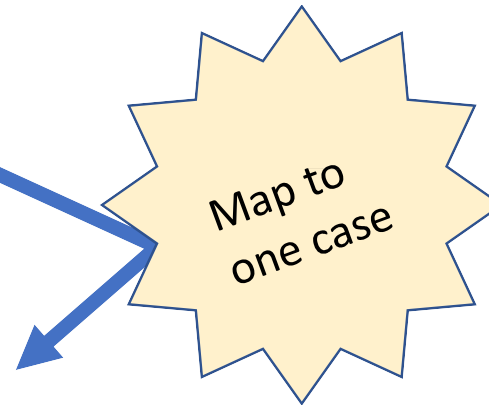
Case 1: Linked list is empty

Case 2: Linked list with a single node

- delete the node
- set head to NULL

Case 3: Linked list has more than one node

- Change head to point to second node
- Delete the first node



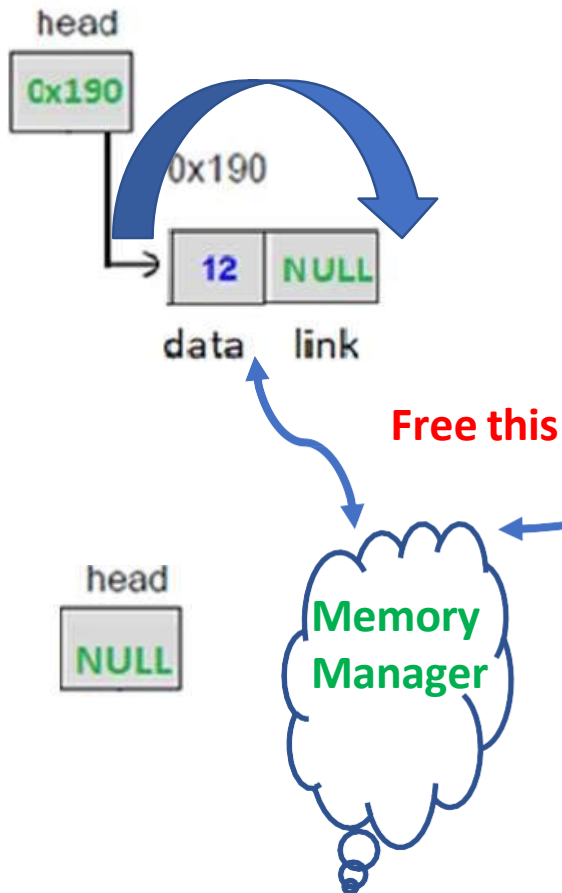
DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List Operations

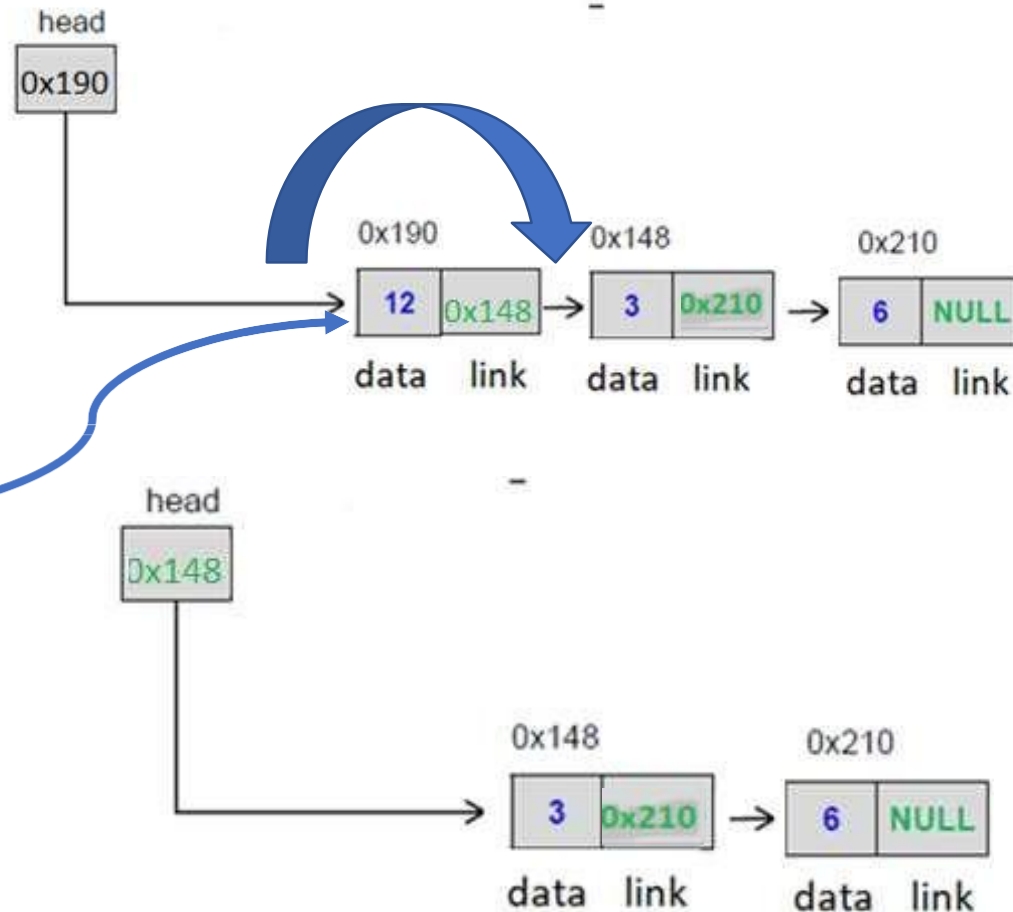


Deleting first node

Only one node in list



More than one node





Deleting last node

Case 1: Linked list is empty

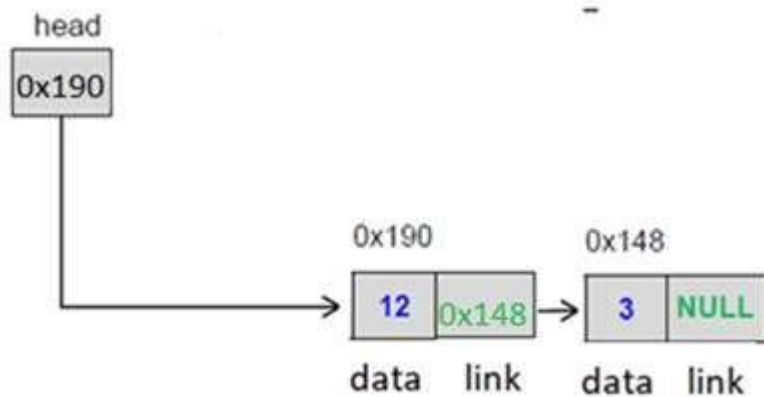
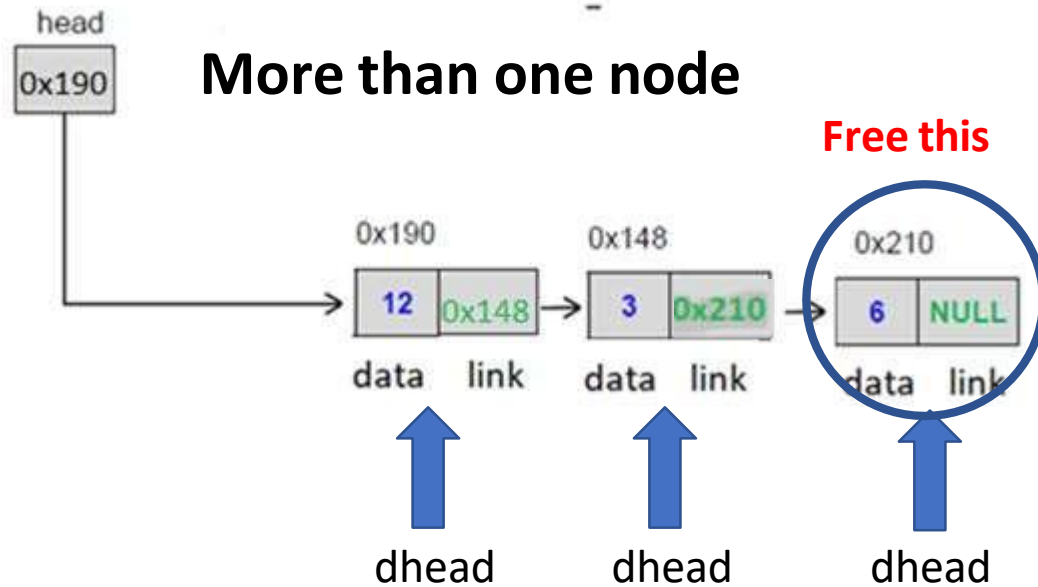
Case 2: Linked list with a single node

- delete the node
- set head to NULL

Case 3: Linked list has more than one node

- Traverse the linked list to point to second last node
- Delete the last node
- Set link field of second last node to NULL

Deleting last node



Deleting node from a given position

If the linked list is not empty

If position is 1

- Delete from the front of the linked list

Else

If position is a valid position

- Traverse linked list to get the desired position
- keep track of previous node
- set previous node link field to link field of current node
- delete the current node

Else

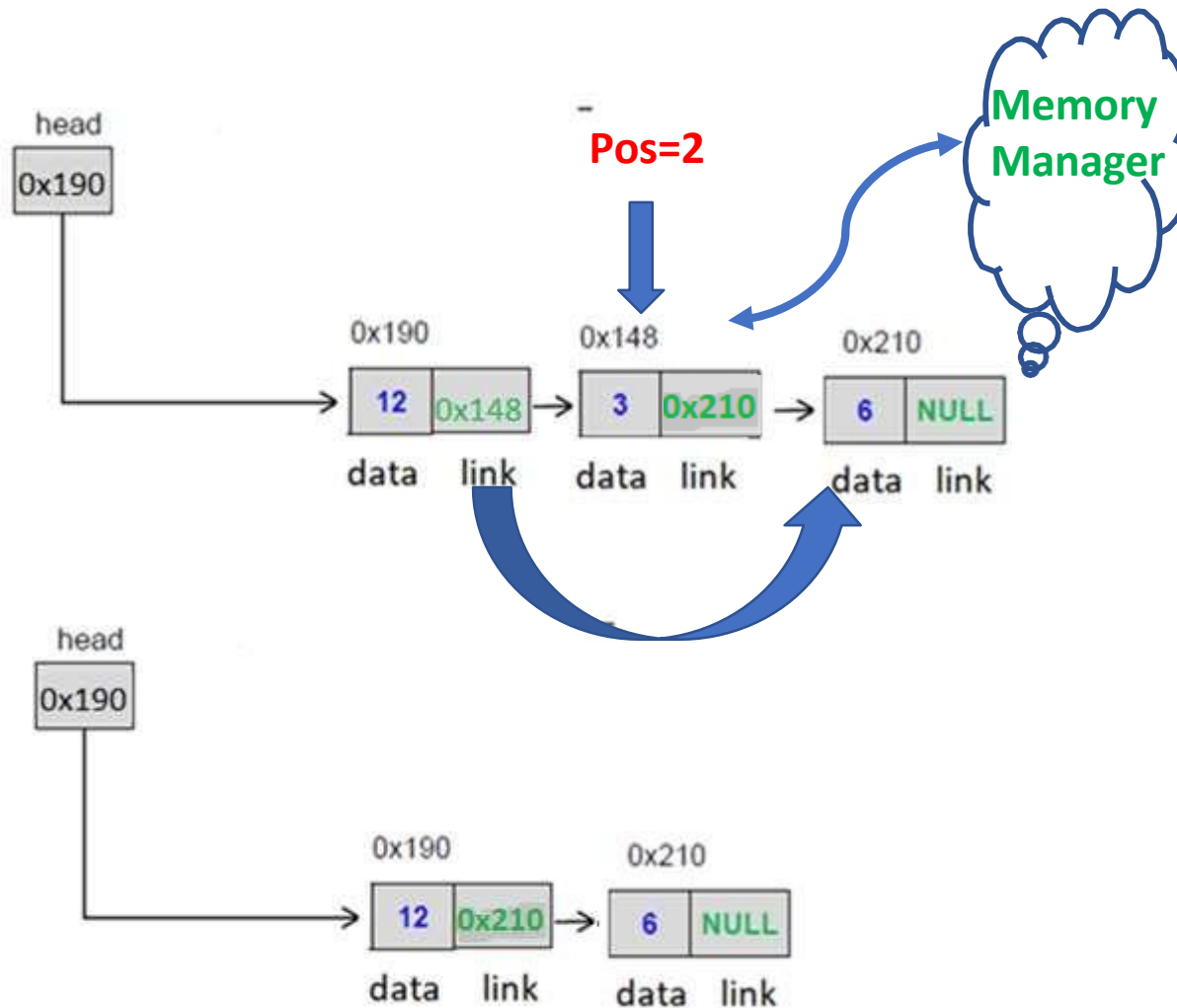
- print invalid position

DATA STRUCTURES AND ITS APPLICATIONS

Singly Linked List Operations



Deleting node from a given position





Singly Linked List delete operation

Apply the concepts to implement following operations for a singly linked list

- Delete a node with given key value
- Delete all alternate nodes
- Delete all the nodes (erase the linked list)



1. Which is the correct sequence of operations to delete the head node of an SLL?

- a) `head = head->next; free(head);`
- b) `temp = head; head = head->next; free(temp);`
- c) `free(head); head = head->next;`
- d) `head->next = head; free(head);`

1. Which is the correct sequence of operations to delete the head node of an SLL?

- a) `head = head->next; free(head);`
- b) `temp = head; head = head->next; free(temp);`
- c) `free(head); head = head->next;`
- d) `head->next = head; free(head);`



2. To delete the node after a given node p, which sequence is correct?

- a) `p->next = p->next->next; free(p);`
- b) `temp = p->next; p->next = temp->next; free(temp);`
- c) `temp = p; p = p->next; free(temp);`
- d) `free(p->next); p->next = p->next->next;`



2. To delete the node after a given node p, which sequence is correct?

- a) `p->next = p->next->next; free(p);`
- b) `temp = p->next; p->next = temp->next; free(temp);`
- c) `temp = p; p = p->next; free(temp);`
- d) `free(p->next); p->next = p->next->next;`



3. When deleting the last node in an SLL (without tail pointer), what is required?

- a) Traverse to the second-last node, set its next to NULL, and free the last node.
- b) Set head = NULL directly.
- c) Free the last node and leave the second-last node unchanged.
- d) No traversal is required.



3. When deleting the last node in an SLL (without tail pointer), what is required?

- a) Traverse to the second-last node, set its next to NULL, and free the last node.
- b) Set head = NULL directly.
- c) Free the last node and leave the second-last node unchanged.
- d) No traversal is required.



4. What happens if you attempt to delete from an empty SLL?

- a) The program executes without issue.
- b) A segmentation fault occurs due to NULL pointer access.
- c) The head pointer is automatically initialized.
- d) A new node is created.



4. What happens if you attempt to delete from an empty SLL?

- a) The program executes without issue.
- b) A segmentation fault occurs due to NULL pointer access.
- c) The head pointer is automatically initialized.
- d) A new node is created.



5. To delete a node at position n in an SLL (where $n > 1$), which of the following is correct?

- a) Traverse n nodes and free the node at n .
- b) Traverse $n-1$ nodes to find the previous node, then adjust its next and free the target node.
- c) Swap data of the n th and $(n-1)$ th nodes and free the $(n-1)$ th node.
- d) Always delete from the head if $n > 1$.



5. To delete a node at position n in an SLL (where $n > 1$), which of the following is correct?

- a) Traverse n nodes and free the node at n .
- b) Traverse $n-1$ nodes to find the previous node, then adjust its next and free the target node.
- c) Swap data of the n th and $(n-1)$ th nodes and free the $(n-1)$ th node.
- d) Always delete from the head if $n > 1$.



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu



DATA STRUCTURES AND APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List

Vandana M L

Department of Computer Science and Engineering



A doubly linked list contains **three** fields:

- Data
- link to the next node
- link to the previous node.

DATA STRUCTURES AND APPLICATIONS

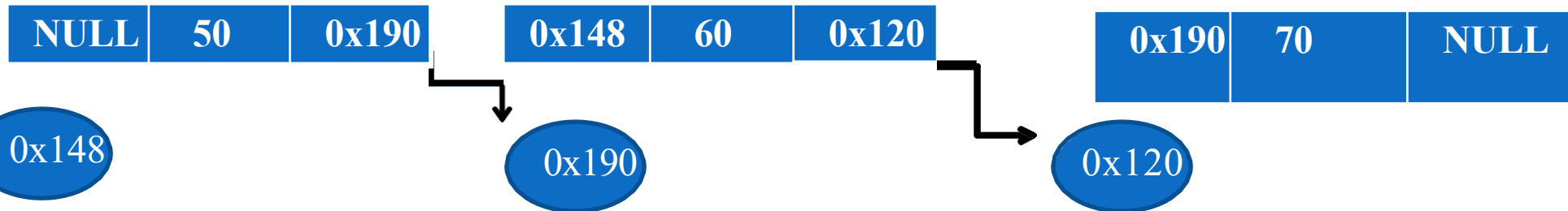
Doubly Linked List :Node Structure



A

B

C





➤ Advantages:

- Can be traversed in either direction (may be essential for some programs)
- Some operations, such as deletion and inserting before a node, become easier

➤ Disadvantages:

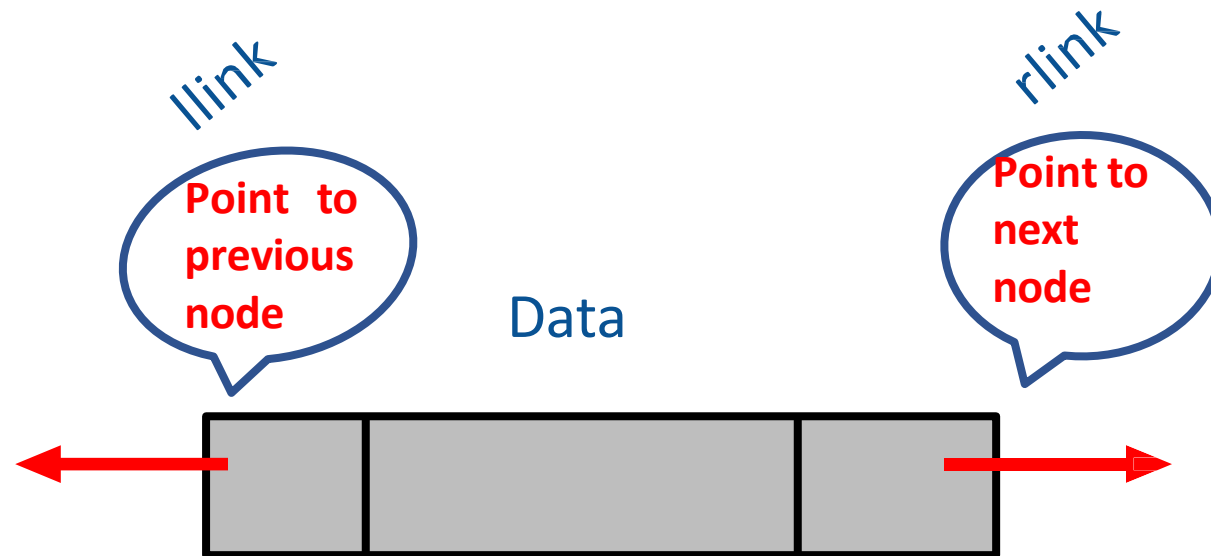
- Requires more space
- List manipulations are slower (because more links must be changed)
- Greater chance of having bugs (because more links must be manipulated)

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Node definition

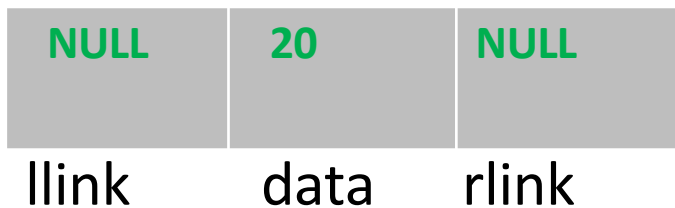


```
struct node
{
    int data;
    node*llink;
    node*rlink;
};
```



Creating a node

- Allocate memory for the node dynamically
- If the memory is allocated successfully
set the data part to user defined value
set the llink (address of previous node) and rlink (address of next node) part to NULL





Inserting a node

There are 3 cases

- Insertion at the beginning
- Insertion at the end
- Insertion at a given position



Insertion at the beginning

What all will change

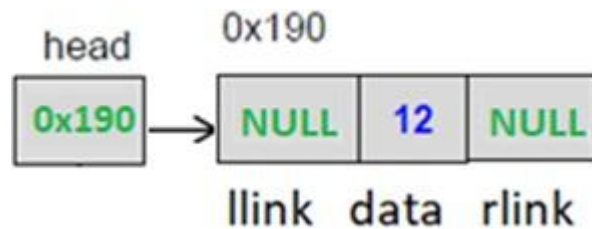
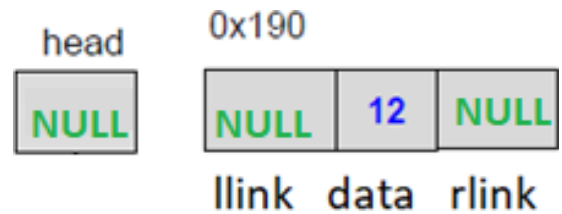
If the linked list empty(case 1)

Head/Start pointer

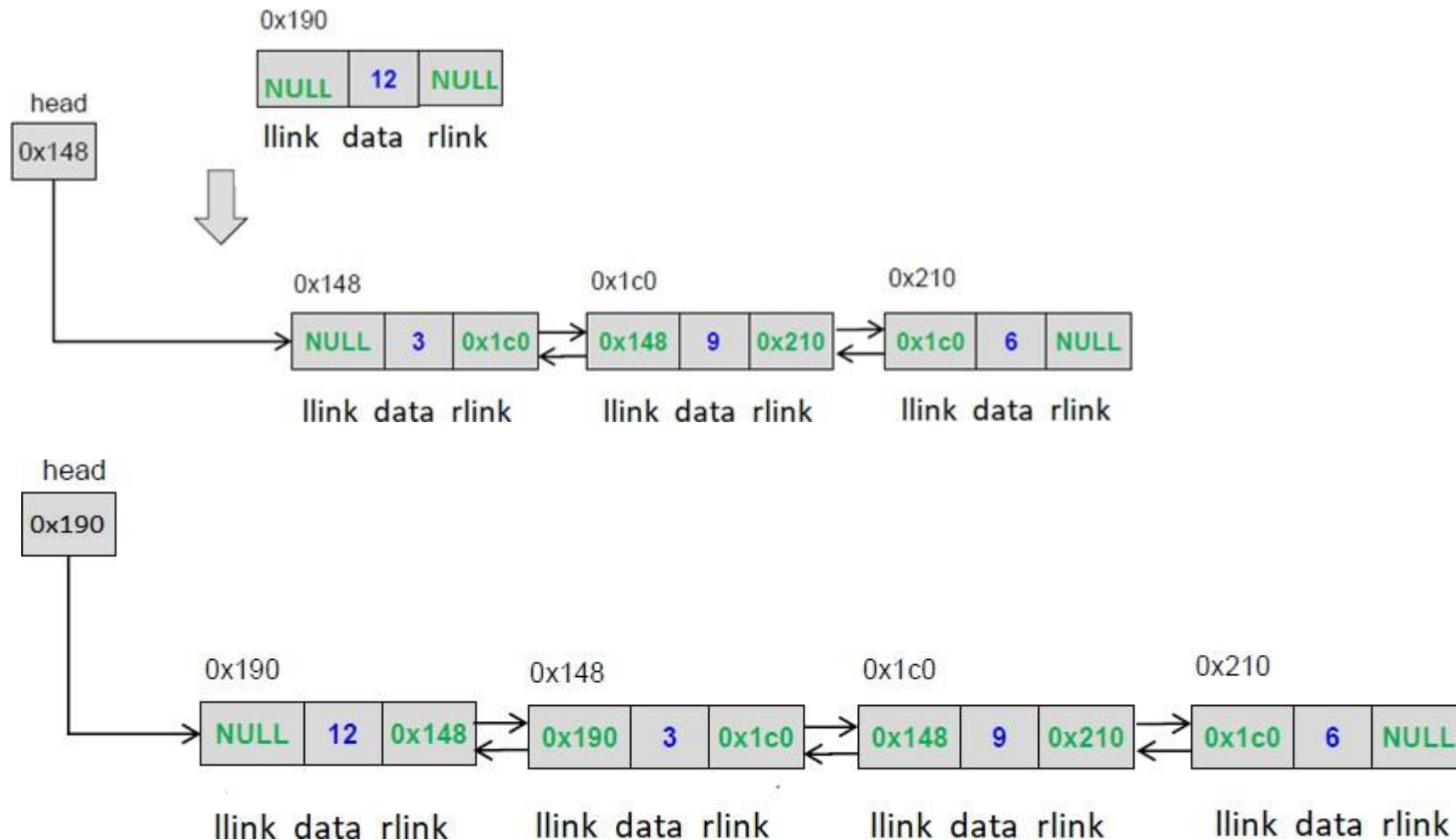
else (case2)

- Head/Start pointer
- New front's llink and rlink
- Old front's llink

Insertion at the beginning (Case1)



Insertion at the beginning(Case 2)





Insertion at the end

What all will change

If the linked list empty(same as case 1 of insert at front)

Head/Start pointer(case 2)

else

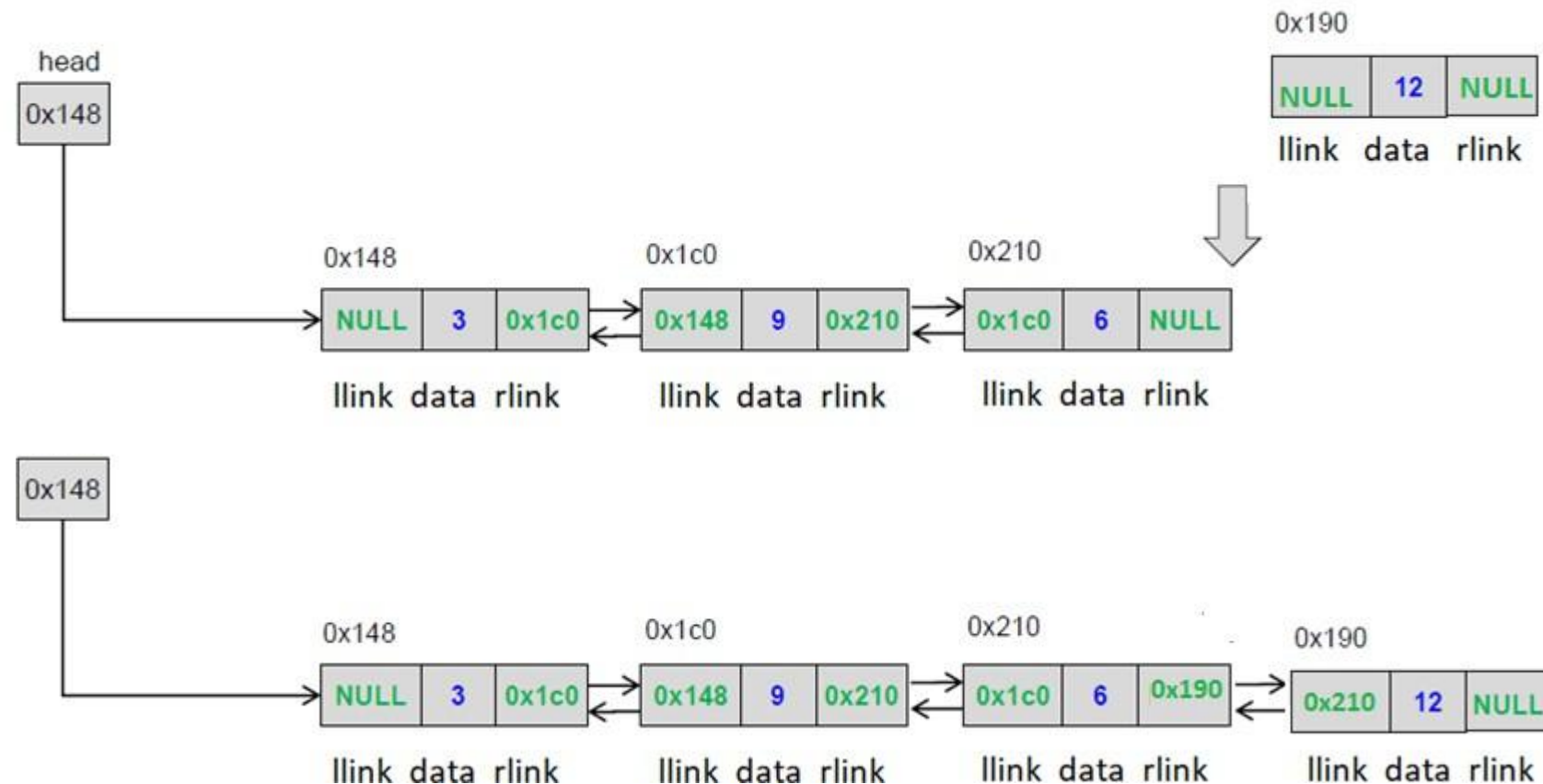
- Last node's rlink
- New node's llink

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Implementation



Insertion at the end





Insertion at the given position

- Create a node

If the list is empty

- make the start pointer point towards the new node;

Else

- Traverse the linked list to reach given position
- Keep track of the previous node

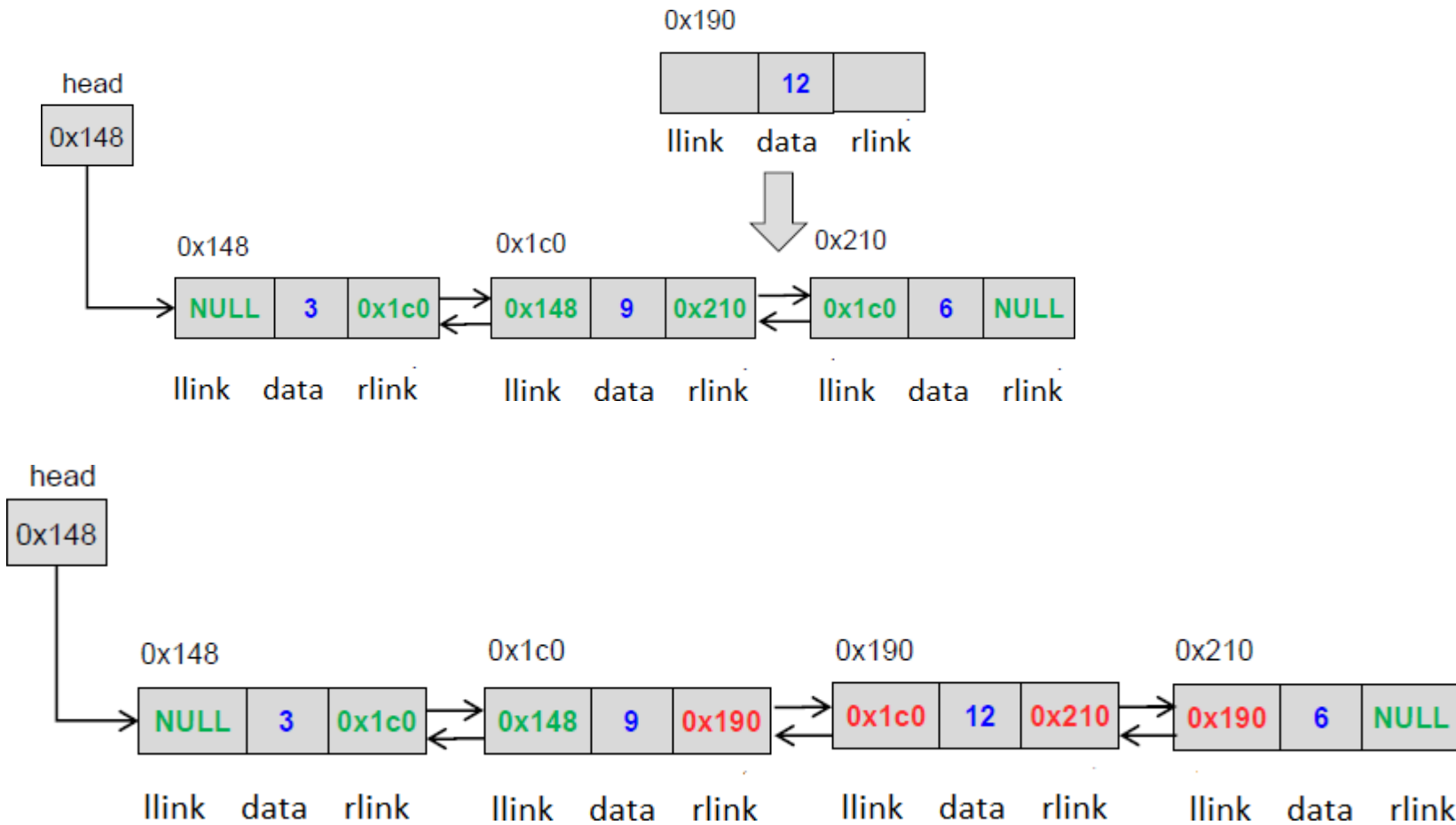
If it is an intermediate position

- Change previous node rlink to point to the newnode
- Newnode's llink to point to previous node and rlink to point to the next node
- Next node llink to point to the newnode

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Implementation

Insertion at the given position





Deleting a node

There are 3 cases

- Deleting first node
- Deleting last node
- Deleting a node at a given position



Deleting a node

There are 3 cases

- Deleting first node
- Deleting last node
- Deleting a node at a given position

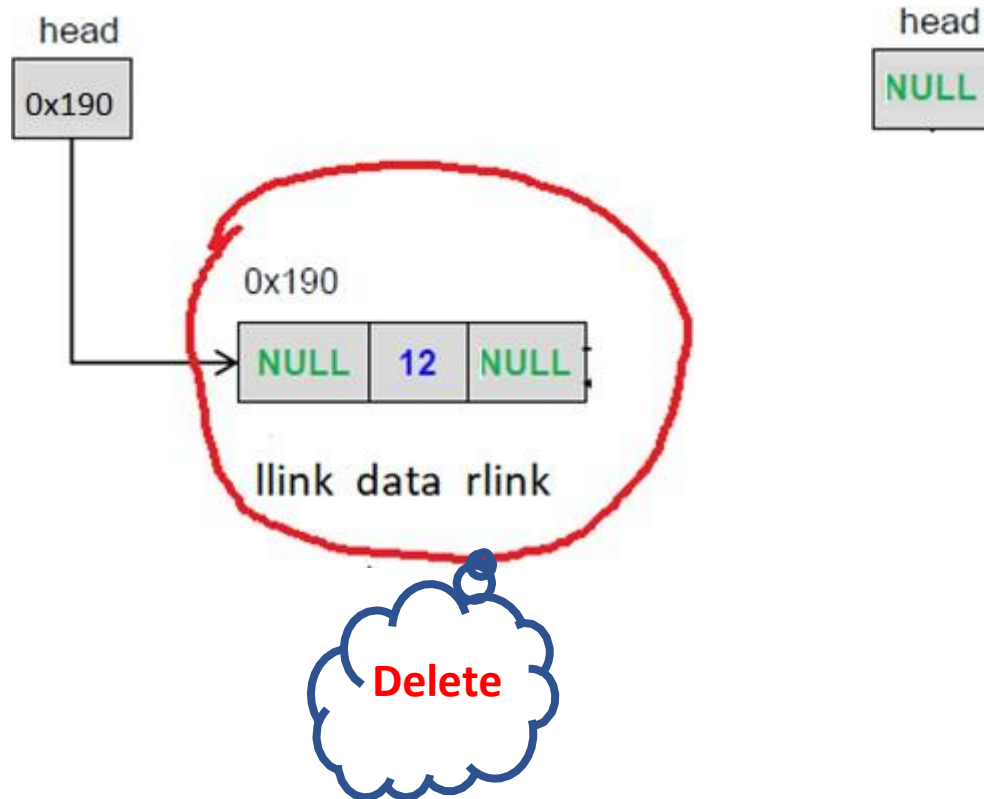
Deleting first node

What will change??

- Case I : Empty Linked List
- Case II : Linked list with a single node
 - first node gets freed up
 - head points to NULL
- Case III : Linked List with more than one node
 - Second node llink gets changed to NULL
 - first node gets freed off
 - head points to second node

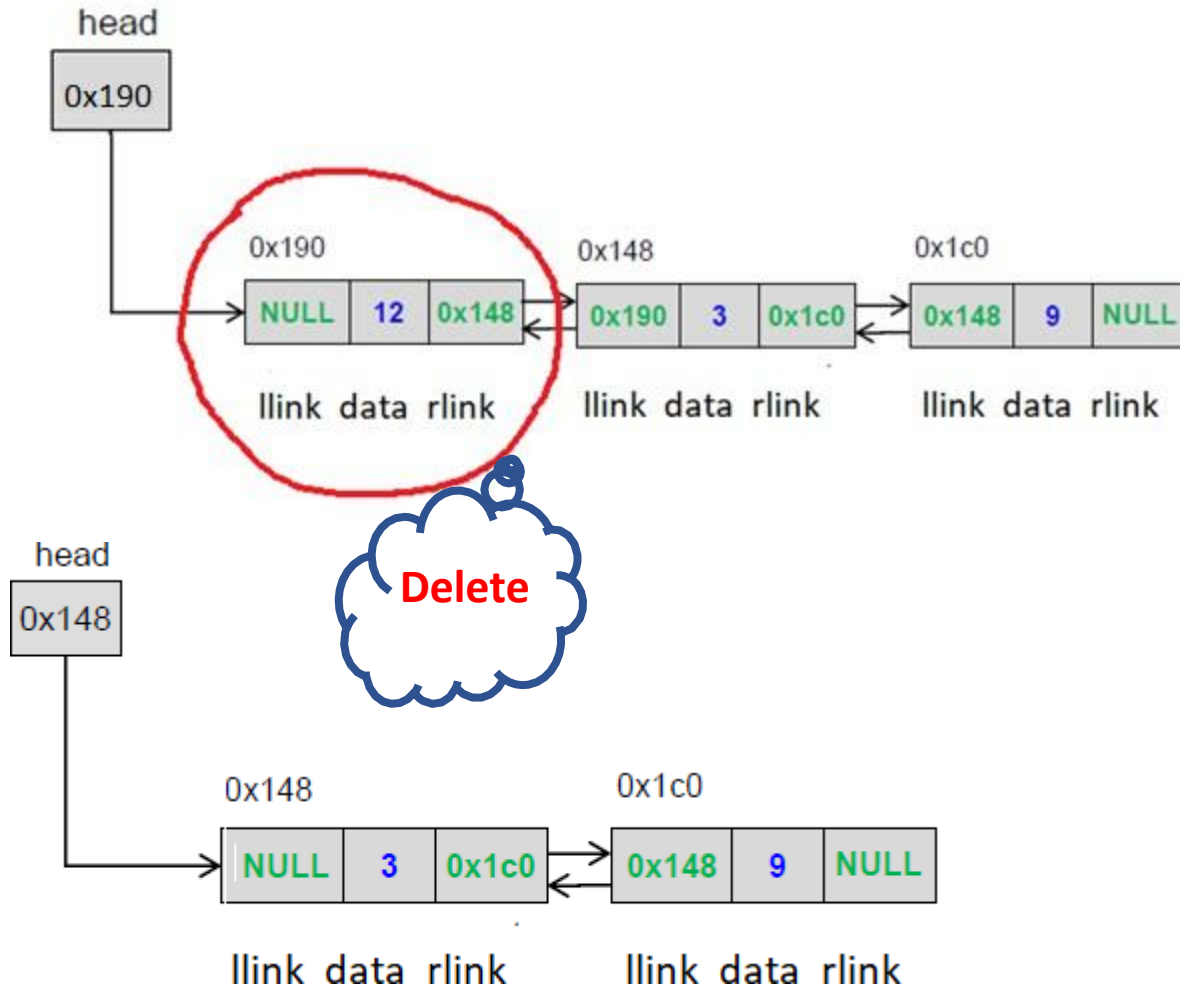
Deleting first node

- Case II : Linked list with a single node



Deleting first node

- Case III : Linked List with more than one node





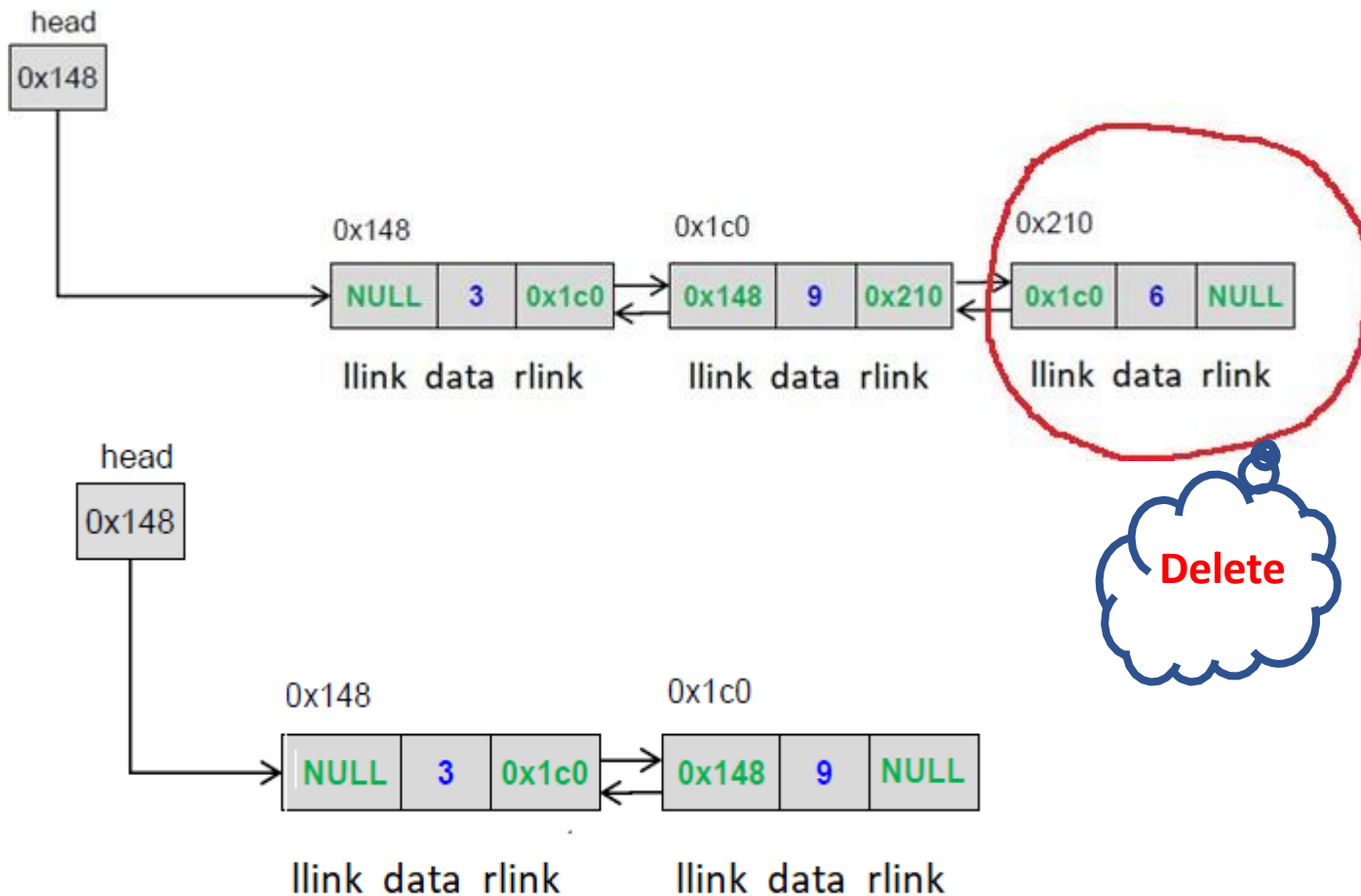
Deleting last node

What will change??

- Case I : Empty Linked List
- Case II : Linked list with a single node
 - first node gets freed up
 - head points to NULL
- Case III : Linked List with more than one node
 - Second last node rlink point to NULL
 - last node gets freed up

Deleting last node

- Case II : Linked List with more than one node



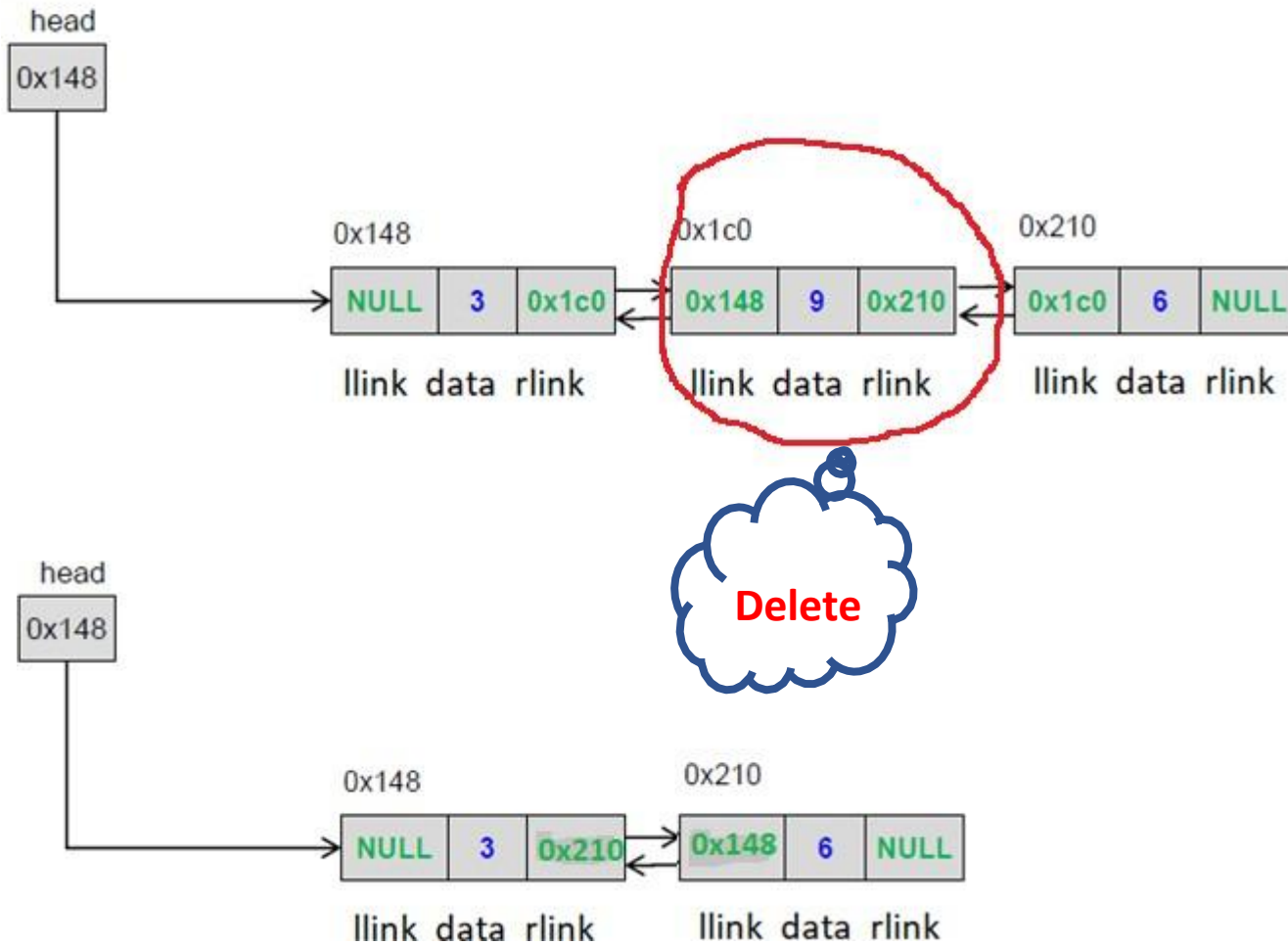
DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Implementation



Deleting a node at intermediate position

- Case II : Linked List with more than one node





Doubly Linked List insert operation

Apply the concepts to implement following operations for a Doubly linked list

- reverse a doubly linked list
- Find the node pairs with a given sum in a doubly linked list
- Insert a node after a node with a given value
- Remove duplicate nodes from a doubly linked list



1. Which of the following is a disadvantage of a DLL compared to an SLL?

- a) Faster traversal in both directions.
- b) Requires extra memory for storing the prev pointer in each node.
- c) Easier insertion and deletion from the middle.
- d) Can handle reverse traversal efficiently.

1. Which of the following is a disadvantage of a DLL compared to an SLL?

- a) Faster traversal in both directions.
- b) Requires extra memory for storing the prev pointer in each node.
- c) Easier insertion and deletion from the middle.
- d) Can handle reverse traversal efficiently.



2. Which sequence correctly inserts a new node newNode at the head of a DLL?

- a) `newNode->next = head; head = newNode;`
- b) `newNode->next = head; head->prev = newNode; head = newNode;`
- c) `head->next = newNode; newNode->prev = head; head = newNode;`
- d) `newNode->prev = NULL; head = newNode;`



2. Which sequence correctly inserts a new node newNode at the head of a DLL?

a) `newNode->next = head; head = newNode;`

b) `newNode->next = head; head->prev = newNode; head = newNode;`

c) `head->next = newNode; newNode->prev = head; head = newNode;`

d) `newNode->prev = NULL; head = newNode;`



3. To insert at the end of a DLL, which step is incorrect?

- a) Traverse to the last node.
- b) Set `last->next = newNode`.
- c) Set `newNode->prev = last`.
- d) Set `newNode->next = head`.



3. To insert at the end of a DLL, which step is incorrect?

- a) Traverse to the last node.
- b) Set `last->next = newNode`.
- c) Set `newNode->prev = last`.
- d) Set `newNode->next = head`.



4. To insert newNode after a node p in a DLL, which of the following steps is essential but often missed?

- a) `newNode->next = p->next;`
- b) `p->next = newNode;`
- c) `p->next->prev = newNode;` (if `p->next` is not NULL)
- d) All of the above.



4. To insert newNode after a node p in a DLL, which of the following steps is essential but often missed?

- a) `newNode->next = p->next;`
- b) `p->next = newNode;`
- c) `p->next->prev = newNode;` (if `p->next` is not NULL)
- d) All of the above.

5. When deleting a node target (not head/tail) in a DLL, which of the following is correct?

- a) `target->prev->next = target->next; target->next->prev = target->prev; free(target);`
- b) `target->prev = target->next; target->next = target->prev; free(target);`
- c) `target->next->prev = NULL; free(target);`
- d) `target->prev = NULL; free(target);`

5. When deleting a node target (not head/tail) in a DLL, which of the following is correct?

a) `target->prev->next = target->next; target->next->prev = target->prev; free(target);`

b) `target->prev = target->next; target->next = target->prev; free(target);`

c) `target->next->prev = NULL; free(target);`

d) `target->prev = NULL; free(target);`



Vandana M L

Department of Computer Science and Engineering

vandanamd@pes.edu



DATA STRUCTURES AND APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List

Vandana M L

Department of Computer Science and Engineering



A doubly linked list contains **three** fields:

- Data
- link to the next node
- link to the previous node.

DATA STRUCTURES AND APPLICATIONS

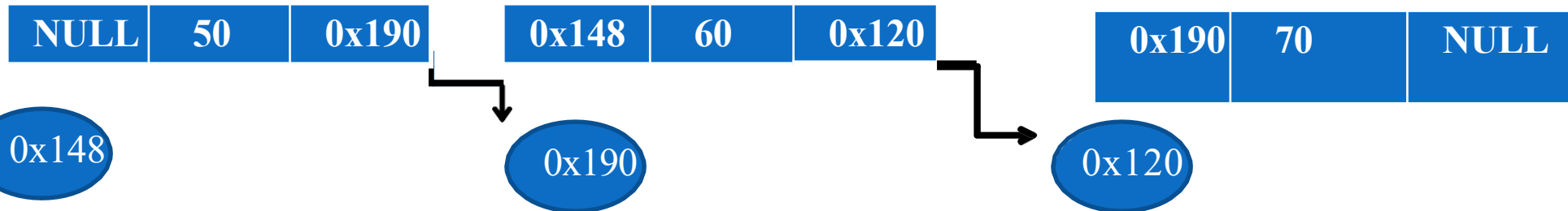
Doubly Linked List :Node Structure



A

B

C





➤ Advantages:

- Can be traversed in either direction (may be essential for some programs)
- Some operations, such as deletion and inserting before a node, become easier

➤ Disadvantages:

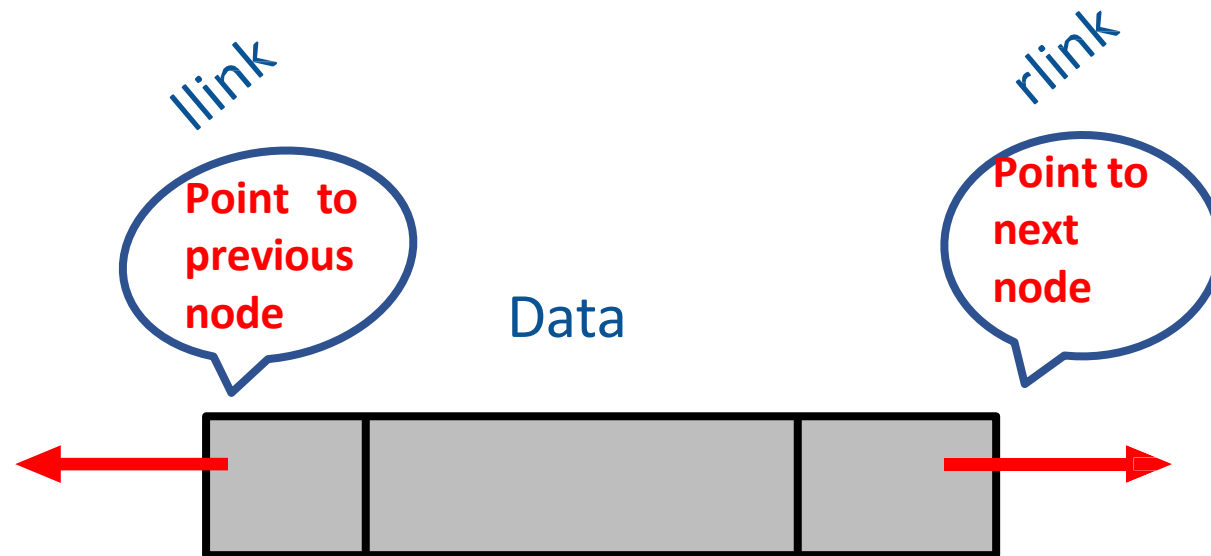
- Requires more space
- List manipulations are slower (because more links must be changed)
- Greater chance of having bugs (because more links must be manipulated)

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Node definition

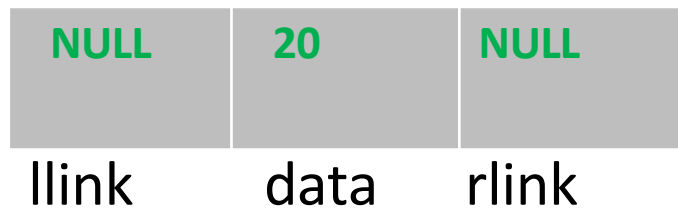


```
struct node
{
    int data;
    node*llink;
    node*rlink;
};
```



Creating a node

- Allocate memory for the node dynamically
- If the memory is allocated successfully
set the data part to user defined value
set the llink (address of previous node) and rlink (address of next node) part to NULL





Inserting a node

There are 3 cases

- Insertion at the beginning
- Insertion at the end
- Insertion at a given position



Insertion at the beginning

What all will change

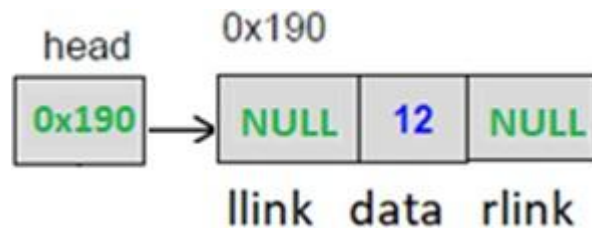
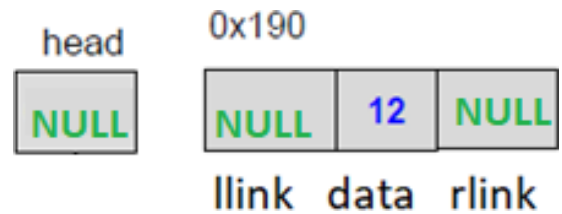
If the linked list empty(case 1)

Head/Start pointer

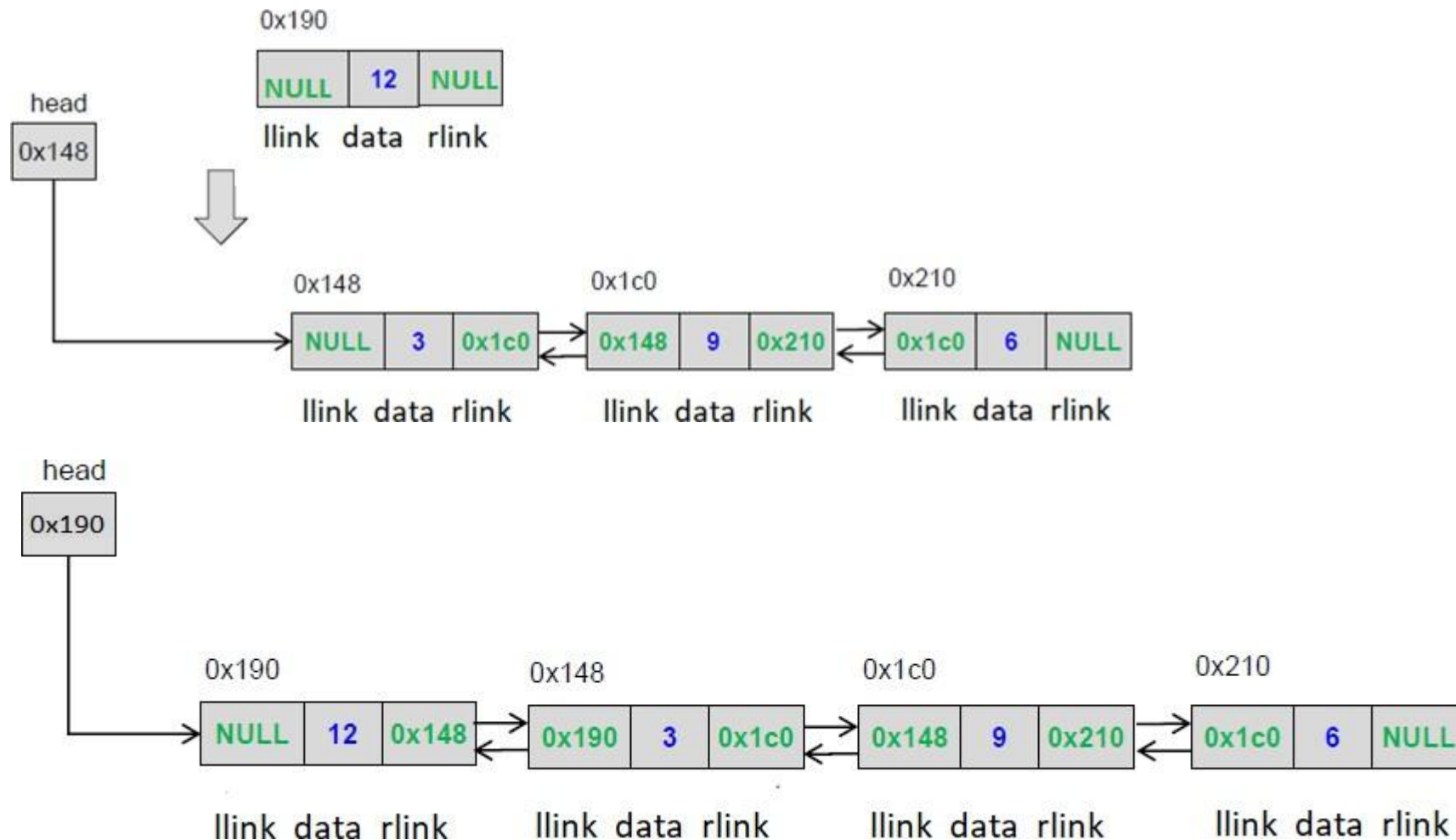
else (case2)

- Head/Start pointer
- New front's llink and rlink
- Old front's llink

Insertion at the beginning (Case1)



Insertion at the beginning(Case 2)





Insertion at the end

What all will change

If the linked list empty(same as case 1 of insert at front)

Head/Start pointer(case 2)

else

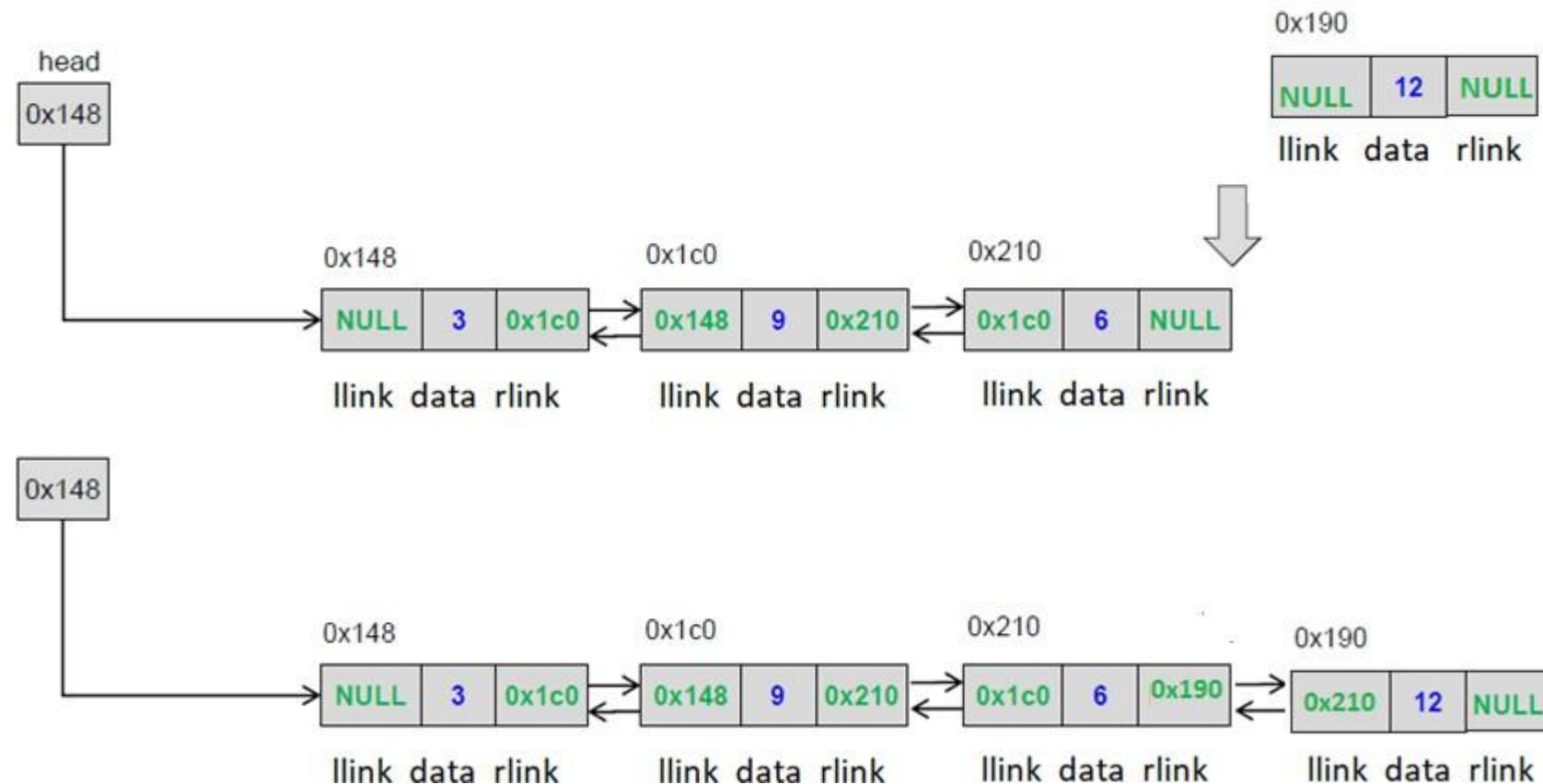
- Last node's rlink
- New node's llink

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Implementation



Insertion at the end





Insertion at the given position

- Create a node

If the list is empty

- make the start pointer point towards the new node;

Else

- Traverse the linked list to reach given position
- Keep track of the previous node

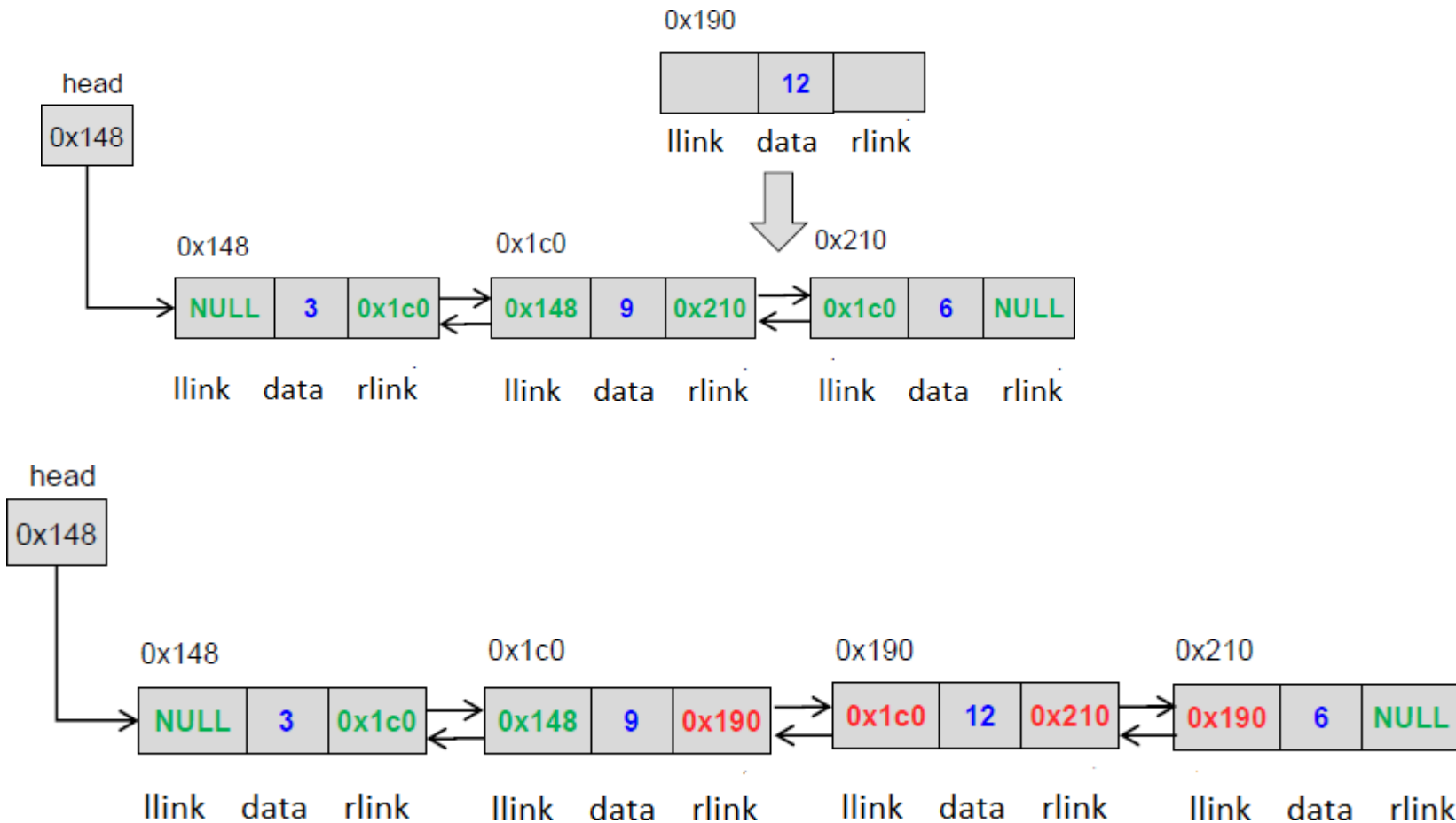
If it is an intermediate position

- Change previous node rlink to point to the newnode
- Newnode's llink to point to previous node and rlink to point to the next node
- Next node llink to point to the newnode

DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Implementation

Insertion at the given position





Deleting a node

There are 3 cases

- Deleting first node
- Deleting last node
- Deleting a node at a given position



Deleting a node

There are 3 cases

- Deleting first node
- Deleting last node
- Deleting a node at a given position

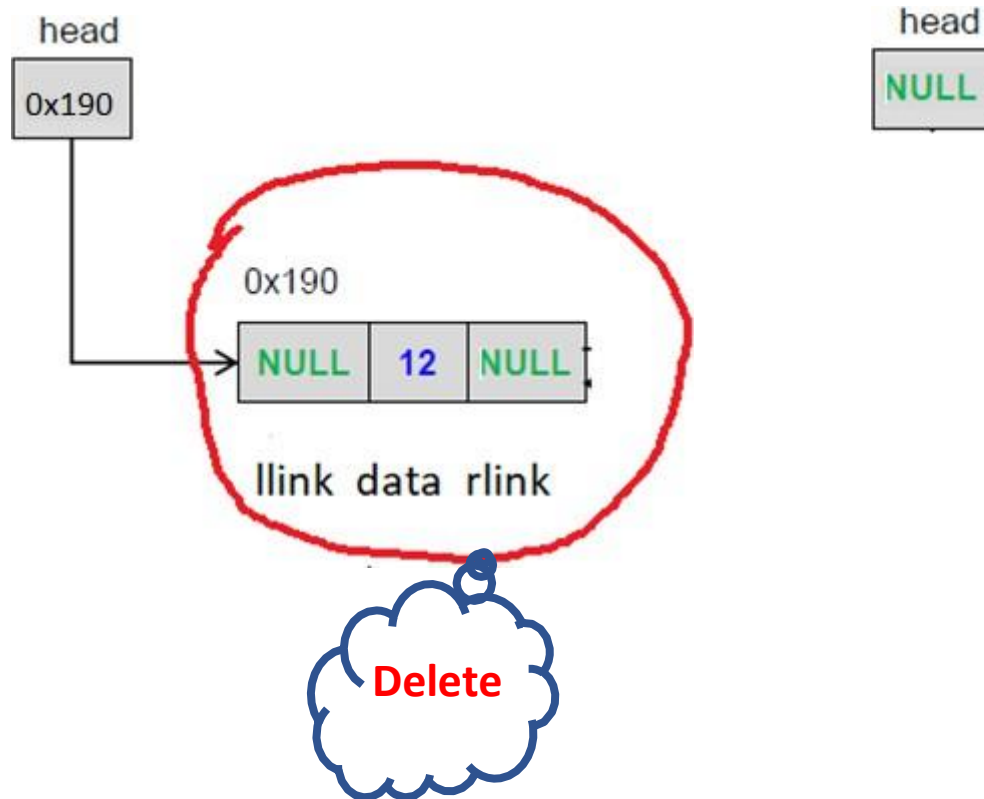
Deleting first node

What will change??

- Case I : Empty Linked List
- Case II : Linked list with a single node
 - first node gets freed up
 - head points to NULL
- Case III : Linked List with more than one node
 - Second node llink gets changed to NULL
 - first node gets freed off
 - head points to second node

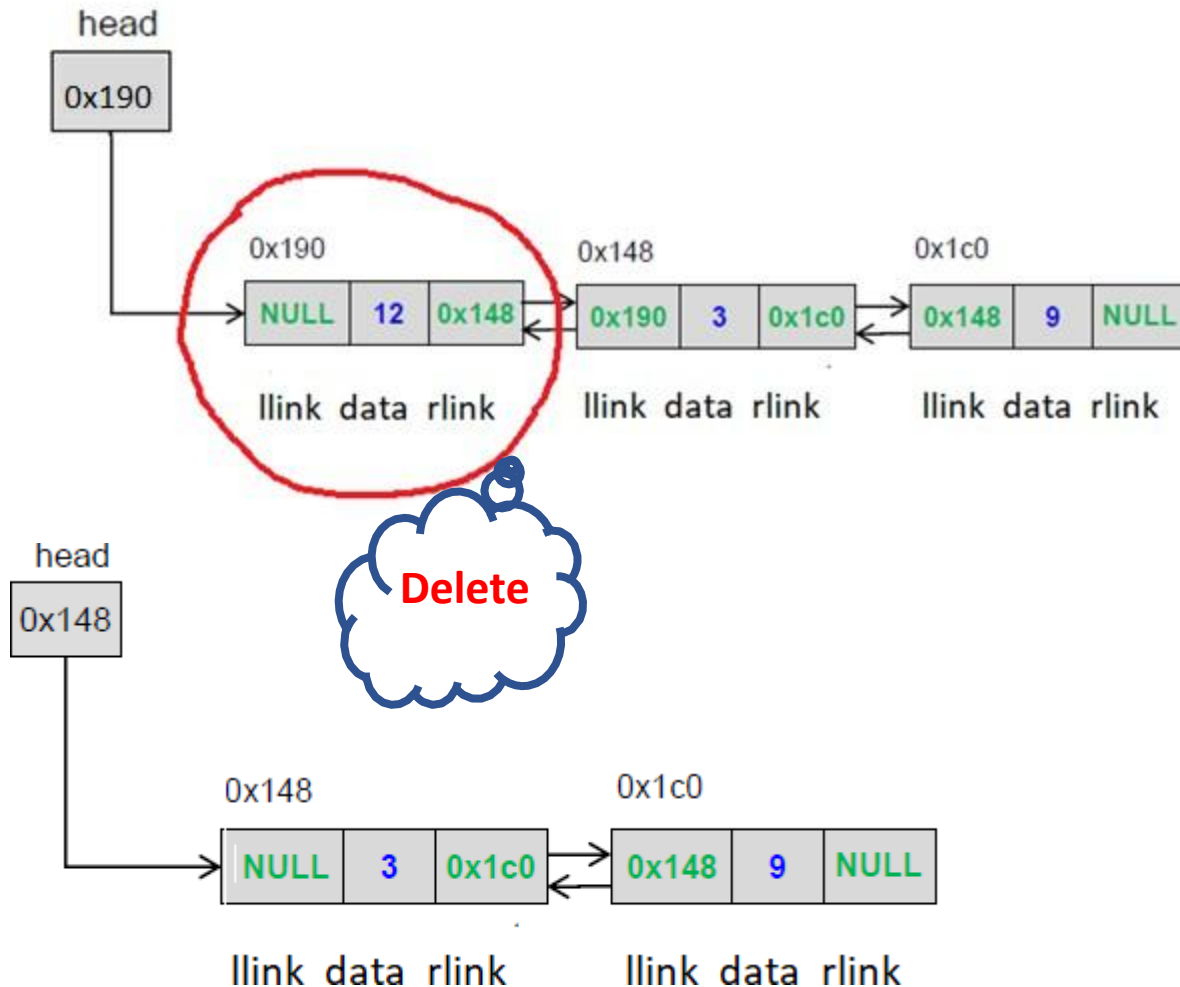
Deleting first node

- Case II : Linked list with a single node



Deleting first node

- Case III : Linked List with more than one node





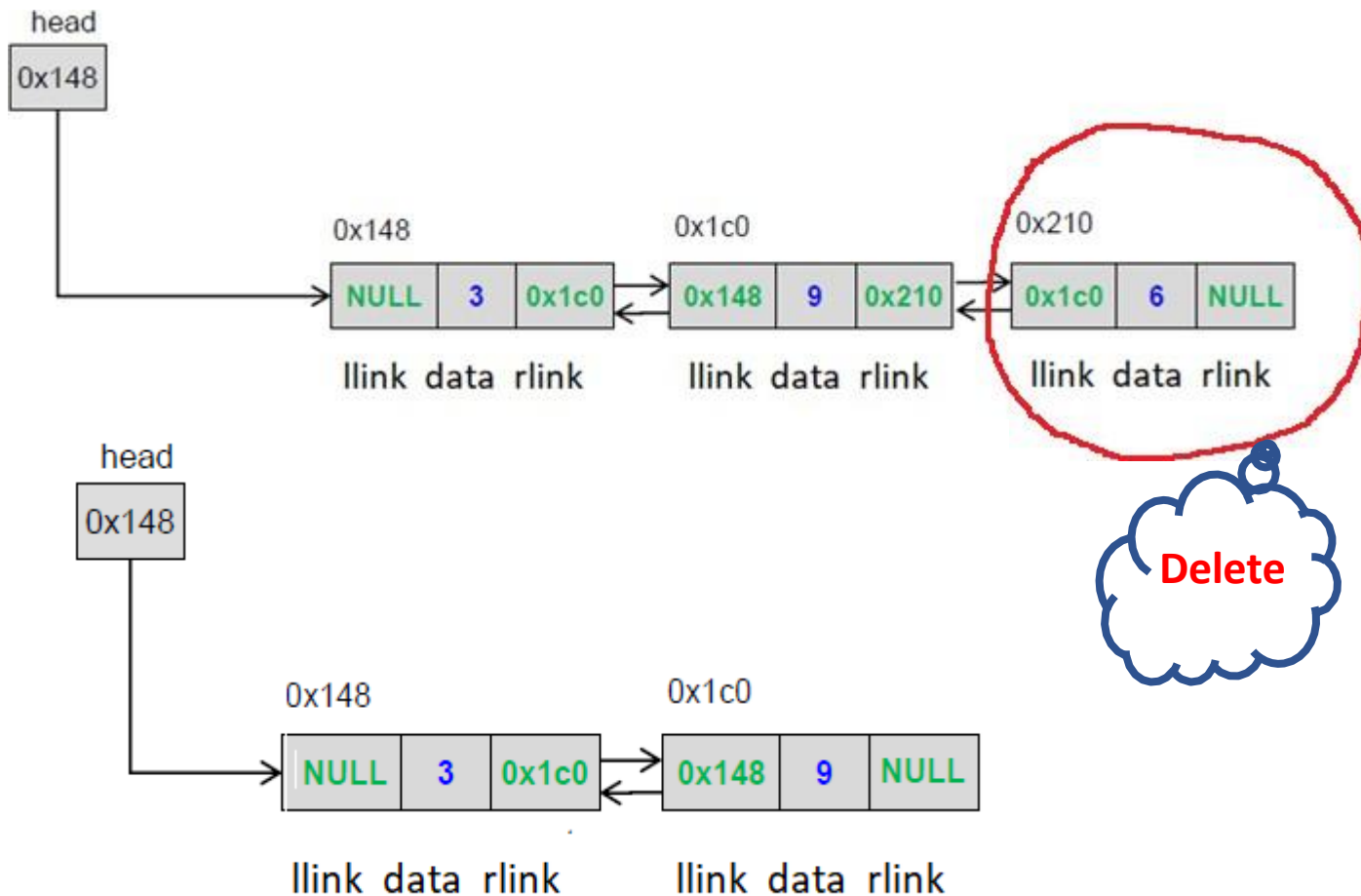
Deleting last node

What will change??

- Case I : Empty Linked List
- Case II : Linked list with a single node
 - first node gets freed up
 - head points to NULL
- Case III : Linked List with more than one node
 - Second last node rlink point to NULL
 - last node gets freed up

Deleting last node

- Case II : Linked List with more than one node



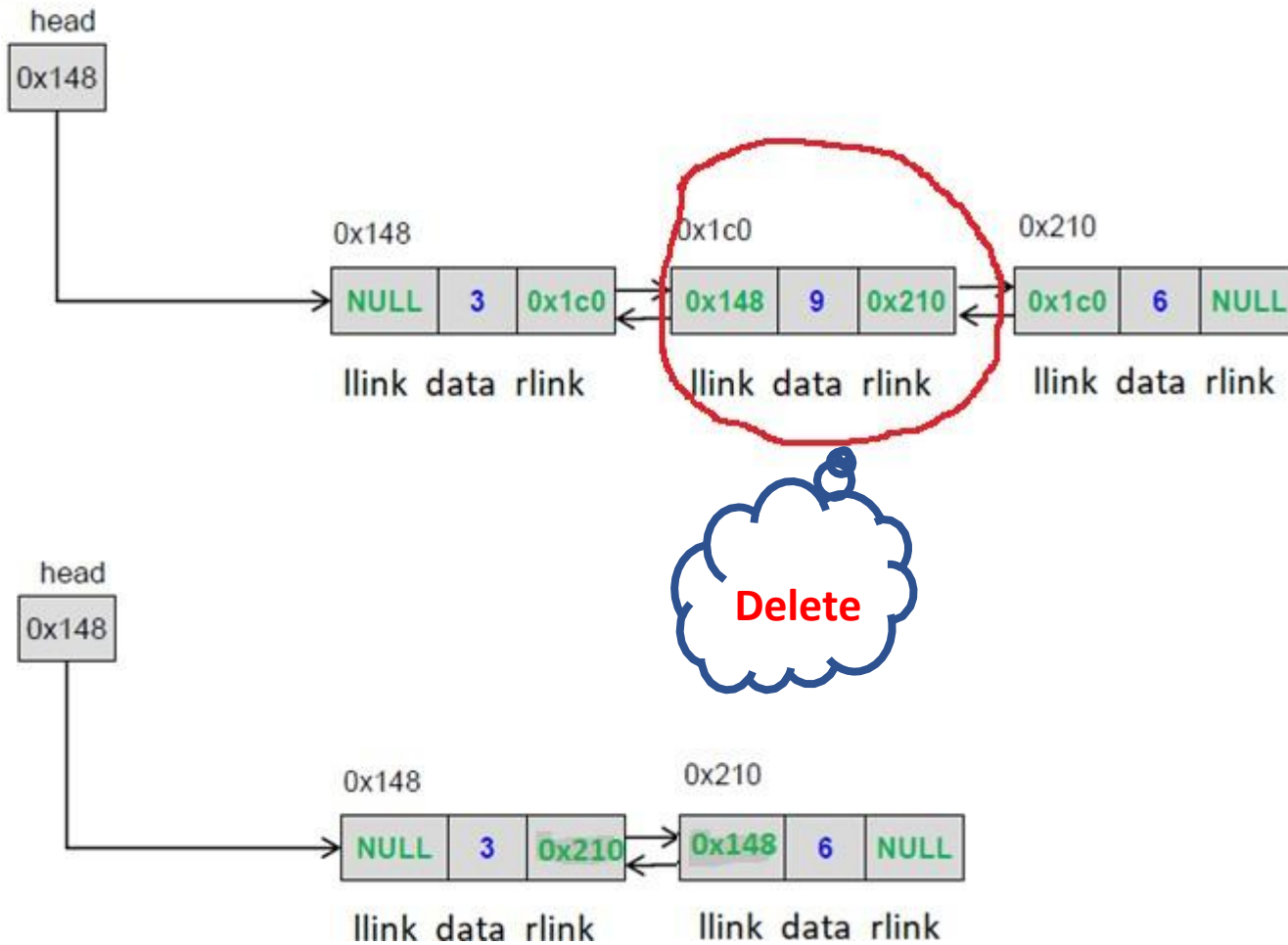
DATA STRUCTURES AND APPLICATIONS

Doubly Linked List Implementation



Deleting a node at intermediate position

- Case II : Linked List with more than one node





Doubly Linked List insert operation

Apply the concepts to implement following operations for a Doubly linked list

- reverse a doubly linked list
- Find the node pairs with a given sum in a doubly linked list
- Insert a node after a node with a given value
- Remove duplicate nodes from a doubly linked list



1. Which of the following is a disadvantage of a DLL compared to an SLL?

- a) Faster traversal in both directions.
- b) Requires extra memory for storing the prev pointer in each node.
- c) Easier insertion and deletion from the middle.
- d) Can handle reverse traversal efficiently.



1. Which of the following is a disadvantage of a DLL compared to an SLL?

- a) Faster traversal in both directions.
- b) Requires extra memory for storing the prev pointer in each node.
- c) Easier insertion and deletion from the middle.
- d) Can handle reverse traversal efficiently.



2. Which sequence correctly inserts a new node newNode at the head of a DLL?

- a) `newNode->next = head; head = newNode;`
- b) `newNode->next = head; head->prev = newNode; head = newNode;`
- c) `head->next = newNode; newNode->prev = head; head = newNode;`
- d) `newNode->prev = NULL; head = newNode;`



2. Which sequence correctly inserts a new node newNode at the head of a DLL?

a) `newNode->next = head; head = newNode;`

b) `newNode->next = head; head->prev = newNode; head = newNode;`

c) `head->next = newNode; newNode->prev = head; head = newNode;`

d) `newNode->prev = NULL; head = newNode;`



3. To insert at the end of a DLL, which step is incorrect?

- a) Traverse to the last node.
- b) Set `last->next = newNode`.
- c) Set `newNode->prev = last`.
- d) Set `newNode->next = head`.



3. To insert at the end of a DLL, which step is incorrect?

- a) Traverse to the last node.
- b) Set `last->next = newNode`.
- c) Set `newNode->prev = last`.
- d) Set `newNode->next = head`.



4. To insert newNode after a node p in a DLL, which of the following steps is essential but often missed?

- a) `newNode->next = p->next;`
- b) `p->next = newNode;`
- c) `p->next->prev = newNode;` (if `p->next` is not NULL)
- d) All of the above.



4. To insert newNode after a node p in a DLL, which of the following steps is essential but often missed?

- a) `newNode->next = p->next;`
- b) `p->next = newNode;`
- c) `p->next->prev = newNode;` (if `p->next` is not NULL)
- d) All of the above.



5. When deleting a node target (not head/tail) in a DLL, which of the following is correct?

- a) `target->prev->next = target->next; target->next->prev = target->prev; free(target);`
- b) `target->prev = target->next; target->next = target->prev; free(target);`
- c) `target->next->prev = NULL; free(target);`
- d) `target->prev = NULL; free(target);`

5. When deleting a node target (not head/tail) in a DLL, which of the following is correct?

- a) `target->prev->next = target->next; target->next->prev = target->prev; free(target);`
- b) `target->prev = target->next; target->next = target->prev; free(target);`
- c) `target->next->prev = NULL; free(target);`
- d) `target->prev = NULL; free(target);`



Vandana M L

Department of Computer Science and Engineering

vandanamd@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Circular Singly Linked List

Vandana M L

Department of Computer Science and Engineering



Circular linked list is a linked list where all nodes are connected to form a circle.

- Circular Singly Linked List
- Circular Doubly Linked List

With additional head node

Without additional head node

DATA STRUCTURES AND ITS APPLICATIONS

Circular Linked List Operations



Insert a node

- Insert at front
- Insert at end
- Insert at a position
- Ordered insertion

Delete node

- Delete front node
- Delete end node
- Delete a node from position
- Delete a node with a given value

Additional

- Display list
- Concatenate two list
- reverse a list

DATA STRUCTURES AND ITS APPLICATIONS

Circular Linked List: Applications



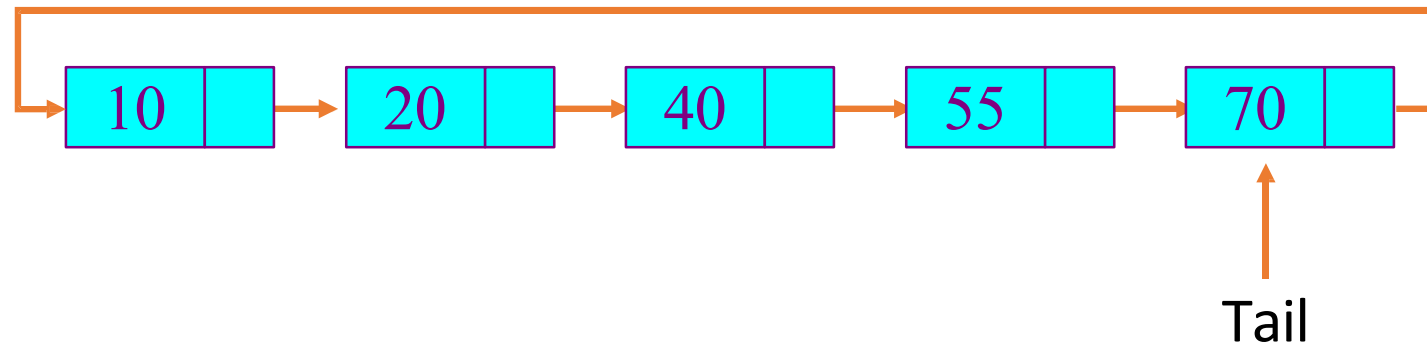
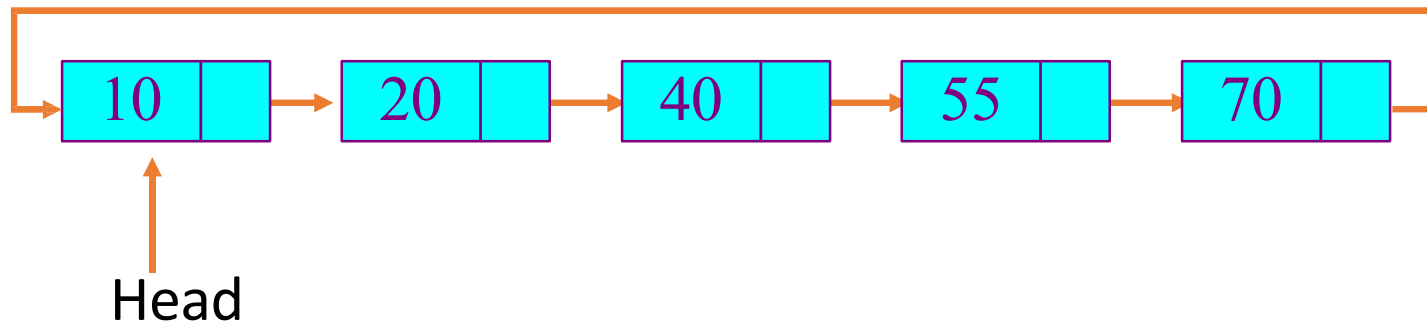
- Useful for implementation of queue, eliminates the need to maintain two pointers as in case of queue implementation using arrays
- Circular linked lists are useful for applications to repeatedly go around the list like playing video and sound files in “looping” mode
- Advanced data structures like Fibonacci Heap Implementation
- Plays a key role in linked implementation of graphs

DATA STRUCTURES AND ITS APPLICATIONS

Circular Singly Linked List



- It supports traversing from the end of the list to the beginning by making the last node point back to the head of the list
- A **Tail pointer** is often used instead of a Head pointer





```
#include <iostream>
using namespace std;

struct Node{
    int data;
    struct Node* next;
};
typedef struct node csll_node;
```



Insertion at the beginning

Insert at the front of linked list

- Create a node

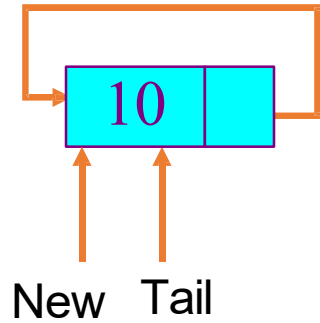
If the list is empty

- make the tail pointer point towards the new node

Else

- Change the new node link field to point to the first node
- Change the last node link to point to the new node

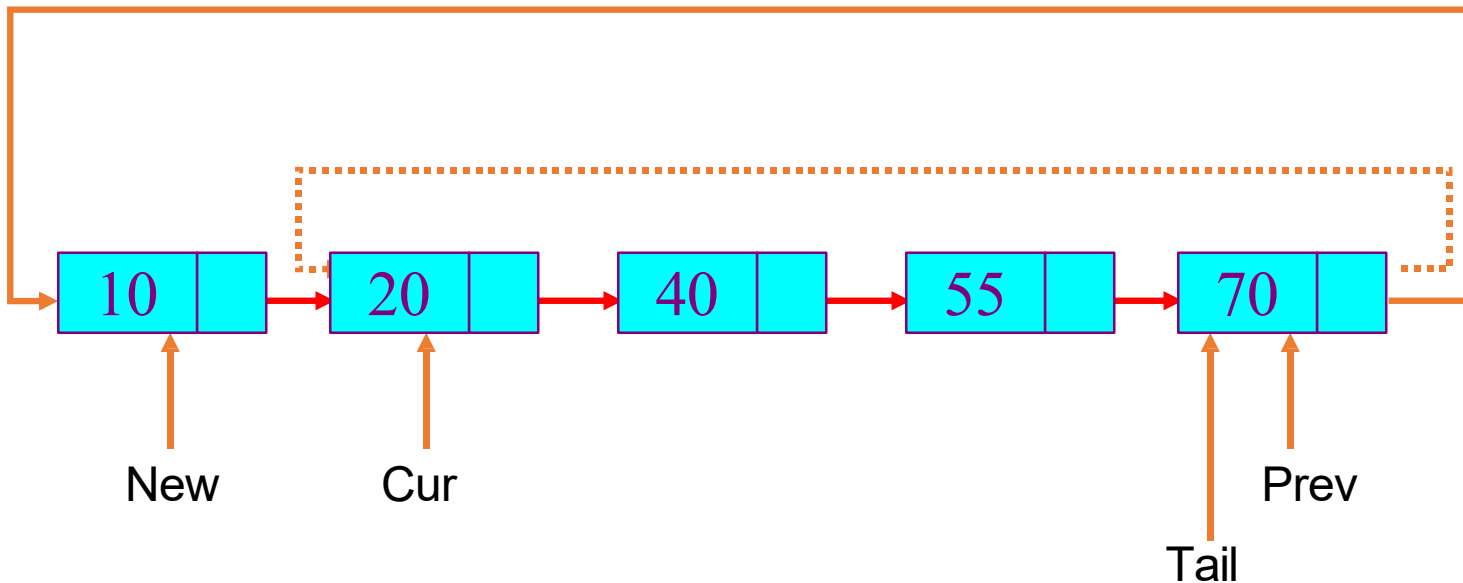
Insertion into an empty list



Insert to head of a Circular Linked List

New->next = Cur; ➡ New->next = Tail->next;

Prev->next = New; ➡ Tail->next = New;



DATA STRUCTURES AND ITS APPLICATIONS

Circular Singly Linked List Operations

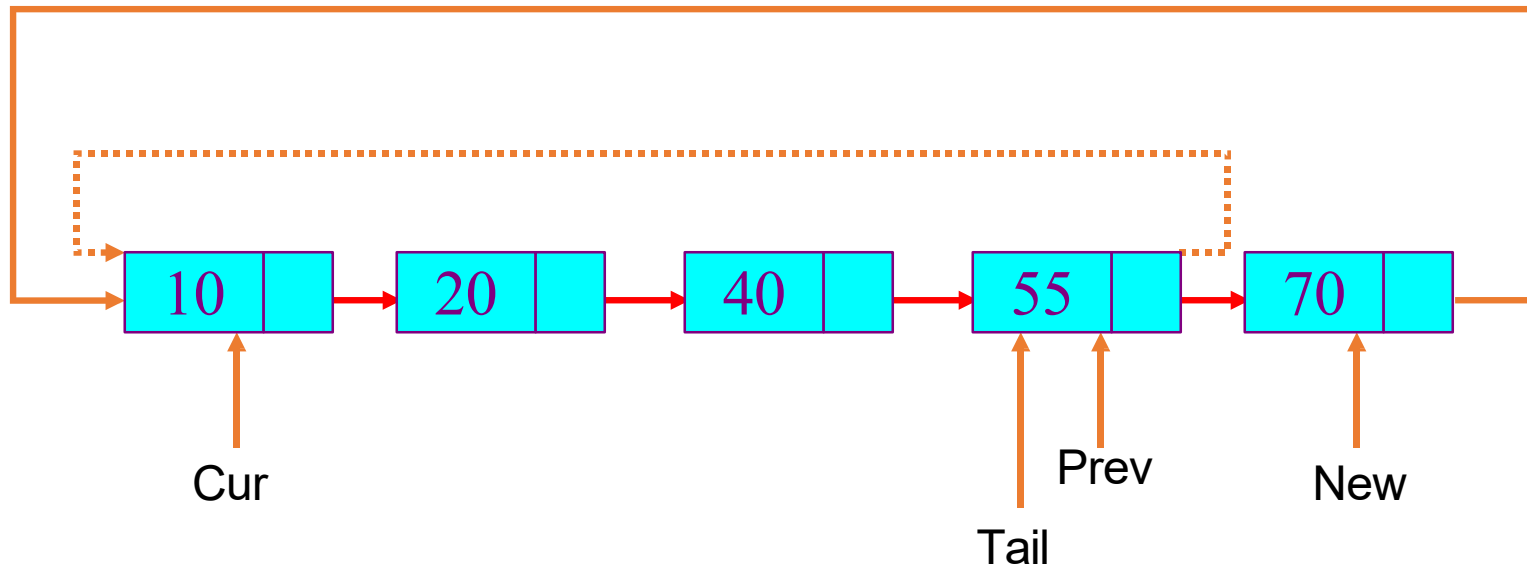


Insert to the end of a Circular Linked List

New->next = Cur; ➡ New->next = Tail->next;

Prev->next = New; ➡ Tail->next = New;

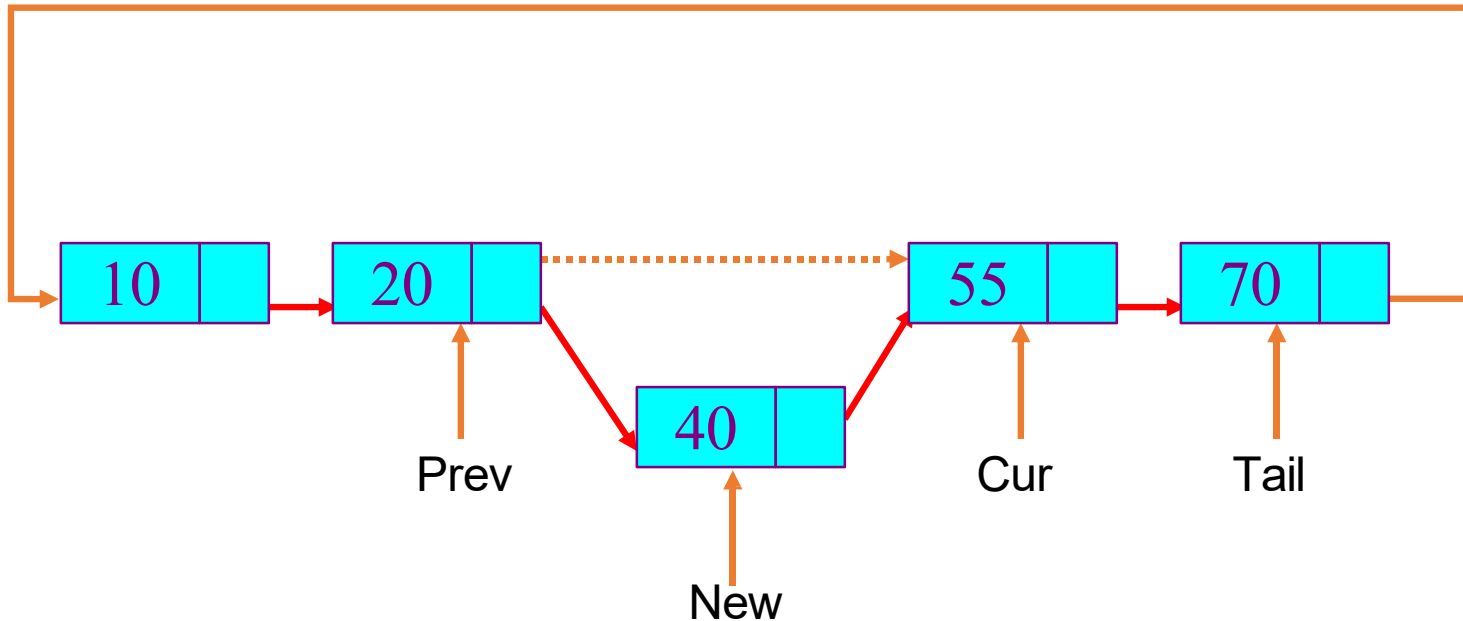
Tail = New;



Insert to the middle of Circular Linked List

New->next = Cur;

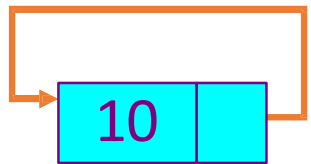
Prev->next = New;



Delete a node from a single-node Circular Linked List

Tail = NULL;

free(Cur);

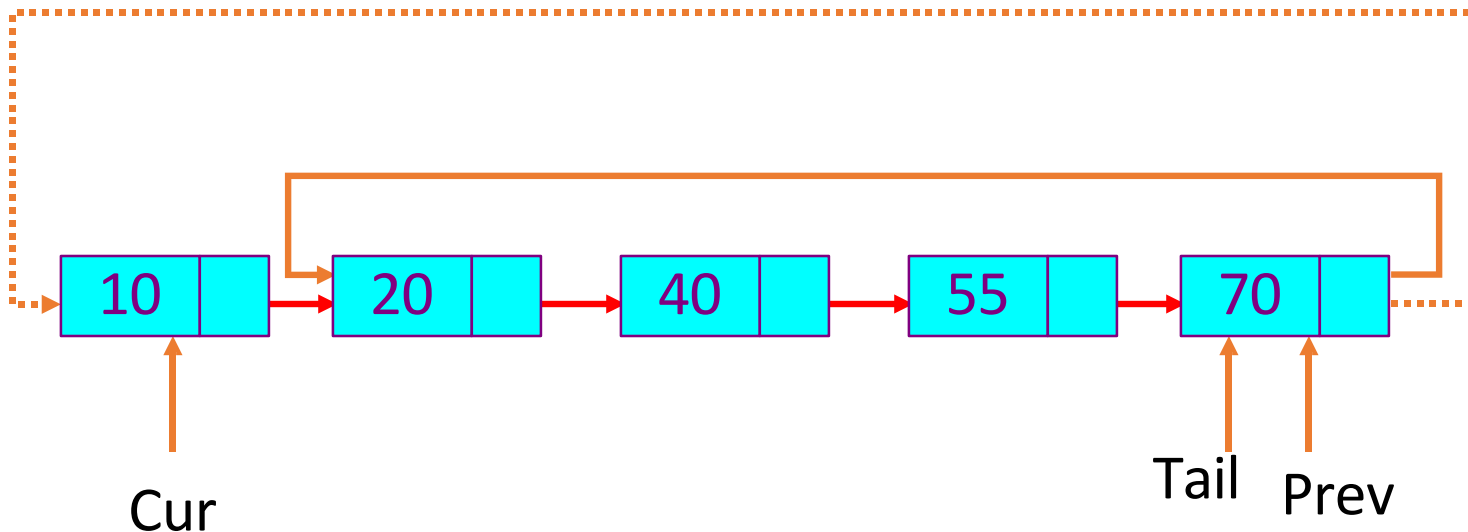


Tail = Cur = Prev

Delete the head node from a Circular Linked List

Prev->next = Cur->next; // same as: Tail->next = Cur->next

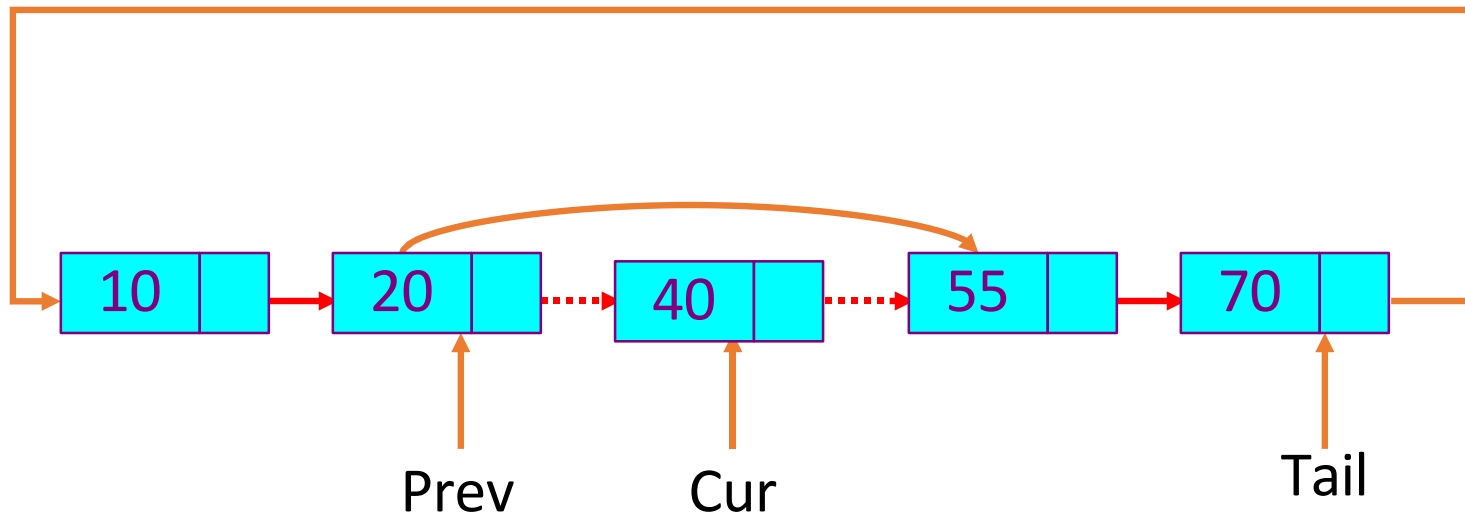
free(cur);



Delete a middle node Cur from a Circular Linked List

Prev->next = Cur->next;

Free(Cur);





Circular Singly Linked List operations

Apply the concepts to implement following operations for a singly linked list

- insert a node after a given node(pointer)
- Insert a node after a node with a given value

1. In a CSL, which is the correct condition for traversal starting from head?

a) while (temp != NULL)

b) while (temp->next != NULL)

c) do { ... } while (temp != head)

d) while (temp->next != head->next)



1. In a CSL, which is the correct condition for traversal starting from head?

a) while (temp != NULL)

b) while (temp->next != NULL)

c) do { ... } while (temp != head)

d) while (temp->next != head->next)



2. Which of the following is true about the head pointer in a CSLL?

- a) head always points to the last node.
- b) head points to the first node, but last->next = head.
- c) head->next = NULL.
- d) head cannot be NULL in any case.



2. Which of the following is true about the head pointer in a CSLL?

- a) head always points to the last node.
- b) head points to the first node, but last->next = head.
- c) head->next = NULL.
- d) head cannot be NULL in any case.

3. To insert a node at the beginning of a CSLL, which of the following sequences is correct?

- a) Add new node, set new->next = head, find last node, set last->next = new, then update head = new.
- b) Set new->next = head, update head = new, done.
- c) Set new->next = head->next, update head to new.
- d) Set head->next = new, then new->next = head.

3. To insert a node at the beginning of a CSLL, which of the following sequences is correct?

- a) Add new node, set new->next = head, find last node, set last->next = new, then update head = new.
- b) Set new->next = head, update head = new, done.
- c) Set new->next = head->next, update head to new.
- d) Set head->next = new, then new->next = head.



4. For inserting a node at the end of a CSLL, which step is mandatory?

- a) Update $\text{last} \rightarrow \text{next} = \text{new}$ and $\text{new} \rightarrow \text{next} = \text{NULL}$.
- b) Find last node (whose $\text{next} = \text{head}$), set $\text{new} \rightarrow \text{next} = \text{head}$, and update $\text{last} \rightarrow \text{next} = \text{new}$.
- c) Set $\text{head} = \text{new}$ if last node exists.
- d) No need to link to head, as list is circular by default.

4. For inserting a node at the end of a CSLL, which step is mandatory?

- a) Update $\text{last} \rightarrow \text{next} = \text{new}$ and $\text{new} \rightarrow \text{next} = \text{NULL}$.
- b) Find last node (whose next = head), set $\text{new} \rightarrow \text{next} = \text{head}$, and update $\text{last} \rightarrow \text{next} = \text{new}$.
- c) Set $\text{head} = \text{new}$ if last node exists.
- d) No need to link to head, as list is circular by default.



5. If a CSLL contains only one node and we delete it, what must happen?

- a) head = NULL.
- b) head->next = head.
- c) head->next = NULL.
- d) Nothing, list remains unchanged.



5. If a CSLL contains only one node and we delete it, what must happen?

- a) **head = NULL.**
- b) head->next = head.
- c) head->next = NULL.
- d) Nothing, list remains unchanged.



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu



Data Structures and its Applications

Dinesh Singh

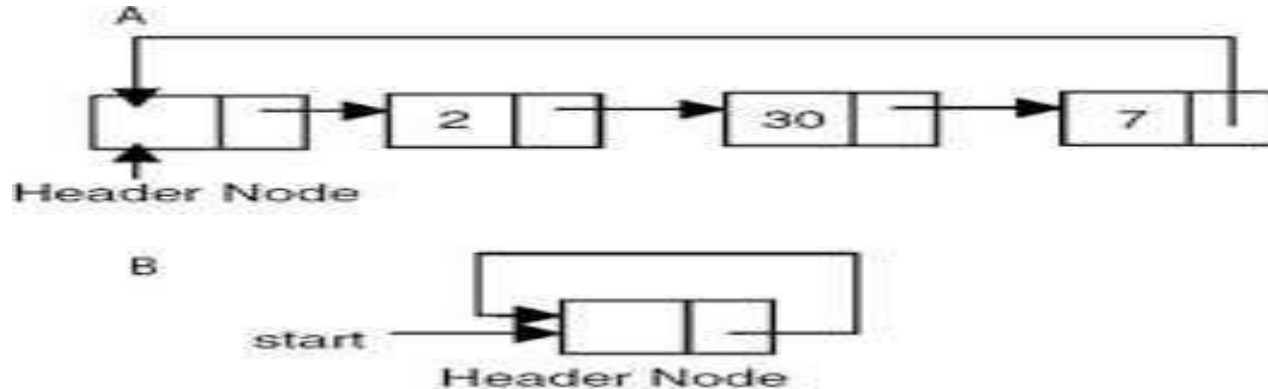
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Circular Linked Lists

Dinesh Singh

Department of Computer Science & Engineering



- The first node in the list is the header node.
- The address part of the last node points to the header node
- Circular list does not have a natural first or the last node
- An External pointer points to the header node and the one following this node is the first node.

Data Structures and its Applications

Operations on Circular Linked Lists



Implementation of some operations on Circular linked Lists with header node

- Insert at the head of a list
- Insert at the end of the List
- Delete a Node given its value

Note: head is a pointer to the header node and the following node is the first node

Creating Header node

```
struct node *create_head()
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=0; // keeps the count of nodes in the list
    temp->next=temp;
    return temp;
}
```

Algorithm to insert a node at the head of the list

insert_head(p,x)

//p pointer to header node, x element to be inserted

//x gets inserted after the header node

allocate memory for the node

initialise the node

//insert the new node after the header node

Copy the value of the next part of the header node into the next
part of the new node

Copy the address of the new node into next part of the header
node

Operations on Circular Linked Lists with header node

```
void insert_head(struct node *p,int x)
//p points to the header node, x element to be inserted
{
    struct node *temp;
    //create node and initialise
    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    // next part of new node points to the node after the header node
    temp->next=p->next;
    p->next=temp; //next part of header node points to the new node
    p->data++;
}
```


Operations on Circular Linked Lists with header node

Algorithm to insert a node at the end of the list

insert_tail(p,x)

//p pointer to header node, x element to be inserted

allocate memory for the node

initialise the node

move to the last node

//insert the new node after the last node

Copy the address of the header node into next of new node

Copy the address of the new node into the next of last node

Operations on Circular Linked Lists with header node

Algorithm to insert a node at the end of the list

```
void insert_tail(struct node *p,int x)
{
    struct node *temp,*q;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;

    q=p->next; // go the first node

    while(q->next!=p) // move to the last node
        q=q->next;

    temp->next=p; // copy address of header node into next of new node
    q->next=temp; // copy the address of new node into next of the last node
    p->data++; // increment the count of nodes in the list
}
```

Operations on Circular Linked Lists with header node

Algorithm to delete a node given its value

delete_node(p,x)

//p pointer to header node, x element to be deleted

move forward until the node to be deleted is found
or header node is reached

If(node found)

delete the node by adjusting the pointers

else

node not found // if header node is reached

```
void delete_node(struct node *p, int x)
{
    //p points to the header node, x is element to be inserted
    struct node *prev,*q;

    q=p->next; // go to the first node
    prev=p;
    //move forward until the data is found or header node is reached
    while((q!=p)&&(q->data!=x))
    {
        prev=q; // keep track of the previous node
        q=q->next;
    }
}
```

```
if(q==p) // header node reached
    printf("Node not found..\n");
else
{
    prev->next=q->next; //delete the node
    free(q);
    p->data--; // decrement the count of nodes in the list
}
}
```

1. In a circular singly linked list (CSL) with a header node, what is the primary advantage of the header node?

- a) It stores data just like other nodes.
- b) It acts as a sentinel to simplify insertion and deletion operations.
- c) It prevents memory leaks.
- d) It is used only for maintaining a count of nodes.

1. In a circular singly linked list (CSL) with a header node, what is the primary advantage of the header node?

- a) It stores data just like other nodes.
- b) It acts as a sentinel to simplify insertion and deletion operations.
- c) It prevents memory leaks.
- d) It is used only for maintaining a count of nodes.

2. To traverse a CSL with a header node, which of the following conditions is most appropriate?

- a) while (ptr != NULL)
- b) while (ptr != header) (starting from header->next)
- c) while (ptr->next != NULL)
- d) while (ptr->next != header->next)

2. To traverse a CSL with a header node, which of the following conditions is most appropriate?

- a) while (ptr != NULL)
- b) while (ptr != header) (starting from header->next)
- c) while (ptr->next != NULL)
- d) while (ptr->next != header->next)

3. When inserting a node immediately after the header node, what is the minimum number of pointer updates required in a CSL with a header node?

- a) 1
- b) 2
- c) 3
- d) It depends on list size.

3. When inserting a node immediately after the header node, what is the minimum number of pointer updates required in a CSL with a header node?

- a) 1
- b) 2
- c) 3
- d) It depends on list size.

4. When deleting the last data node (just before the header node) in a CSL with a header node, which of the following steps is correct?

- a) Find the second-last node, make its next point to header, and free the last node.
- b) Free header node and reconnect list.
- c) Move header pointer one step backward.
- d) Remove the first node instead of the last.

4. When deleting the last data node (just before the header node) in a CSL with a header node, which of the following steps is correct?

- a) Find the second-last node, make its next point to header, and free the last node.
- b) Free header node and reconnect list.
- c) Move header pointer one step backward.
- d) Remove the first node instead of the last.

5. When searching for a key in CSL with a header node (header does not store data), which is the correct loop termination condition?

- a) while (ptr != header)
- b) while (ptr != NULL)
- c) while (ptr->next != NULL)
- d) while (ptr->next != header)

5. When searching for a key in CSL with a header node (header does not store data), which is the correct loop termination condition?

- a) `while (ptr != header)`
- b) `while (ptr != NULL)`
- c) `while (ptr->next != NULL)`
- d) `while (ptr->next != header)`



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering



DATA STRUCTURES AND ITS APPLICATIONS

Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List

Vandana M L

Department of Computer Science and Engineering



Node Structure Definition

A doubly linked list node contain **three** fields:

- Data
- link to the next node
- link to the previous node.

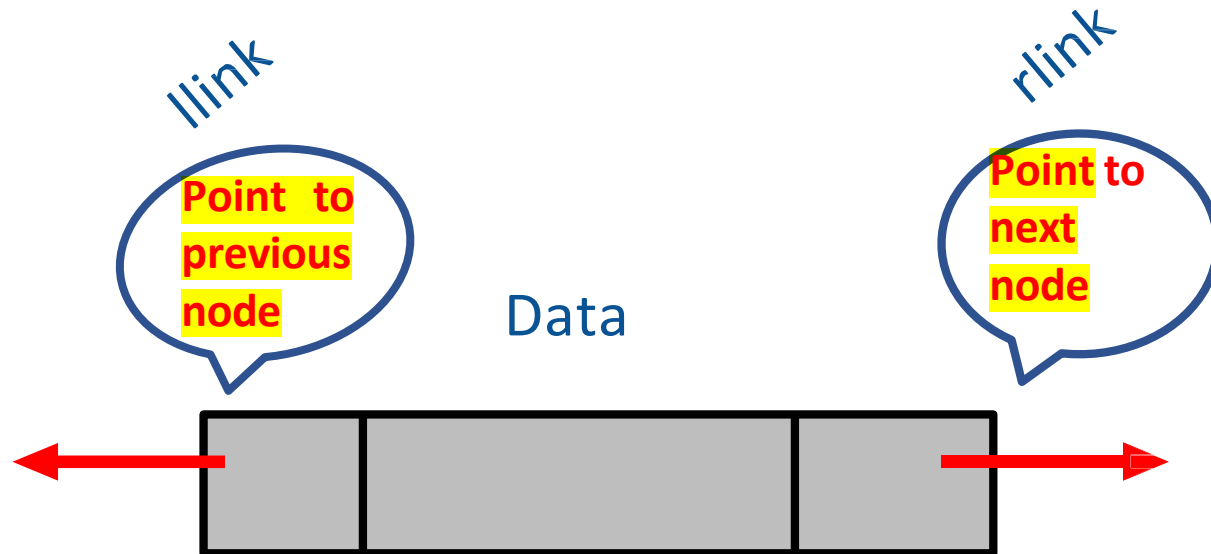
DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List



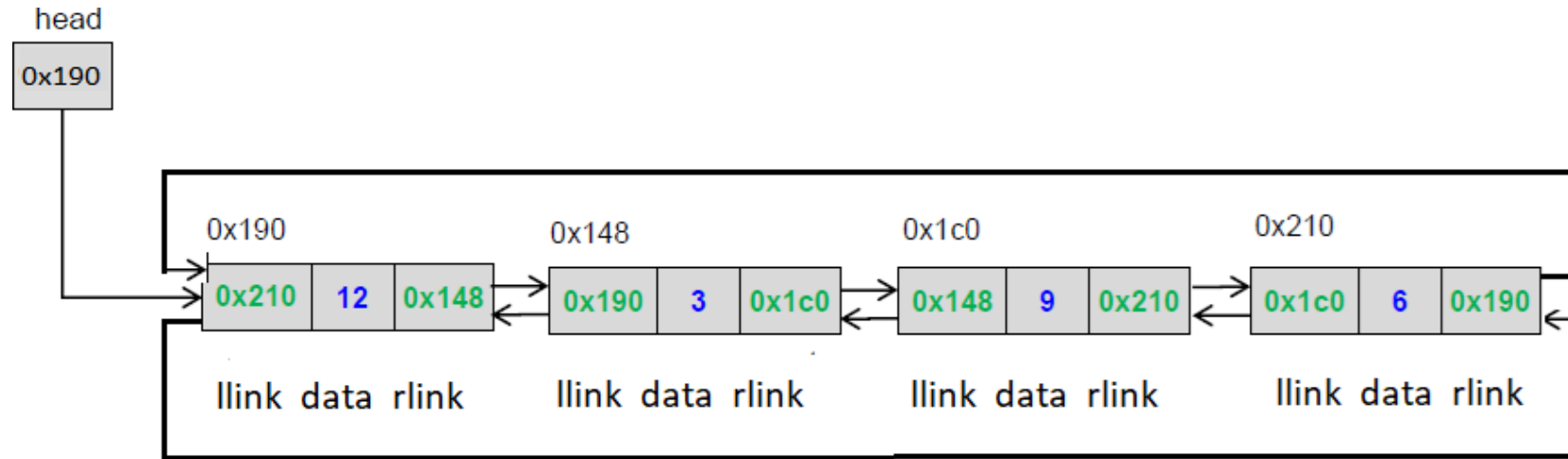
Node Structure Definition

```
struct node
{
    int data;
    struct node* llink;
    struct node* rlink;
};
```



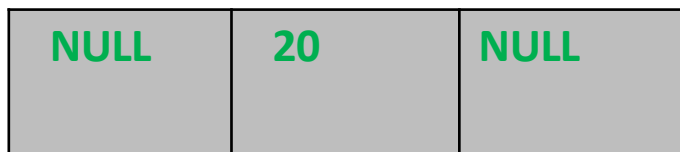
DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List: Example



Creating a node

- Allocate memory for the node dynamically
- If the memory is allocated successfully
 - set the data part
 - set the llink and rlink to NULL





Inserting a node

There are 3 cases

- Insertion at the beginning
- Insertion at the end
- Insertion at a given position



Insertion at the beginning

What all will change

Case 1: linked list empty

- Head pointer

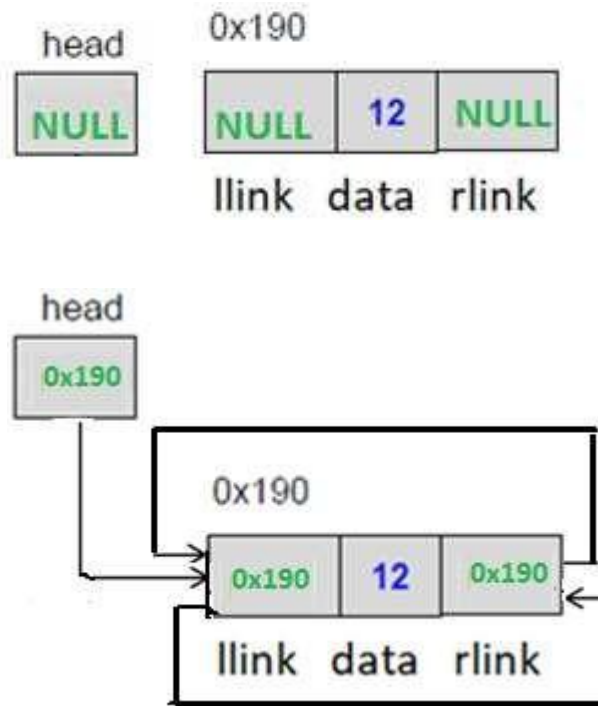
Case 2: linked list is not empty

- Head pointer
- New front node's rlink and llink
- Old front node's llink
- Last node's rlink

DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations

Insertion at the beginning (Case1)



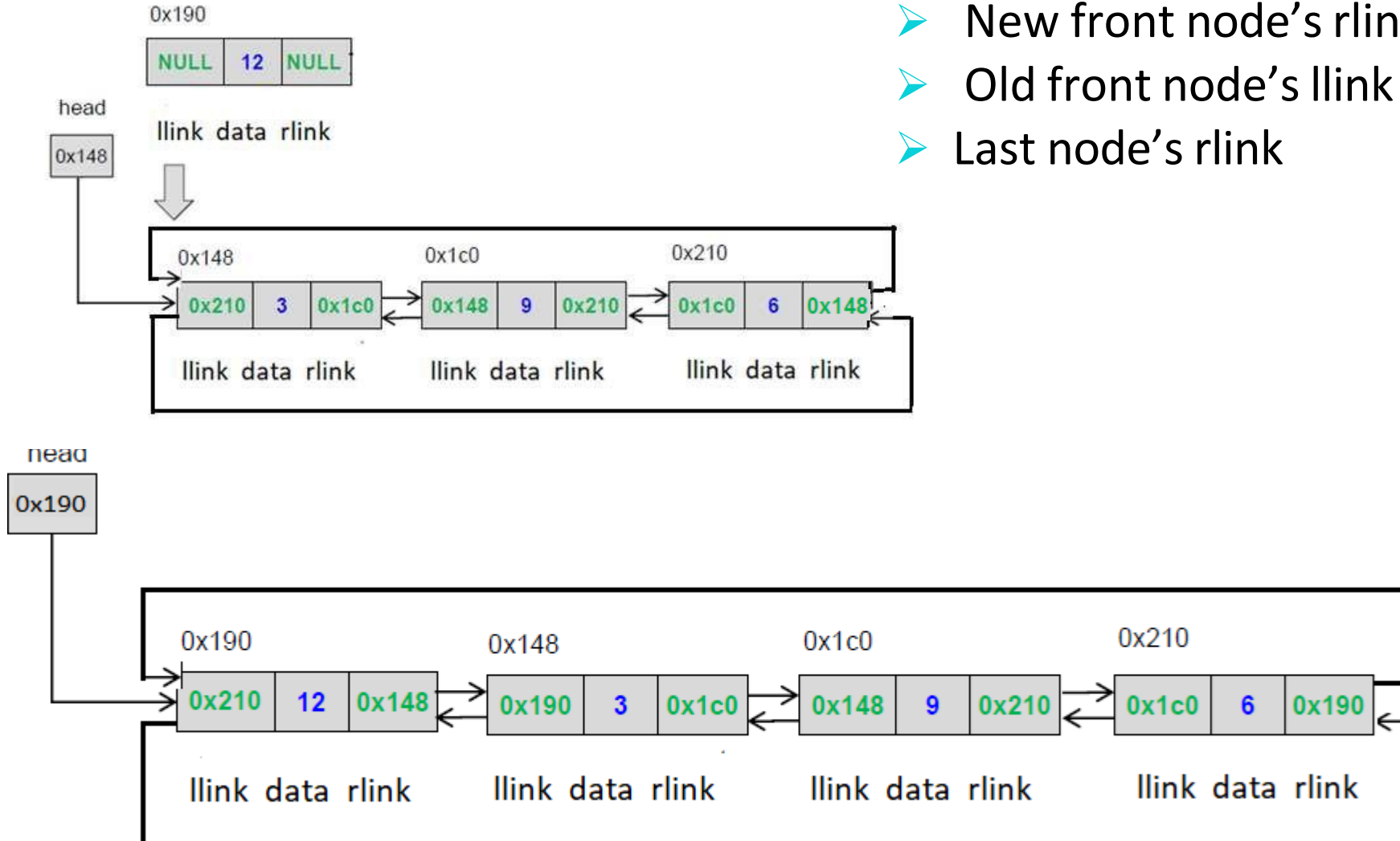
DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations



Insertion at the beginning(Case 2)

- Head pointer
- New front node's rlink and llink
- Old front node's llink
- Last node's rlink





Insertion at the end

What all will change

Case 1: linked list empty

- Head pointer

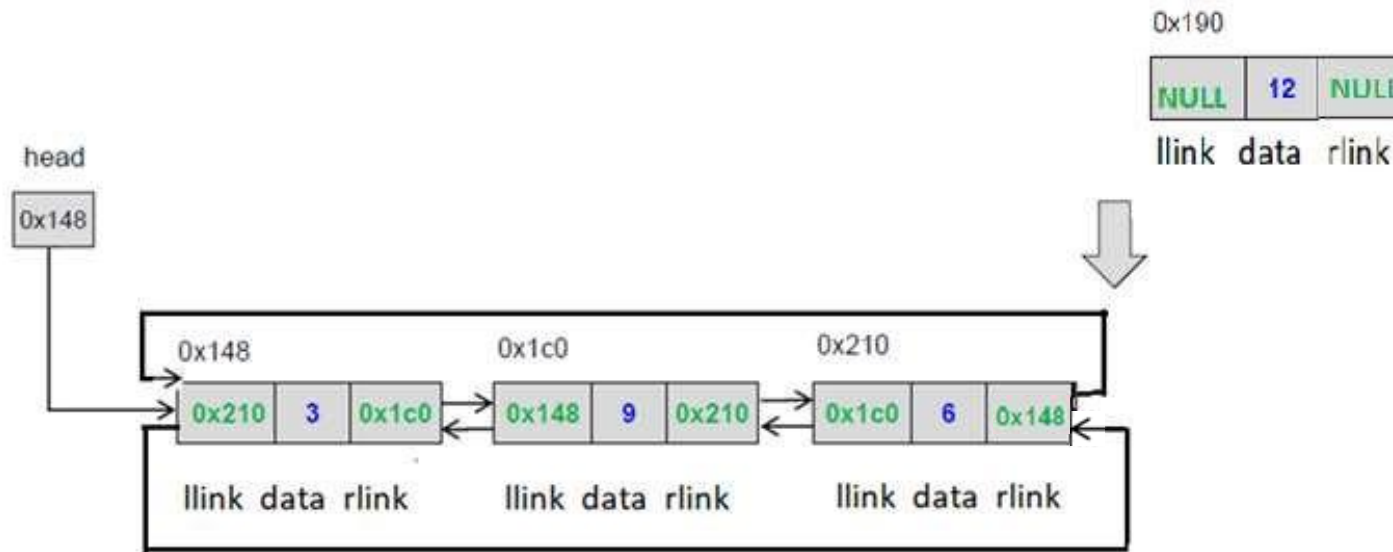
Case 2: linked list is not empty else

- Last node's rlink
- First node llink
- New node's llink and rlink

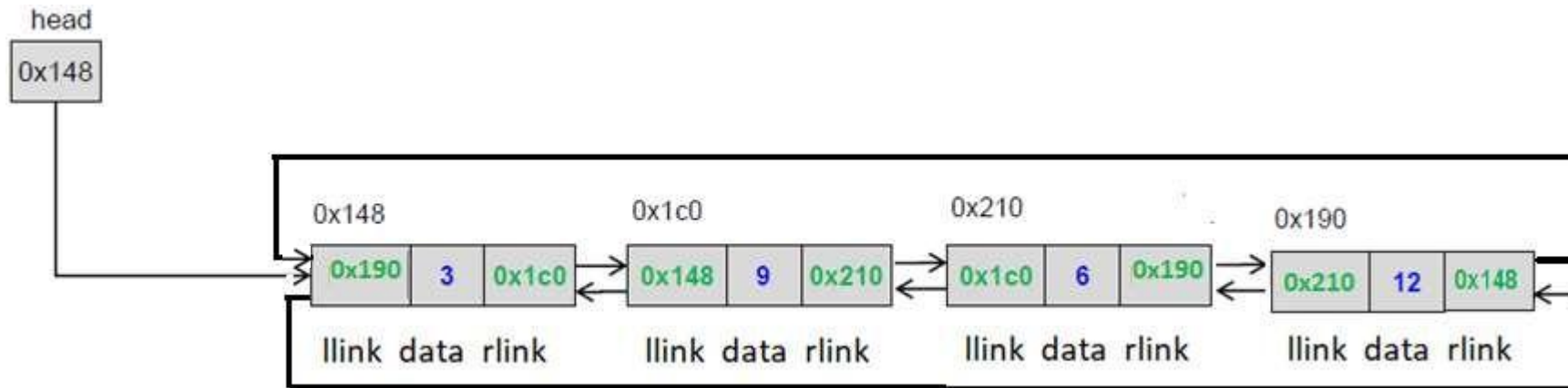
DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations

Insertion at the end



node's rlink
node llink
node's llink and rlink



DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations



Insertion at the given position

- Create a node

If the list is empty

- make the start pointer point towards the new node;

Else

if it is first position

- Insert at front

else

- Traverse the linked list to reach given position

- Keep track of the previous node

If it is valid position

intermediate position

- Change link fields of current previous and intermediate node

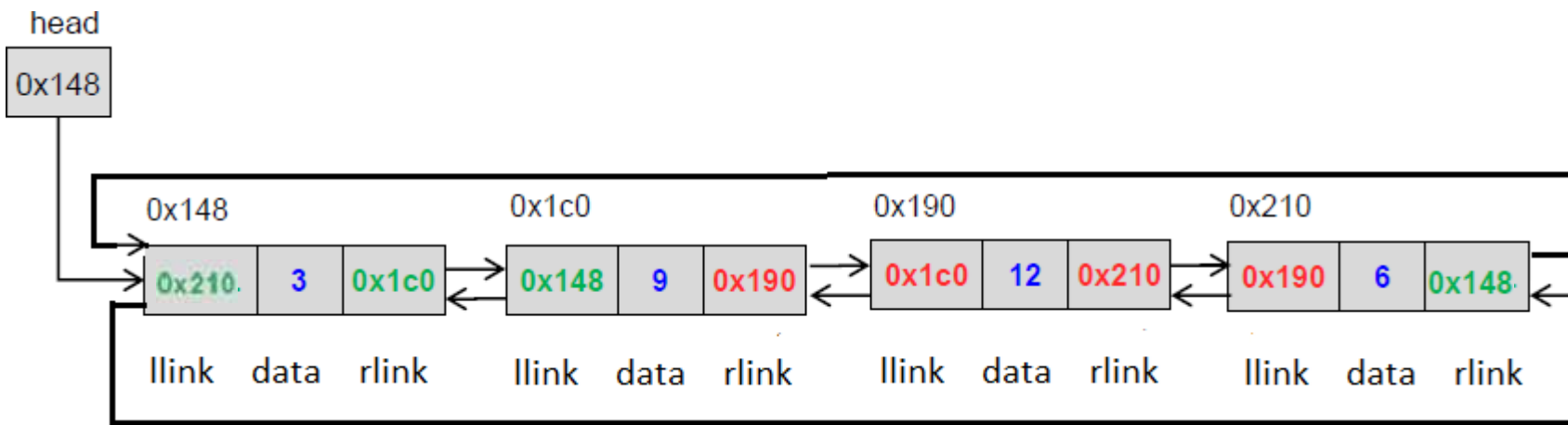
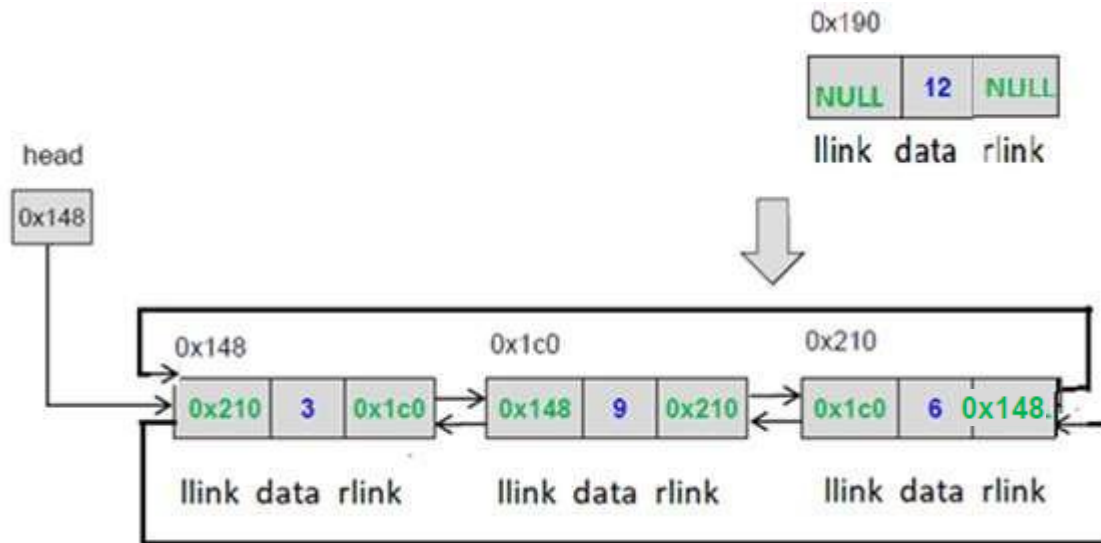
last position

- insert at end

DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations

Insertion at the given position





Deleting a node

There are 3 cases

- Deleting first node
- Deleting last node
- Deleting a node at a given position



Deleting a node

There are 3 cases

- Deleting first node
- Deleting last node
- Deleting a node at a given position



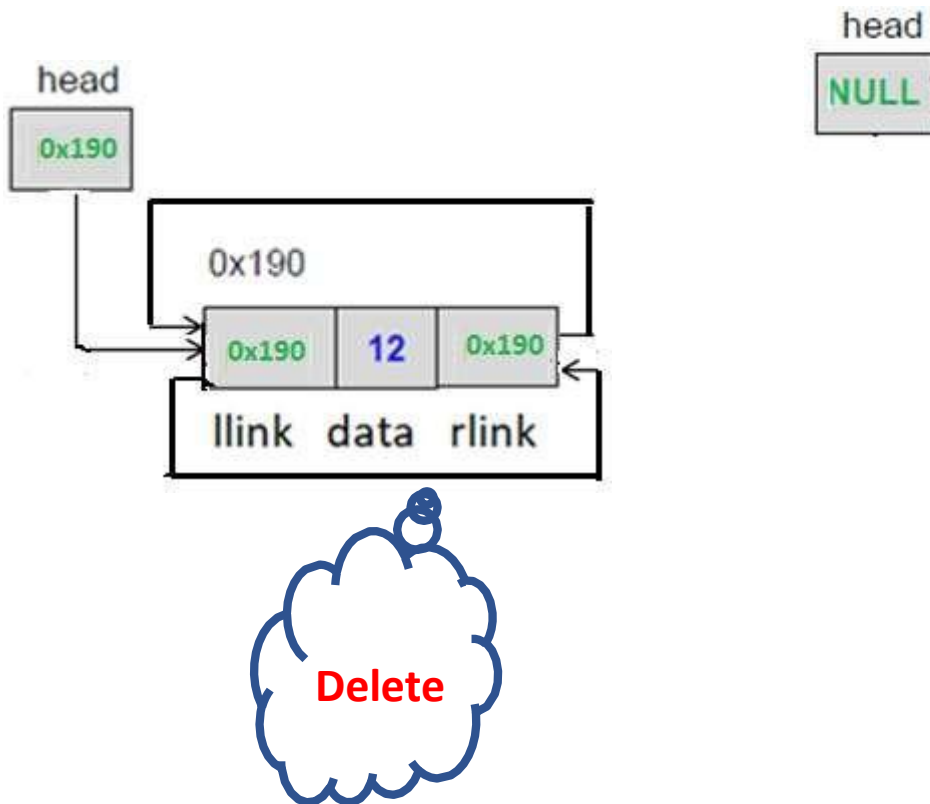
Deleting first node

What will change??

- Case I : Empty Linked List
- Case II : Linked list with a single node
 - first node gets freed up
 - head points to NULL
- Case III : Linked List with more than one node
 - Second node llink
 - last node rlink
 - first node gets freed off
 - head pointer points to second node

Deleting first node

- Case II : Linked list with a single node



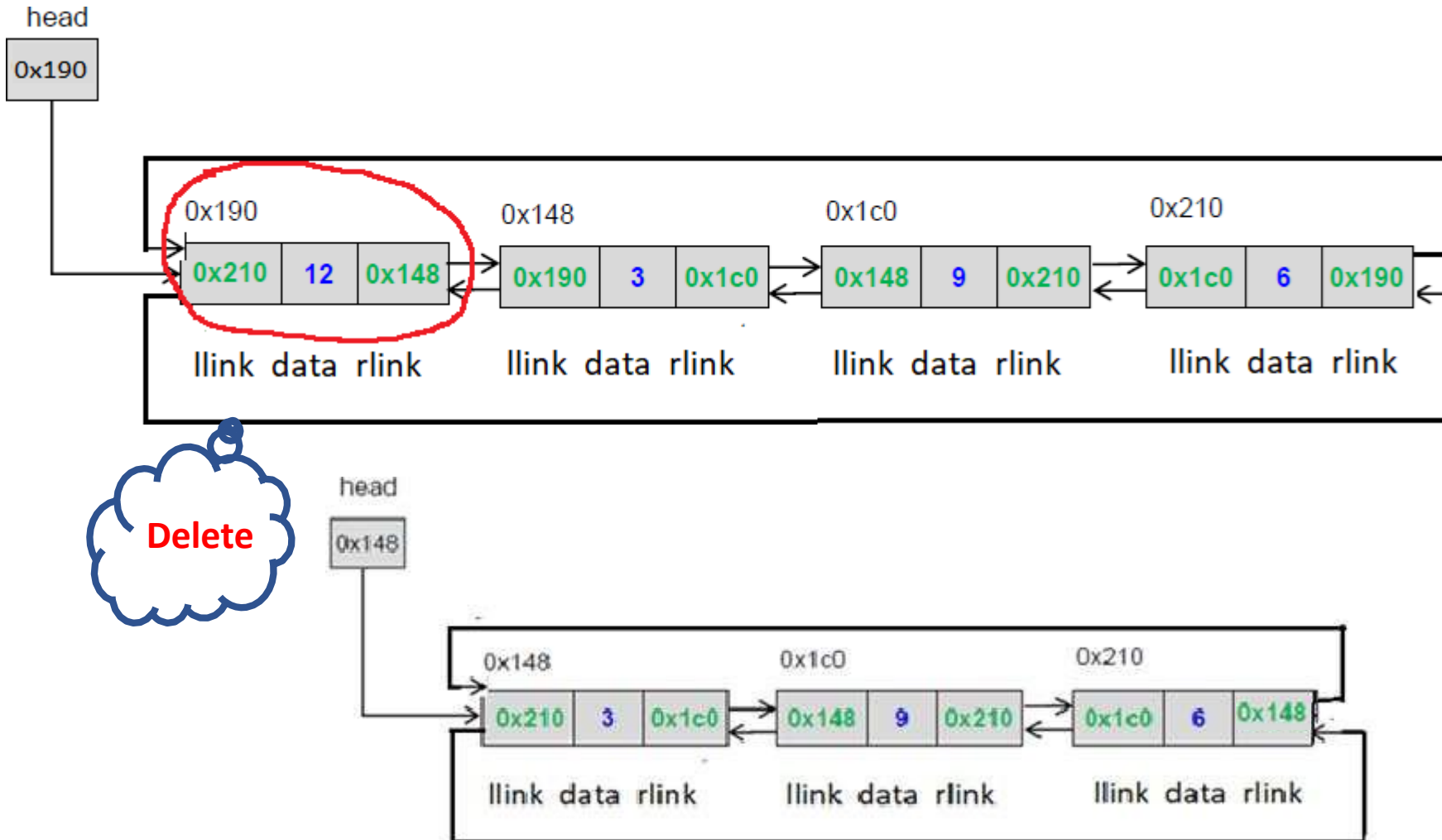
DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations



Deleting first node

- Case III : Linked List with more than one node





Deleting last node

What will change??

- Case I : Empty Linked List
- Case II : Linked list with a single node
 - first node gets freed up
 - head points to NULL
- Case III : Linked List with more than one node
 - Second last node rlink
 - first node llink
 - last node gets freed up

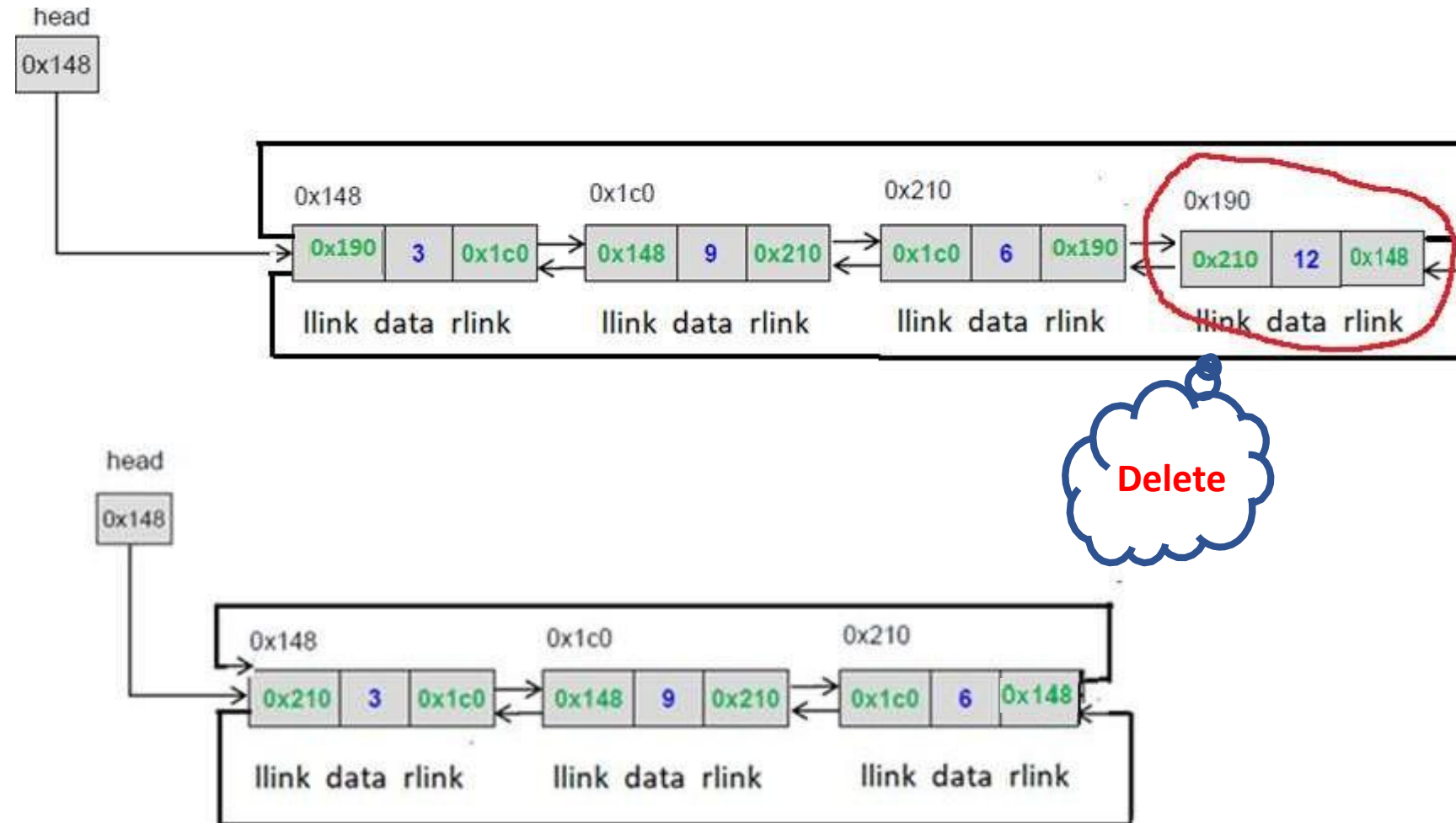
DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations



Deleting last node

➤ Case II : Linked List with more than one node



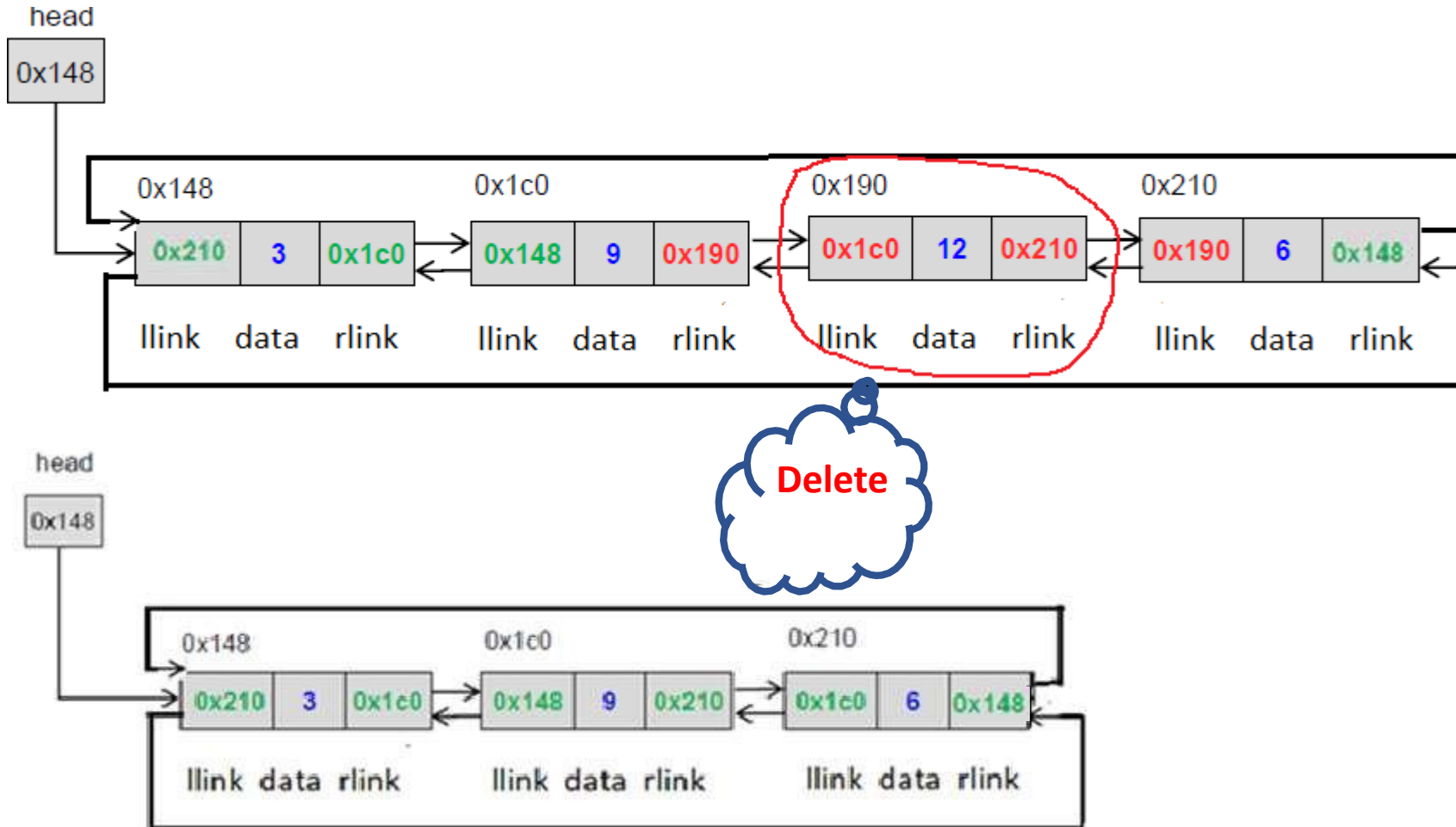
DATA STRUCTURES AND ITS APPLICATIONS

Circular Doubly Linked List Operations



Deleting a node at intermediate position

➤ Case II : Linked List with more than one node





Circular doubly Linked List operations

Apply the concepts to implement following operations for a doubly circular linked list

- Reverse list using recursion
- Search given element in the list
- Find the largest value in the list



1. Which statement correctly describes a Circular Doubly Linked List (CDLL)?

- a) The next pointer of the last node is NULL.
- b) Both the next pointer of the last node and the prev pointer of the head point back to each other.
- c) Only the head node's prev is NULL.
- d) A CDLL cannot have more than two nodes.



1. Which statement correctly describes a Circular Doubly Linked List (CDLL)?

- a) The next pointer of the last node is NULL.
- b) Both the next pointer of the last node and the prev pointer of the head point back to each other.
- c) Only the head node's prev is NULL.
- d) A CDLL cannot have more than two nodes.



2. Which of the following loop structures is best for traversing a CDLL starting from head?

- a) while (temp != NULL)
- b) for (temp = head; temp != NULL; temp = temp->next)
- c) do { ... } while (temp != head)
- d) while (temp->next != NULL)



2. Which of the following loop structures is best for traversing a CDLL starting from head?

- a) while (temp != NULL)
- b) for (temp = head; temp != NULL; temp = temp->next)
- c) do { ... } while (temp != head)
- d) while (temp->next != NULL)



3. To insert a new node at the head of a CDLL, which sequence is correct?

- a) Adjust head->next and head->prev only.
- b) Set new->next = head, new->prev = head->prev, head->prev->next = new, head->prev = new, and update head = new.
- c) Set new->next = head->next and head->next = new.
- d) Update only prev pointers, as next is automatically updated.



3. To insert a new node at the head of a CDLL, which sequence is correct?

- a) Adjust head->next and head->prev only.
- b) Set new->next = head, new->prev = head->prev, head->prev->next = new, head->prev = new, and update head = new.
- c) Set new->next = head->next and head->next = new.
- d) Update only prev pointers, as next is automatically updated.



4. Which pointer updates are required to insert newNode at the end of a CDLL?

- a) Update $\text{tail} \rightarrow \text{next} = \text{newNode}$, $\text{newNode} \rightarrow \text{prev} = \text{tail}$, $\text{newNode} \rightarrow \text{next} = \text{NULL}$.
- b) Update $\text{tail} \rightarrow \text{next} = \text{newNode}$, $\text{newNode} \rightarrow \text{prev} = \text{tail}$, $\text{newNode} \rightarrow \text{next} = \text{head}$, and $\text{head} \rightarrow \text{prev} = \text{newNode}$.
- c) Update only $\text{head} \rightarrow \text{prev} = \text{newNode}$.
- d) Use recursion to find the last node and insert.



4. Which pointer updates are required to insert newNode at the end of a CDLL?

- a) Update `tail->next = newNode`, `newNode->prev = tail`, `newNode->next = NULL`.
- b) Update `tail->next = newNode`, `newNode->prev = tail`, `newNode->next = head`, and `head->prev = newNode`.
- c) Update only `head->prev = newNode`.
- d) Use recursion to find the last node and insert.

5. Which steps correctly delete the head node in a CDLL (with more than one node)?

a) `head = head->next; head->prev = NULL; free(oldHead);`

b) `head->prev->next = head->next; head->next->prev = head->prev; head = head->next; free(oldHead);`

c) `head = NULL; free(head);`

d) `head->next = head; free(head);`

5. Which steps correctly delete the head node in a CDLL (with more than one node)?

a) `head = head->next; head->prev = NULL; free(oldHead);`

b) `head->prev->next = head->next; head->next->prev = head->prev; head = head->next; free(oldHead);`

c) `head = NULL; free(head);`

d) `head->next = head; free(head);`



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

+91 7411716615



DATA STRUCTURES AND ITS APPLICATIONS

Prof. Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Multilist Representation

Prof. Vandana M L

Department of Computer Science and Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Sparse Matrix



Matrix ??

Two Dimensional data 1 1 3 0 4

1 3 5 1 0

9 0 5 1 0

Sparse Matrix??

More zero elements than non zero elements

0 0 3 0 0

0 0 5 1 0

0 0 0 0 0



- 2D Matrix

results in lot of memory wastage as non zero elements are also stored

- Triple Notation

 - Array representation

- Multilist Representation

 - Linked representation hence size can be changed dynamically

DATA STRUCTURES AND ITS APPLICATIONS

Sparse Matrix Representation: Triple Notation



In triple notation sparse matrix is represented as an array of tuple values.

Each tuple consists of

<rowno columnno Value>

The first block in array block holds information regarding

<total no of rows, total no of columns ,value> Total Number of Non-Zero Values

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 4 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \\ 8 & 0 & 6 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$



Triple Notation

Row No	Column No	Value
5	4	6
0	0	2
1	0	4
1	3	3
3	0	8
3	3	1
4	2	6

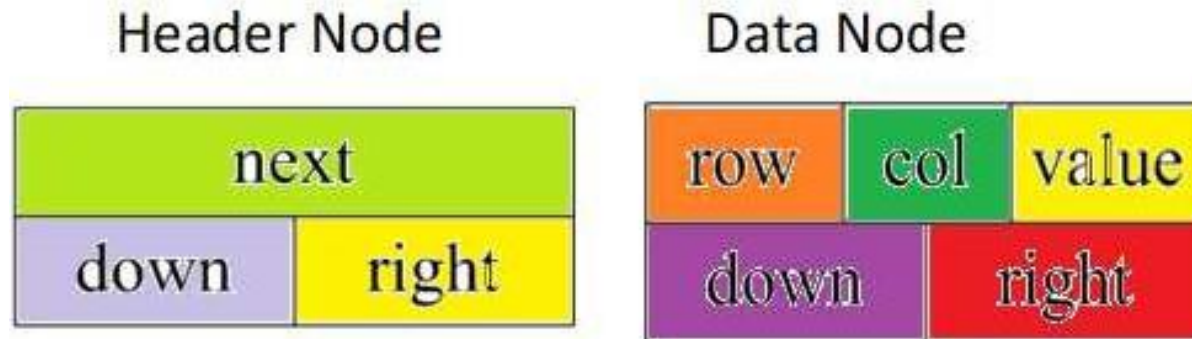
DATA STRUCTURES AND ITS APPLICATIONS

Sparse Matrix Representation: Linked representation



Node Structure

Two types of nodes are used



DATA STRUCTURES AND ITS APPLICATIONS

Sparse Matrix Representation: Linked representation



Node Structure Definition

```
#define MAX_SIZE 50 /* size of  
largest  
matrix */  
typedef enum {head, entry} tagfield;  
typedef struct matrixNode *  
matrixPointer; typedef struct entryNode {  
int row;  
int col;  
int value; };
```

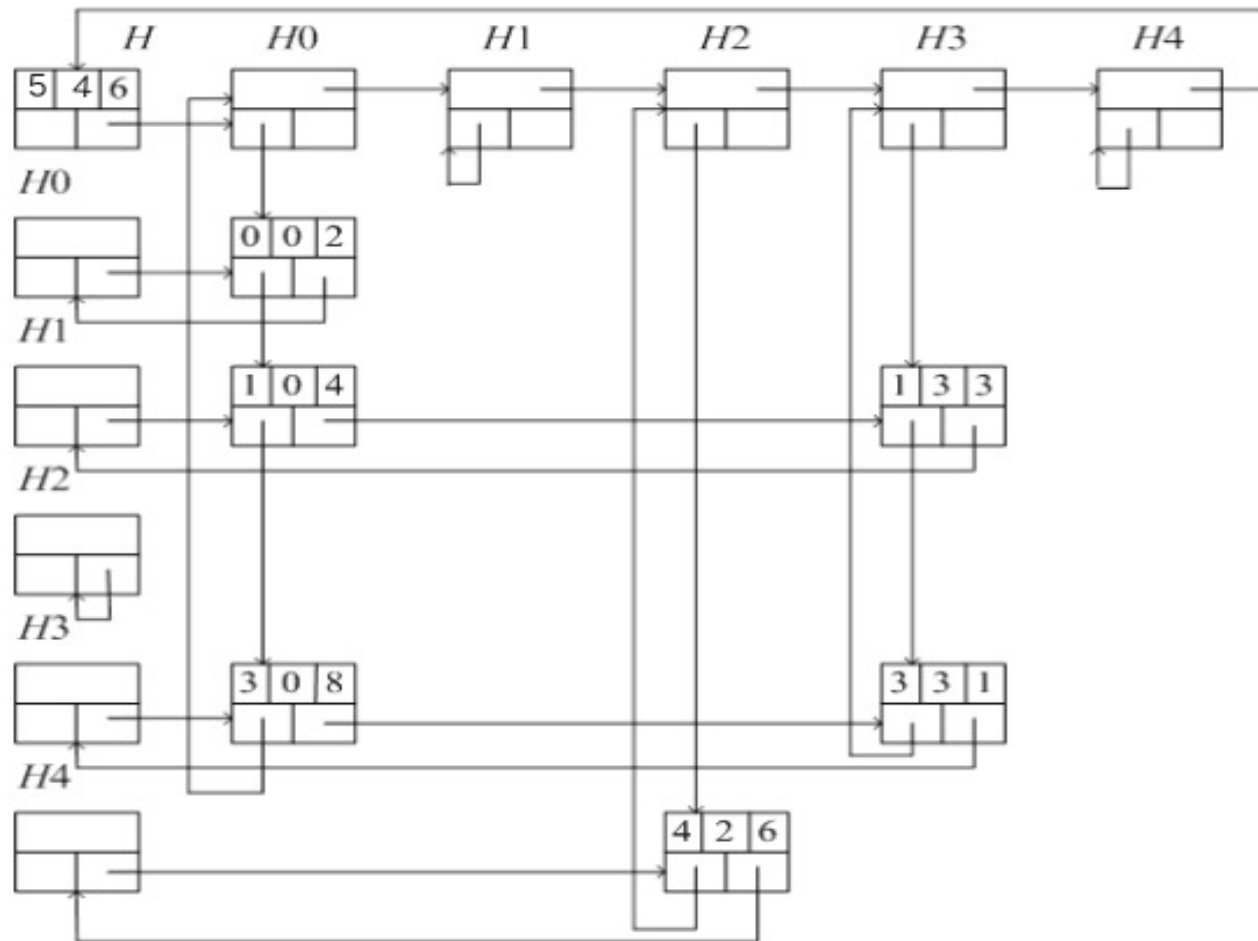
```
typedef struct matrixNode {  
    matrixPointer down;  
    matrixPointer right;  
    tagfield tag;  
    union  
    {  
        matrixPointer  
        next; entryNode  
        entry;  
    }; } u;
```

Example

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 4 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \\ 8 & 0 & 6 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

DATA STRUCTURES AND ITS APPLICATIONS

Sparse Matrix Representation: Linked representation





Sparse matrix representation

- Triple
- Linked Representation

Concepts can be applied to implement the following operations

- Create_SparseMatrix()
- Transpose_of_SparseMatrix()
- Add_SparseMatrices()
- Multiple_SparseMatrices()



1. Which of the following is a typical use-case of a multi-list data structure?

- a) Representing a polynomial equation with multiple variables.
- b) Maintaining a single linked list for multiple queues.
- c) Storing data where each node is linked to multiple independent lists, such as adjacency lists in graphs.
- d) Both (a) and (c).



1. Which of the following is a typical use-case of a multi-list data structure?

- a) Representing a polynomial equation with multiple variables.
- b) Maintaining a single linked list for multiple queues.
- c) Storing data where each node is linked to multiple independent lists, such as adjacency lists in graphs.
- d) Both (a) and (c).



2. Which of the following is a primary advantage of sparse matrix representation?

- a) It reduces the number of rows and columns.
- b) It stores only non-zero elements, reducing memory usage.
- c) It allows faster searching for zero elements.
- d) It increases redundancy.



2. Which of the following is a primary advantage of sparse matrix representation?

- a) It reduces the number of rows and columns.
- b) It stores only non-zero elements, reducing memory usage.
- c) It allows faster searching for zero elements.
- d) It increases redundancy.



3. For an $m \times n$ matrix with k non-zero elements, the 3-tuple representation requires how many rows in total?

- a) k rows.
- b) $k + 1$ rows.
- c) $k + 2$ rows.
- d) $m + n + k$ rows.



3. For an $m \times n$ matrix with k non-zero elements, the 3-tuple representation requires how many rows in total?

- a) k rows.
- b) $k + 1$ rows.
- c) $k + 2$ rows.
- d) $m + n + k$ rows.



4. To convert a regular matrix into a sparse matrix (3-tuple form), which of the following is correct?

- a) Traverse the matrix row-wise, and for each non-zero element, store (row, col, value) in the tuple.
- b) Traverse the matrix column-wise, ignoring zero elements.
- c) Only store diagonal elements.
- d) Store all elements in row-major order.



4. To convert a regular matrix into a sparse matrix (3-tuple form), which of the following is correct?

- a) Traverse the matrix row-wise, and for each non-zero element, store (row, col, value) in the tuple.
- b) Traverse the matrix column-wise, ignoring zero elements.
- c) Only store diagonal elements.
- d) Store all elements in row-major order.



5. In a linked list representation of a sparse matrix, each non-zero element is typically stored as:

- a) A node with (row, column, value) and pointers to the next non-zero element in the same row and column.
- b) A node with only the value and a single next pointer.
- c) A node with (row, column, value) but no pointers.
- d) An array of nodes representing each row.



5. In a linked list representation of a sparse matrix, each non-zero element is typically stored as:

- a) A node with (row, column, value) and pointers to the next non-zero element in the same row and column.
- b) A node with only the value and a single next pointer.
- c) A node with (row, column, value) but no pointers.
- d) An array of nodes representing each row.



Prof. Vandana M L

Department of Computer Science and Engineering

vandanamd@pes.edu



Data Structures and its Applications

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES & ITS APPLICATIONS

UNIT 1: Skip List

Kusuma K V

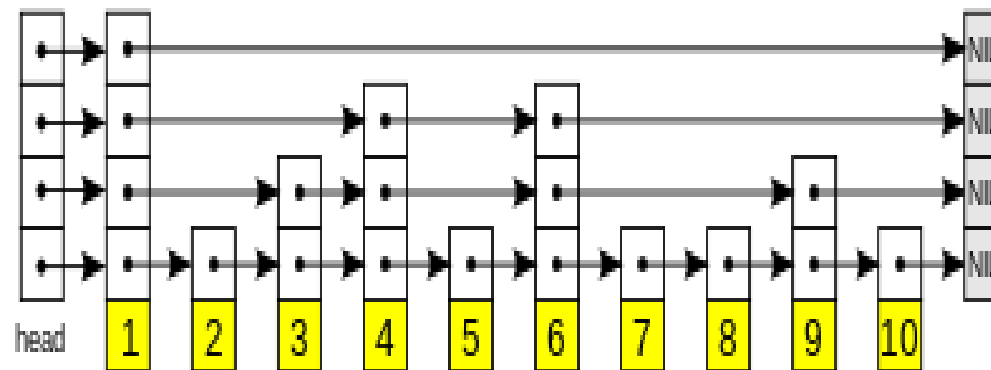
Department of Computer Science & Engineering

- `Size()`:
 - $O(1)$ if size is stored explicitly, else $O(n)$
- `IsEmpty()`:
 - $O(1)$
- `FindElement(k)`:
 - $O(n)$
- `InsertItem(k, e)`:
 - $O(1)$ (assumes item already found)
- `Remove(k)`:
 - $O(1)$ (assumes item already found)

- Skip Lists support $O(\log n)$
 - Insertion
 - Deletion
 - Search
- A relatively recent data structure
 - W. Pugh in 1989
- “A probabilistic alternative to balanced trees”

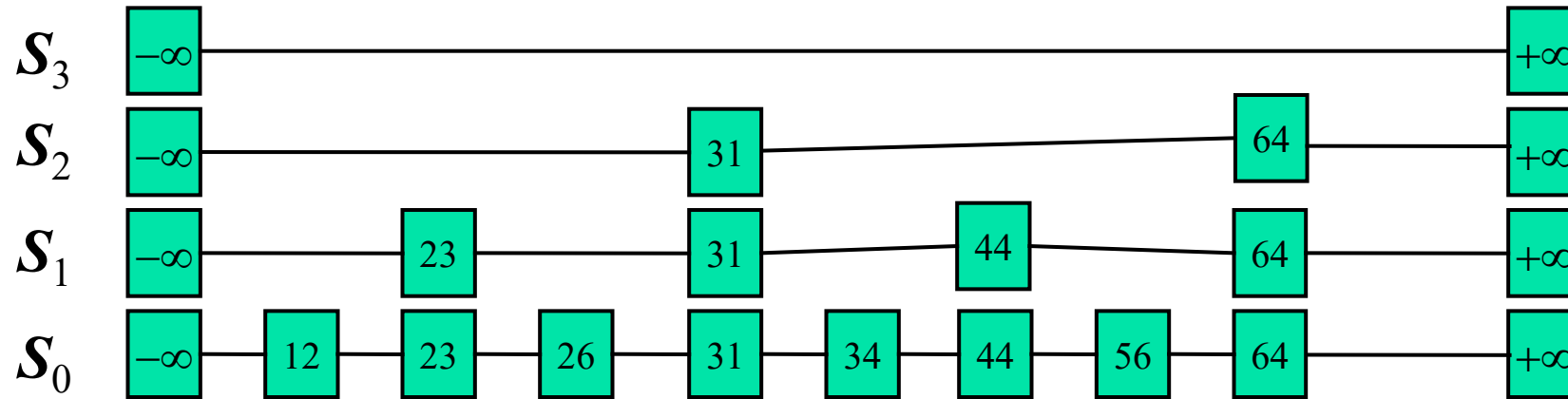
- A skip list is a probabilistic data structure that allows $O(\log n)$ search complexity as well as $O(\log n)$ insertion complexity within an ordered sequence of n elements
- Thus it can get the best features of a sorted array (for searching) while maintaining a linked list-like structure that allows insertion, which is not possible in an array

- Fast search is made possible by maintaining a linked hierarchy of sub sequences, with each successive subsequence skipping over fewer elements than the previous one
- Searching starts in the sparsest subsequence until two consecutive elements have been found, one smaller and one larger than or equal to the element searched for
- Via the linked hierarchy, these two elements link to elements of the next sparsest subsequence, where searching is continued until finally we are searching in the full sequence



- A skip list is a collection of lists at different levels
- The lowest level (0) is a sorted, singly linked list of all nodes
- The first level (1) links alternate nodes
- The second level (2) links every fourth node
- In general, level i links every 2^i th node
- In total, $\lceil \log_2 n \rceil$ levels (i.e. $O(\log_2 n)$ levels)
- Each level has half the nodes of the one below it

- Example of a Perfect Skip List

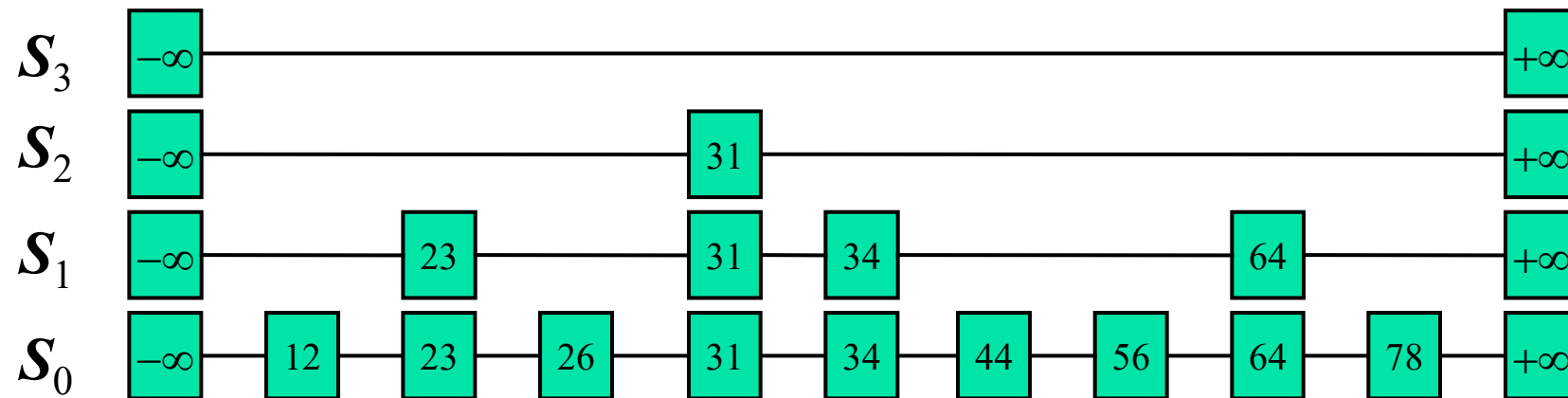


When we add a new node, our beautifully precise structure might become invalid

- We may have to change the level of every node
- One option is to move all the elements around
- But it takes $O(n)$ time, which is back where we began
- Is it possible to achieve a net gain?

Randomized Skip List

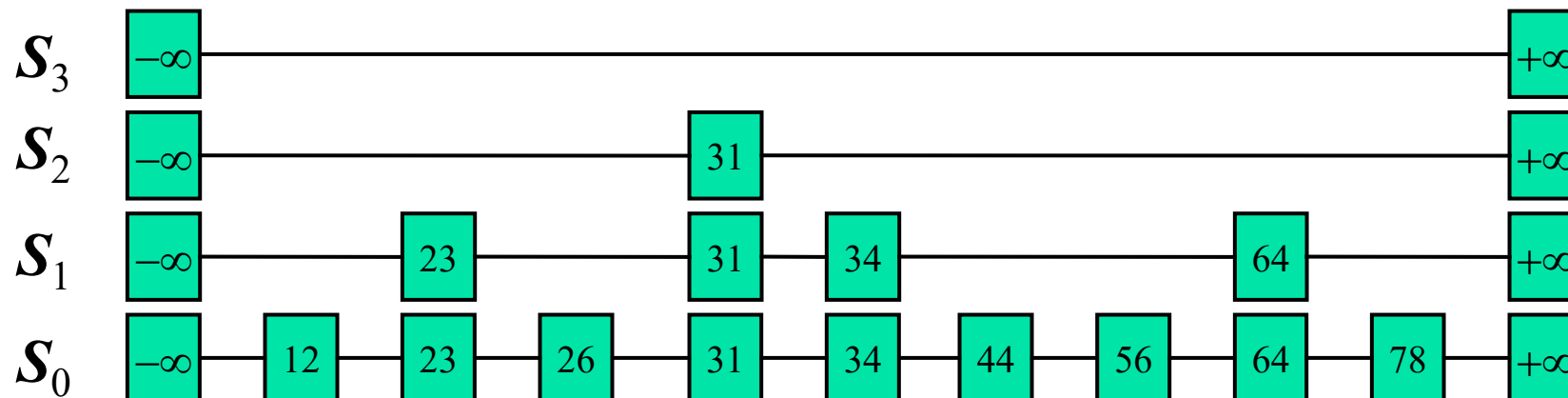
Example of a Randomized Skip List



Skip List

Skip List definition

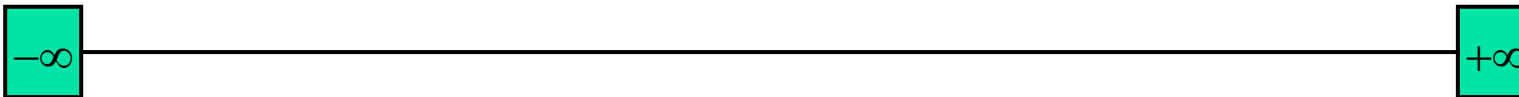
- A skip list for a set S of distinct (key, element) items is a series of lists $S_0, S_1, \dots, S_h$ such that:
 - Each list S_i contains the special keys $+\infty$ and $-\infty$
 - List S_0 contains the keys of S in non decreasing order
 - Each list is a subsequence of the previous one, i.e.,
$$S_0 \supseteq S_1 \supseteq \dots \supseteq S_h$$
- List S_h contains only the two special keys



Initialization

A new list is initialized as follows:

- A node NIL ($+\infty$) is created and its key is set to a value greater than the greatest key that could possibly be used in the list
- Another node NIL ($-\infty$) is created, value set to lowest key that could be used



- Presentation Slides: Advanced Data Structures
Dr. RamaMoorthy Srinath, Professor, CSE, PESU

1. **What is the main advantage of using a skip list over a balanced binary search tree?**
 - a) Skip lists require less memory.
 - b) Skip lists are easier to implement and provide probabilistic balancing without complex rotations.
 - c) Skip lists always guarantee $O(\log n)$ performance without exceptions.
 - d) Skip lists can store duplicate elements but BSTs cannot.

1. What is the main advantage of using a skip list over a balanced binary search tree?
 - a) Skip lists require less memory.
 - b) Skip lists are easier to implement and provide probabilistic balancing without complex rotations.
 - c) Skip lists always guarantee $O(\log n)$ performance without exceptions.
 - d) Skip lists can store duplicate elements but BSTs cannot.

2. What is the role of randomization in a skip list?

- a) It determines the position of nodes in the base linked list.
- b) It decides the height of the towers (levels) for each node.
- c) It avoids duplicate elements automatically.
- d) It ensures all keys are evenly spaced.

2. What is the role of randomization in a skip list?

- a) It determines the position of nodes in the base linked list.
- b) It decides the height of the towers (levels) for each node.
- c) It avoids duplicate elements automatically.
- d) It ensures all keys are evenly spaced.

3. Skip lists can be viewed as a combination of which two data structures?

- a) Linked List and Hash Table
- b) Linked List and Binary Tree
- c) Multiple Linked Lists stacked in levels
- d) Doubly Linked List and Heap

3. Skip lists can be viewed as a combination of which two data structures?

- a) Linked List and Hash Table
- b) Linked List and Binary Tree
- c) Multiple Linked Lists stacked in levels
- d) Doubly Linked List and Heap

4. Which of the following applications is best suited for skip lists?

- a) Implementing memory allocation.
- b) Designing a concurrent, lock-free, ordered data structure.
- c) Solving shortest path problems in graphs.
- d) Replacing arrays for random access.

4. Which of the following applications is best suited for skip lists?

- a) Implementing memory allocation.
- b) Designing a concurrent, lock-free, ordered data structure.
- c) Solving shortest path problems in graphs.
- d) Replacing arrays for random access.

7. Which of the following statements is false about skip lists?

- a) Skip lists use multiple levels of linked lists to improve search performance.
- b) The bottom level is a sorted linked list containing all elements.
- c) Skip lists guarantee $O(1)$ search in all cases.
- d) The height of a skip list grows logarithmically with the number of elements (on average).

7. Which of the following statements is false about skip lists?

- a) Skip lists use multiple levels of linked lists to improve search performance.
- b) The bottom level is a sorted linked list containing all elements.
- c) Skip lists guarantee $O(1)$ search in all cases.
- d) The height of a skip list grows logarithmically with the number of elements (on average).



THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu



Data Structures & Its Applications

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES & ITS APPLICATIONS

UNIT 1: Skip List

Kusuma K V

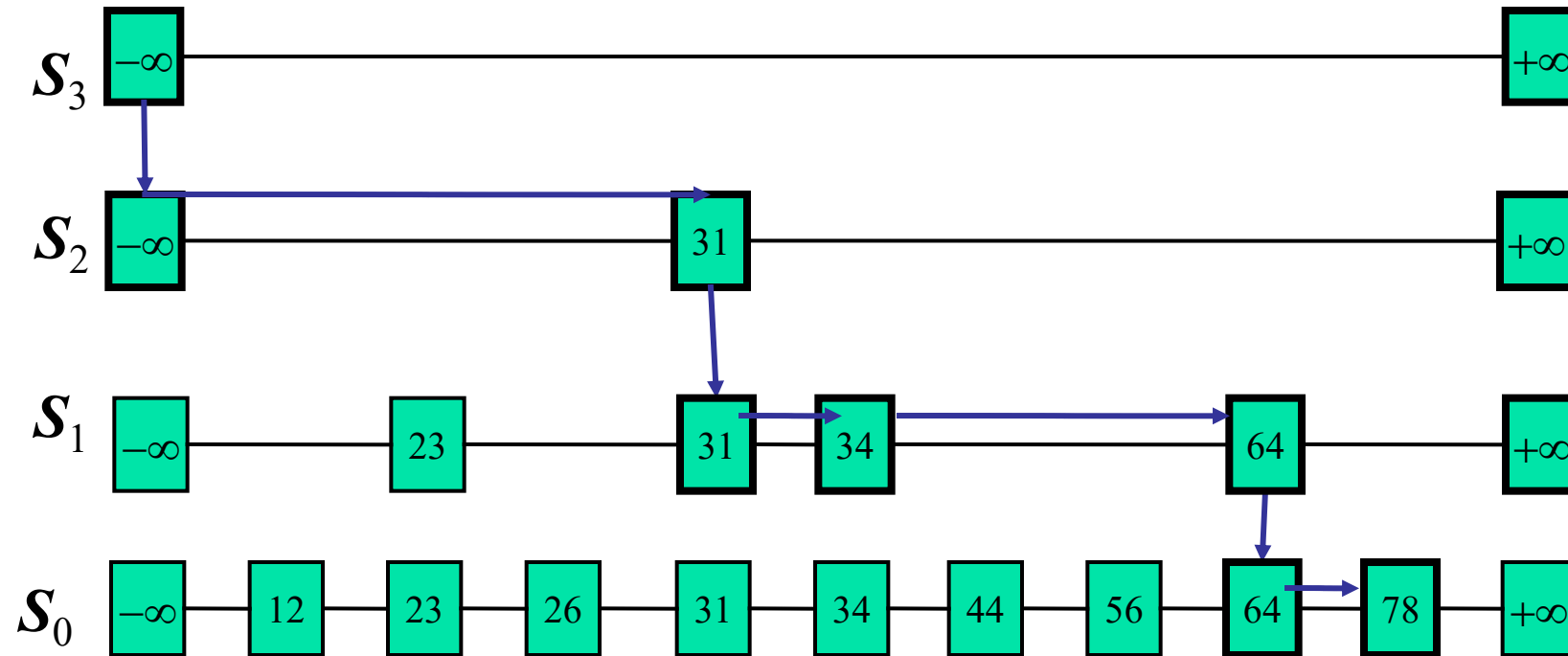
Department of Computer Science & Engineering

Search

We search for a **key x** in a skip list as follows:

- We start at the first position of the top list
- At the current position p, **we compare x with y i.e., $\text{key}(\text{after}(p))$**
 - $x = y$: we return $\text{element}(\text{after}(p))$**
 - $x > y$: we “scan forward”**
 - $x < y$: we “drop down”**
- **If we try to drop down past the bottom list, we return **NO_SUCH_KEY****
- Example: search for 78

Searching in Skip List Example



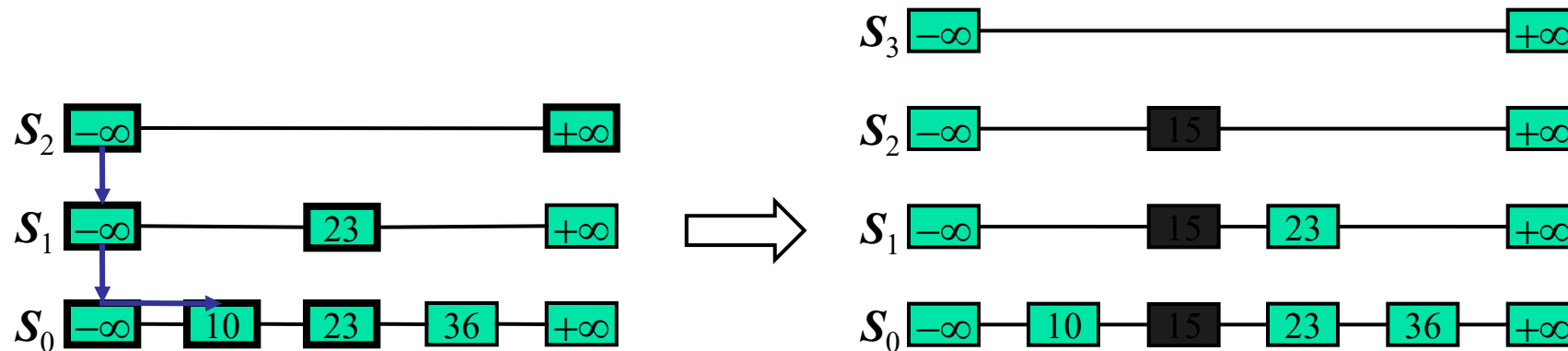
Insertion

- The insertion algorithm for skip lists uses randomization to decide how many references to the new item should be added to the skip list
- We then insert the new item in this bottom-level list at its appropriate position. After inserting the new item at this level we “flip a coin”
 - If the flip comes up tails, then we stop right there.
 - If the flip comes up heads, we move to next higher level and insert in this level at the appropriate position

- A randomized algorithm performs coin tosses (i.e., uses random bits) to control its execution
- Its running time depends on the outcomes of the coin tosses
- We analyze the expected running time of a randomized algorithm under the following assumptions
 - the coins are unbiased, and
 - the coin tosses are independent
- The worst-case running time of a randomized algorithm is large but has very low probability (e.g., it occurs when all the coin tosses give “heads”)

Insertion in Skip List Example

- Suppose we want to insert 15
- Do a search, and find the spot between 10 and 23
- Suppose the coin comes up “head” two times

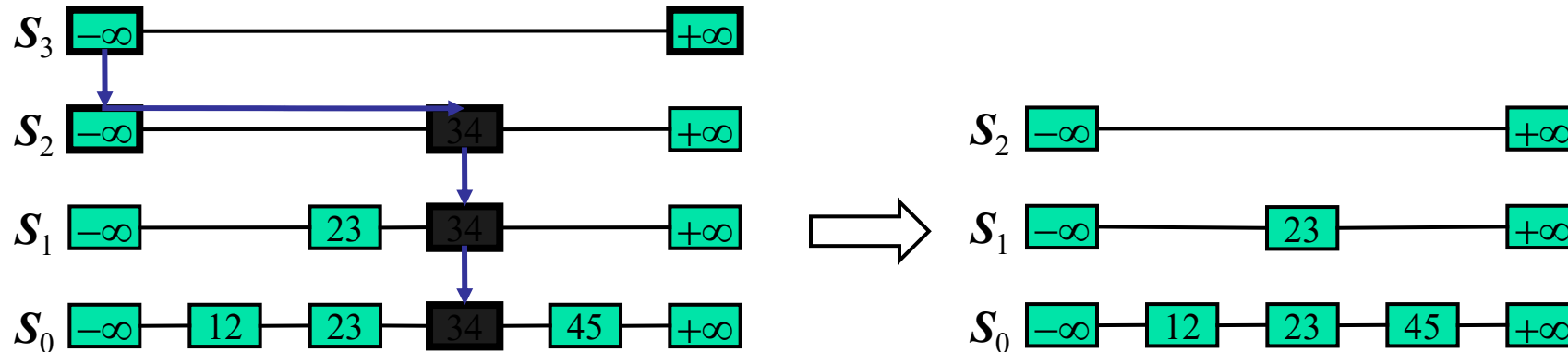


Deletion

- We begin by performing a search for the given key k.
- If a position p with key k is not found, then we return the **NO_SUCH_KEY** element
- Otherwise, if a position p with key k is found (it would be found on the bottom level), **then we remove all the position above p**
- If **more than one upper level is empty, remove it**

Deletion in Skip List Example

- Suppose we want to delete 34
- Do a search, find the spot between 23 and 45
- Remove all the position above p

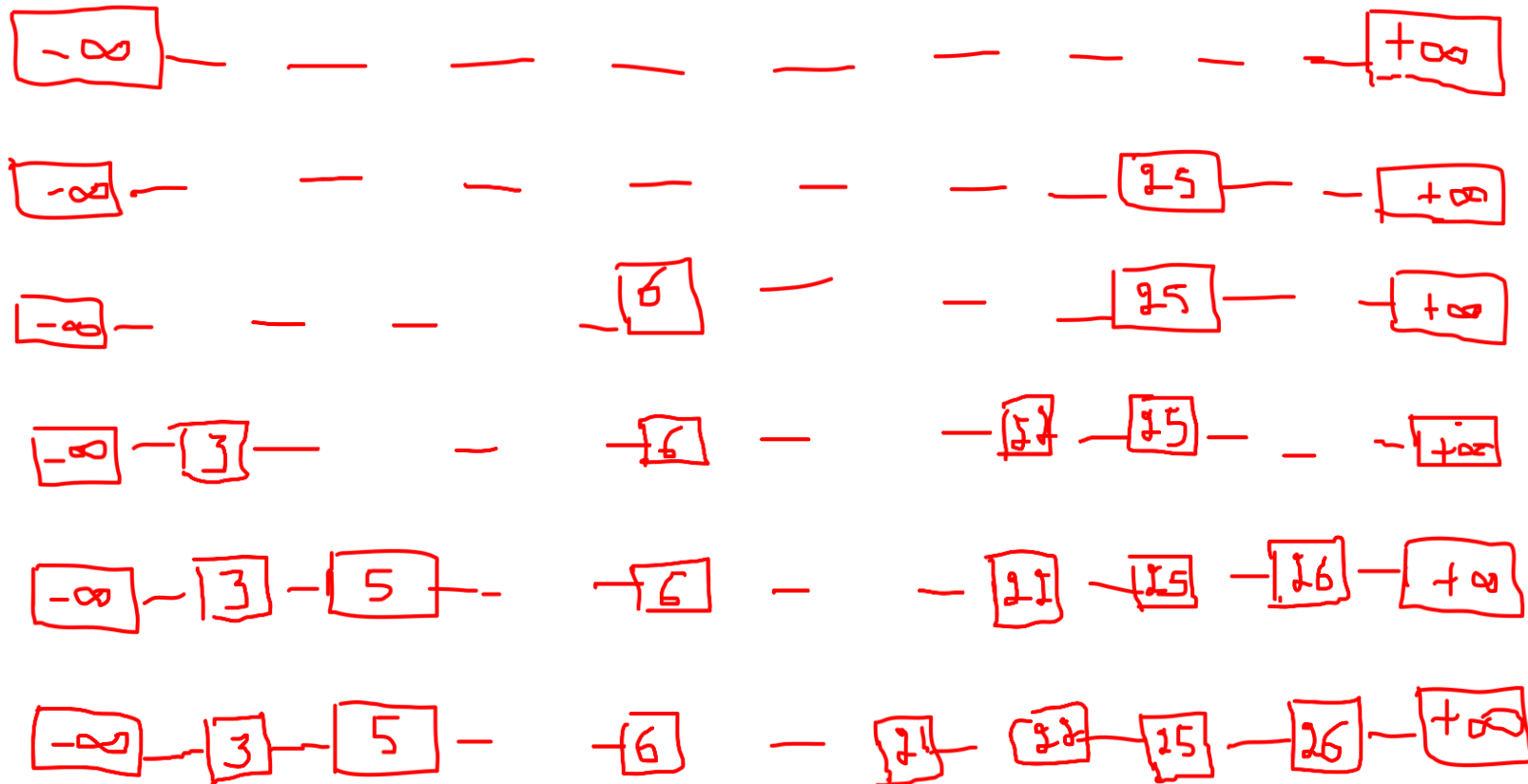


Starting with an empty skip list, insert these keys, with these “randomly generated” levels

- 5, with level 1
- 26, with level 1
- 25, with level 4
- 6, with level 3
- 21, with level 0
- 3, with level 2
- 22, with level 2

Note that the ordering of the keys in the skip list is determined by the value of the keys only; the levels of the nodes are determined by the random number generator only

5 with level 1, 26 with level 1, 25 with level 4, 6 with level 3
21 with level 0, 3 with level 2, 22 with level 2



- Presentation Slides: Advanced Data Structures
Dr. RamaMoorthy Srinath, Professor, CSE, PESU

1. While searching for key K in a skip list, what is the correct rule for horizontal movement at the current level?

- a) Move right while $\text{next.key} \leq K$.
- b) Move right while $\text{next.key} < K$; drop down when $\text{next.key} \geq K$ or $\text{next} == \text{NULL}$.
- c) Always move right exactly once per level.
- d) Drop down first, then move right to find K.

1. While searching for key K in a skip list, what is the correct rule for horizontal movement at the current level?
 - a) Move right while `next.key <= K`.
 - b) Move right while `next.key < K`; drop down when `next.key >= K` or `next == NULL`.
 - c) Always move right exactly once per level.
 - d) Drop down first, then move right to find K.

2. Which sequence best describes inserting a new key K?

- a) Random level → Allocate node → Search path → Link node in found level only.
- b) Search path (collect update[]) → Random level for node → Allocate node → Splice node into all levels $\leq$ nodeLevel.
- c) Allocate node → Splice at level 0 → Randomly add higher links later.
- d) Search path → Allocate node → Splice only at top level.

2. Which sequence best describes inserting a new key K?

- a) Random level → Allocate node → Search path → Link node in found level only.
- b) Search path (collect update[]) → Random level for node → Allocate node → Splice node into all levels $\leq$ nodeLevel.
- c) Allocate node → Splice at level 0 → Randomly add higher links later.
- d) Search path → Allocate node → Splice only at top level.

3. How is the level (height) of a newly inserted node typically determined?

- a) Fixed at 1 for all nodes.
- b) Count set bits in key value.
- c) Repeatedly “flip a coin” with probability p of going up a level until fail or max level reached.
- d) Proportional to current list size ($\lceil \log_2 n \rceil$).

3. How is the level (height) of a newly inserted node typically determined?

- a) Fixed at 1 for all nodes.
- b) Count set bits in key value.
- c) Repeatedly “flip a coin” with probability p of going up a level until fail or max level reached.
- d) Proportional to current list size ($\lceil \log_2 n \rceil$).

4. To delete key K, which approach is correct?

- a) Search only level 0; unlink node; done.
- b) Search from top without storing predecessors; when found, drop to level 0 and delete.
- c) Search building update[] of last nodes $< K$ at each level; if found, relink $\text{update}[i] \rightarrow \text{forward}[i]$ to skip target at all involved levels; free node.
- d) Mark node deleted; let garbage collector collapse levels.

4. To delete key K, which approach is correct?

- a) Search only level 0; unlink node; done.
- b) Search from top without storing predecessors; when found, drop to level 0 and delete.
- c) Search building update[] of last nodes $< K$ at each level; if found, relink $update[i] \rightarrow forward[i]$ to skip target at all involved levels; free node.
- d) Mark node deleted; let garbage collector collapse levels.

5. After deleting a node, when (if ever) should the current highest level of the skip list be reduced?

- a) Never; levels are fixed after first insertion.
- b) Whenever any node is deleted.
- c) When the header's forward pointer at the top level becomes NULL (no elements at that level).
- d) When $\text{current size} < \text{previous size}/2$.

5. After deleting a node, when (if ever) should the current highest level of the skip list be reduced?

- a) Never; levels are fixed after first insertion.
- b) Whenever any node is deleted.
- c) When the header's forward pointer at the top level becomes NULL (no elements at that level).
- d) When $\text{current size} < \text{previous size}/2$.



THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Stacks

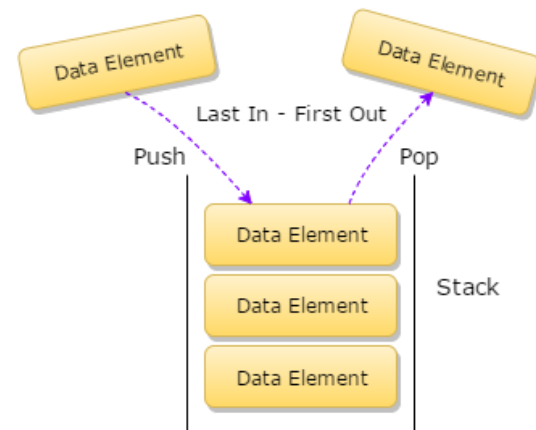
Dinesh Singh

Department of Computer Science & Engineering

Data Structures and its Applications

Stacks - Definition

- A Stack is a data Structure in which all the insertions and deletions of entries are at one end. This end is called the TOP of the stack.
- When an item is added to a stack it is called push into the stack
- When an item is removed it is called pop from the stack.
- The Last item pushed onto a stack is always the first that will be popped from the stack.
- This property is called the *last in, first out* or LIFO for short



Data Structures and its Applications

Stacks – Representation in C

A stack in C is declared as a structure containing two objects :

- An array to hold the elements of the stack
- An Integer to indicate the position of the current stack top within the array
- Stack of integers can be done by the following declaration

```
#define STACKSIZE 100
```

```
struct stack
```

```
{
```

```
    int top;
```

```
    int items[STACKSIZE]
```

```
};
```

Once this is done, actual stack can be declared by

```
struct stack s;
```

Data Structures and its Applications

Stacks – Representation in C

Items need not be restricted to integers, items can be of any type.

A stack can contain items of different types by using C unions.

```
#define STACKSIZE 100
```

```
#define INT 1
```

```
#define FLOAT 2
```

```
#define STRING 3
```

```
struct stackelement {
```

```
    int etype;
```

```
    union{
```

```
        int ival;
```

```
        float fval;
```

```
        char *pavl; //pointer to string
```

```
    } element;
```

```
};
```

Data Structures and its Applications

Stacks – Representation in C

```
struct stack
```

```
{  
    int top;  
    struct stackelement items[STACKSIZE];  
};
```

- The above declaration defines a stack whose items can either be integers, floating point numbers or string depending on the value of etype (previous slide).

Data Structures and its Applications

Stacks – Implementation of operations of stack

Operations on stack

- Inserting an element on to the stack : push
- Deleting an element from the stack : pop
- Checking the top element : peep
- Checking if the stack is empty : **empty**
- Checking if the stack is full : **overflow**

Representation of stack will be as follows

```
#define STACKSIZE 100
```

```
struct stack
```

```
{
```

```
    int top;
```

```
    int items[STACKSIZE]
```

```
};
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

```
void push(struct stack *ps, int x)
```

```
/*ps is pointer to the structure representing stack, x is integer to be inserted  
top is integer that indicates the position of the current stack top within the  
array, items is an integer array that represents stack, STACK_SIZE is the  
maximum size of the stack */
```

```
{  
    if (ps->top == STACKSIZE -1) //check if the stack is full  
        printf("STACK FULL Cannot insert..");  
    else  
    {  
        ++(ps->top); //increment top  
        ps->items[ps->top]=x; //insert the element at a location top  
    }  
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

```
int pop(struct stack *ps )
```

```
/*ps is pointer to the structure representing stack, top is integer that indicates  
the position of the current stack top within the array , items is an integer  
array that represents stack, STACK_SIZE is the maximum size of the stack */
```

```
{
```

```
if (ps->top == -1) // check if the stack is the empty
```

```
    printf("STACK EMPTY Cannot DELETE..");
```

```
else
```

```
{
```

```
    x=ps->items[ps->top]; //delete the element
```

```
    --(ps->top); //decrement top
```

```
    return x;
```

```
}
```

```
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

```
int display(struct stack *ps )
```

```
/*ps is pointer to the structure representing stack, top is integer that indicates  
the position of the current stack top within the array , items is an integer  
array that represents stack, STACK_SIZE is the maximum size of the stack */
```

```
{
```

```
if (ps->top == -1) // check if the stack is the empty
```

```
    printf("STACK EMPTY ");
```

```
else
```

```
{
```

```
    for (i=ps->top;i>=0;i--) // displays the elements from top
```

```
        printf("%d",ps->items[i]);
```

```
}
```

```
}
```


Data Structures and its Applications

Stacks – Implementation of operations of stack

```
int peep(struct stack *ps )
{
    if (ps->top == -1)
        printf("STACK EMPTY ..");
    else
    {
        x=ps->items[ps->top]; //get the element
        return x;
    }
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

```
int empty(struct stack *ps )
{
    if (ps->top == -1)
        return 1;
    return 0;
}

int overflow(struct stack *ps)
{
    if (ps->top==STACKSIZE-1)
        return 1;
    return 0;
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

implementation of stack operations where the items array and top are separate variables (Not part of structure)

```
void push(int *s, int *top, int x)
{
    if(*top==STACKSIZE-1)//check if the stack is full
    {
        printf("stack overflow..cannot insert");
        return 0;
    }
    else
    {
        ++*top; //increment the top
        s[*top]=x; //insert the element
    }
    return 1;
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

- Implementation of stack operations where the items and the top are separate variables (Not part of structure)

```
int pop(int *s, int *top)
{
    if(*top==-1)//check if the stack is empty
    {
        printf("Stack empty .. Cannot delete");
        return -1;
    }
    else
    {
        x=s[*top]; //insert the element
        --*top; //decrement top
        return x; // return the deleted element
    }
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

- Implementation of stack operations where the items and the top are separate variables (Not part of structure)

```
display(int *s, int *top)
{
    if(*top==-1)
        printf("Empty stack");
    else
    {
        for(i=*top;i>=0;i--) // display the elements from the top
            printf("%d",s[i]);
    }
}
```

Data Structures and its Applications

Stacks – Application of Stack

- Write an algorithm to determine if an input character string is of the form $x C y$ where x is a string consisting of the letters 'A' and 'B' and where y is the reverse of x . At each point you may read only the character of the string

```
int check(t)
```

```
//the function returns 1 if string t is of the form x C y, else returns 0
```

```
//uses stack s and its operations push and pop
```

```
{
```

```
    i=0;
```

```
    while(t[i]!='C') //push all the characters of the string into the stack
```

```
until C is encountered
```

```
{
```

```
    push(&s, t[i]);
```

```
    i=i+1;
```

```
}
```

- 1. If a stack is implemented using a fixed-size array of size N , which of the following statements is true regarding overflow conditions?**
- a) Overflow occurs when $\text{top} = N-1$.
 - b) Overflow occurs when $\text{top} = N$.
 - c) Overflow depends on the element type, not the array size.
 - d) Overflow never occurs in an array-based stack.

1. If a stack is implemented using a fixed-size array of size N , which of the following statements is true regarding overflow conditions?

- a) Overflow occurs when $\text{top} = N-1$.
- b) Overflow occurs when $\text{top} = N$.
- c) Overflow depends on the element type, not the array size.
- d) Overflow never occurs in an array-based stack.

2. When two stacks are implemented in a single array (sharing memory from opposite ends), under what condition is the array completely full?

a) $\text{top1} + \text{top2} = N$

b) $\text{top2} - \text{top1} = 1$

c) $\text{top1} == \text{top2}$

d) $\text{top1} == \text{top2} + 1$

2. When two stacks are implemented in a single array (sharing memory from opposite ends), under what condition is the array completely full?

a) $\text{top1} + \text{top2} = N$

b) $\text{top2} - \text{top1} = 1$

c) $\text{top1} == \text{top2}$

d) $\text{top1} == \text{top2} + 1$

Data Structures and its Applications

Multiple-Choice-Questions(MCQ)

3. Which of the following is stored in a stack frame during a function call in C?

- a) Return address only.
- b) Local variables, return address, and saved registers.
- c) Global variables and return address.
- d) Only function parameters.

3. Which of the following is stored in a stack frame during a function call in C?

- a) Return address only.
- b) Local variables, return address, and saved registers.
- c) Global variables and return address.
- d) Only function parameters.

4. Which of the following applications cannot be implemented using a stack?

- a) Undo/Redo feature in text editors.
- b) Parenthesis matching in expressions.
- c) Level-order traversal of a binary tree.
- d) Evaluating postfix expressions.

4. Which of the following applications cannot be implemented using a stack?

- a) Undo/Redo feature in text editors.
- b) Parenthesis matching in expressions.
- c) Level-order traversal of a binary tree.
- d) Evaluating postfix expressions.



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Stacks – Linked List Implementation

Dinesh Singh

Department of Computer Science & Engineering

- A stack can be easily implemented through the linked list. In the Implementation by a linked list, the stack contains a top pointer. which is “head” of the stack. The pushing and popping of items happens at the head of the list.

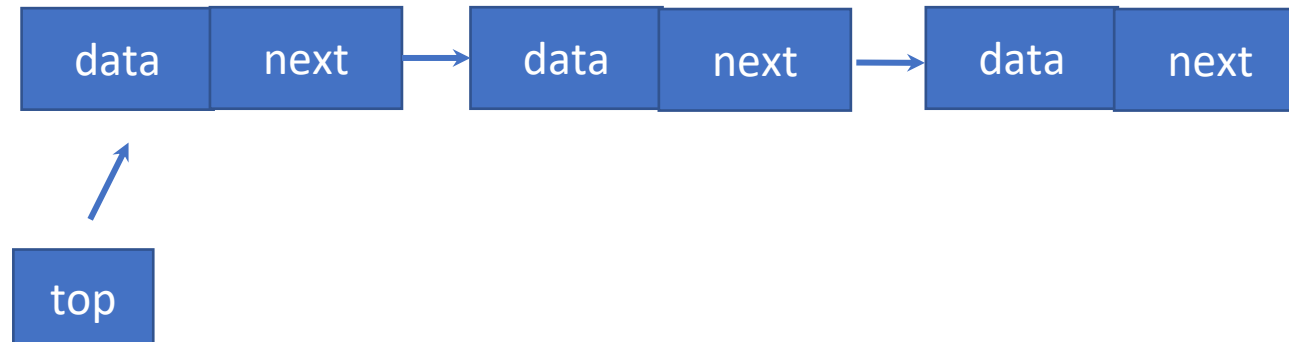
Structure of the stack

struct node

```
{  
    int data;  
    struct node *next;  
}
```

struct node *top

top=NULL;



Note :

- Items of the stack represented as the linked list.
- Each item is a node
- Top is a pointer that points to the first node (top of the stack)
- Top is initially NULL (Empty stack)
- Insertion and deletion happens at the front of the list
- stack size is not limited.

Operations on the stack

- Push : Inserting an element at the front of the list
- Pop : delete an element from the front of the list
- Display : displaying the list

Data Structures and its Applications

Stacks – linked list implementation



//implements the push operation

```
void push(int x, struct node **top)
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=*top; // insert in front of the list
    *top=temp; // make top points to the new top node
}
```

Data Structures and its Applications

Stacks – linked list implementation



//implements the pop operation
//returns top element, -1 if stack is empty

```
int pop(struct node **top)
{
    int x;
    struct node *q;

    if(*top==NULL)
    {
        printf("Empty Stack\n");
        return -1;
    }
}
```

//implements the pop operation

else

```
{  
    q=*top;  
    x=q->data; // get the top element  
    *top=q->next; // make top point to the next top  
    free(q); // free the memory of the node  
    return(x);  
}  
}
```

Data Structures and its Applications

Stacks – linked list implementation



//implements the display operation

```
void display(struct node *top)
{
    if(top==NULL)
        printf("Empty Stack\n");
    else
    {
        while(top!=NULL)
        {
            printf("%d->",top->data);
            top=top->next;
        }
    }
}
```

Another representation of structure of stack

struct node

```
{  
    int data;  
    struct node *next;  
};
```

struct stack

```
{  
    struct node *top;  
}
```

struct stack s;

s.top=NULL;

Data Structures and its Applications

Stacks – linked list implementation



//implements the push operation

void push(int x, struct stack * s)

//s is pointer to structure stack

{

struct node *temp;

temp=(struct node*)malloc(sizeof(struct node));

temp->data=x;

temp->next=s->top; // insert in front of the list

s->top=temp; // make top points to the new top node

}

Data Structures and its Applications

Stacks – linked list implementation



//implements the pop operation
//returns top element, -1 if stack is empty

```
int pop(struct stack *s)
{
    int x;
    struct node *q;

    if(s->top==NULL)
    {
        printf("Empty Stack\n");
        return -1;
    }
}
```

//implements the pop operation

```
else
{
    q=s->top;
    x=q->data; // get the top element
    s->top=q->next; // make top point to the next top
    free(q); // free the memory of the node
    return(x);
}
```

Data Structures and its Applications

Stacks – linked list implementation



//implements the display operation

```
void display(struct stack *s)
{
    struct node *q;
    if(s->top==NULL)
        printf("Empty Stack\n");
    else
    {
        q=s->top;
        while(q!=NULL)
        {
            printf("%d->",q->data);
            q=q->next;
        }
    }
}
```

Write an Algorithm to print a string in the reverse order

//prints the text in a reverse order

reverse(t)

```
{  
    i=0;  
//push all the characters on to the stack  
    while(t[i]!='\0')  
    {  
        push(s,t[i]);  
        i=i+1;  
    }
```

pop all the characters from the stack until the stack is empty

```
while(!empty(s))  
{  
    x= pop(s);  
    print(x)  
}  
}
```

- 1. In a stack implemented using a linked list, where is the top of the stack maintained?**
- a) At the head of the linked list
 - b) At the tail of the linked list
 - c) At the middle node
 - d) It depends on the implementation

1. In a stack implemented using a linked list, where is the top of the stack maintained?
- a) At the head of the linked list
 - b) At the tail of the linked list
 - c) At the middle node
 - d) It depends on the implementation

2. What is the time required to perform the push() operation in a stack implemented using a linked list?

- a) Constant time
- b) Logarithmic time
- c) Linear time
- d) Quadratic time

2. What is the time required to perform the push() operation in a stack implemented using a linked list?

- a) **Constant time**
- b) Logarithmic time
- c) Linear time
- d) Quadratic time

3. What happens when pop() is called on an empty stack implemented using a linked list?

- a) It deletes a NULL node
- b) It returns garbage value
- c) It results in underflow condition
- d) It resets the top pointer to head

3. What happens when pop() is called on an empty stack implemented using a linked list?

- a) It deletes a NULL node
- b) It returns garbage value
- c) It results in underflow condition
- d) It resets the top pointer to head

4. Which of the following is true about a stack implemented using a linked list vs. an array?

- a) Linked list implementation is faster for push() and pop()
- b) Array implementation requires dynamic memory allocation
- c) Linked list implementation has no fixed size limit (until memory is exhausted)
- d) Both a and c

4. Which of the following is true about a stack implemented using a linked list vs. an array?

- a) Linked list implementation is faster for push() and pop()
- b) Array implementation requires dynamic memory allocation
- c) Linked list implementation has no fixed size limit (until memory is exhausted)**
- d) Both a and c

5. Which of the following is correct for push() operation in a linked list implementation?

- a) Create a new node and add it at the end
- b) Create a new node and add it at the beginning, change top (pointer)
- c) Move the tail pointer backward
- d) None of the above

5. Which of the following is correct for push() operation in a linked list implementation?

- a) Create a new node and add it at the end
- b) Create a new node and add it at the beginning, change top (pointer)**
- c) Move the tail pointer backward
- d) None of the above



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Application of stacks– Functions, nested functions and Recursion

Dinesh Singh

Department of Computer Science & Engineering

Application of stacks – Functions, nested functions

Activation record :

- When **functions are called**, The system (or the program) **must remember the place where the call was made**, so that it can **return there after the function is complete**.
- It must also remember all the **local variables, processor registers, and the like**, so that information will not be lost while the function is working.
- This information is considered as **large data structure** . This structure is sometimes called the **invocation record or the activation record** for the function call.

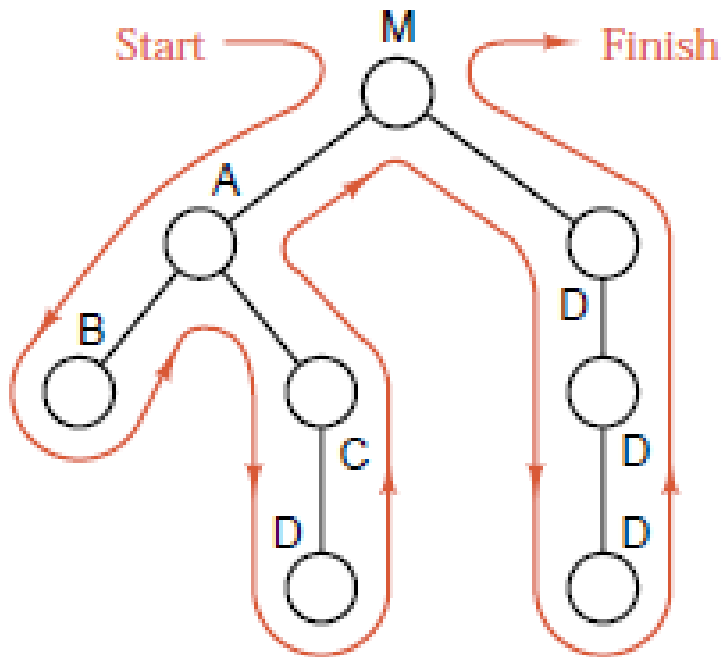
Application of stacks – Functions, nested functions and Recursion

- Suppose that there are three functions called A,B and C. M is the main function.
- Suppose that A invokes B and B invokes C. Then B will not have finished its work until C has finished and returned. Similarly, A is the first to start work, but it is the last to be finished, not until sometime after B has finished and returned.
- Thus the sequence by which function activity proceeds is summed up as the property last in, first out.
- The machine's task of assigning temporary storage area (activation records) used by functions would be in same order (LIFO).
- Since LIFO property is used, the machine allocates these records in the stack
- Hence a stack plays a key role in invoking functions in a computer system.

Application of stacks – Functions, nested functions

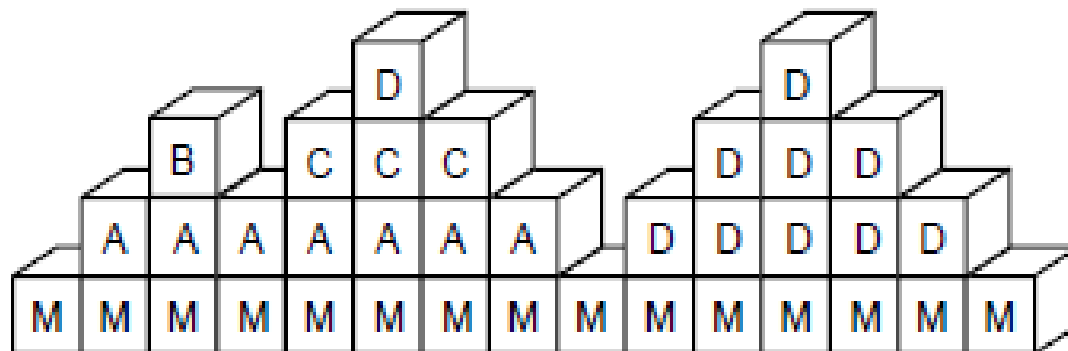
Example :

Consider the following showing the order in which the functions are invoked

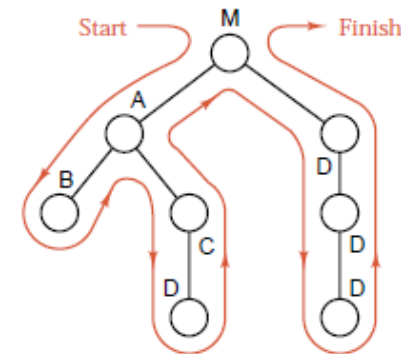


Application of stacks – Functions, nested functions

The Sequence of stack frames of the activation records of the function calls is given below. Each vertical column shows the contents of the stack at a given time, and changes to the stack are portrayed by reading through the frames from left to right.



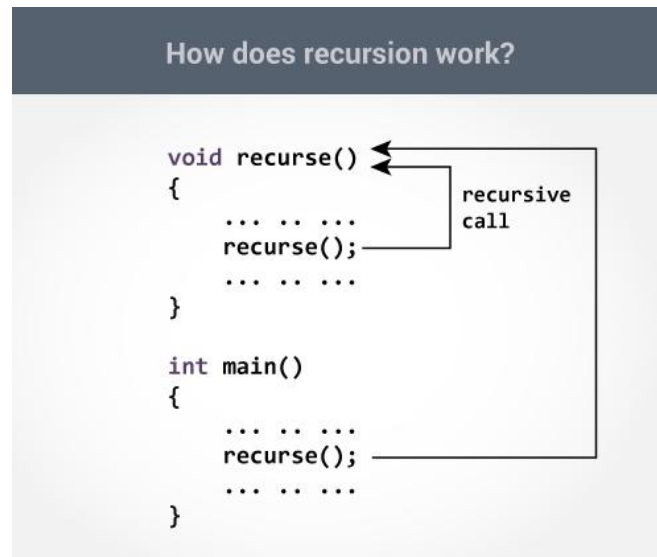
Time →



↑
Stack
space
for
data

Recursion :

- Recursion is a computer programming technique involving the use of a procedure, subroutine or a function.
- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function.



How a particular problem is solved using recursion?

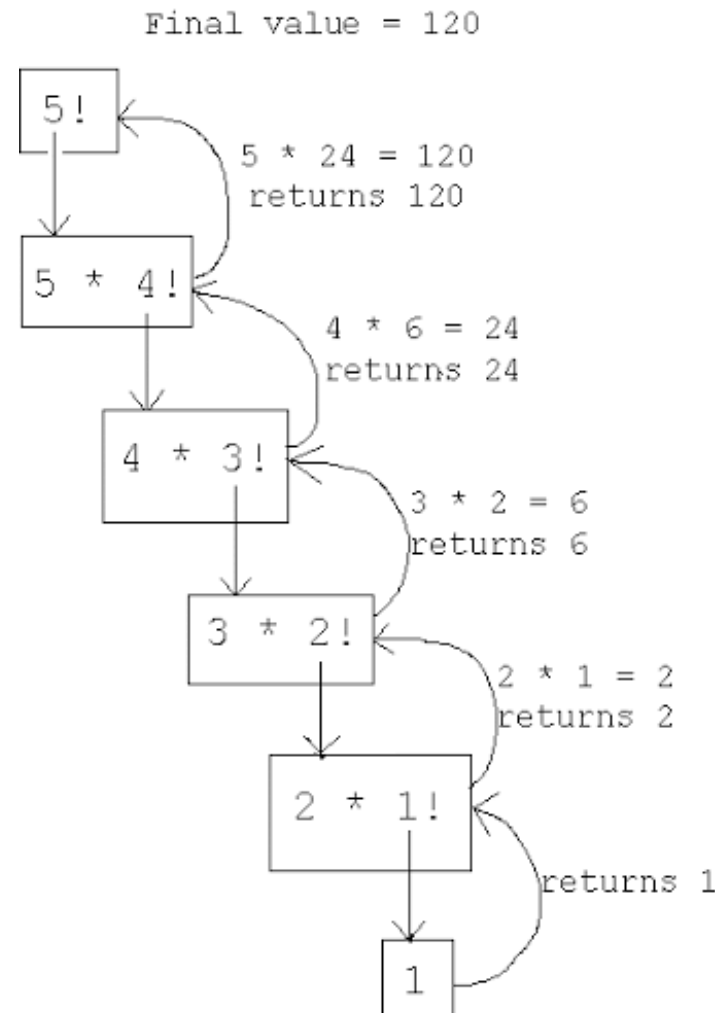
- The idea is to represent a problem in terms of one or more smaller problems.
- A way is found to solve these smaller problems and then build up a solution to the entire problem.
- The sub problems are in turn broken down into smaller sub problems until the solution to the smallest sub problem is known.
- The solution to the smallest sub problem is called the base case.
- In the recursive program, the solution to the base case is provided

Example 1: Factorial of a Number n

Recursive definition :

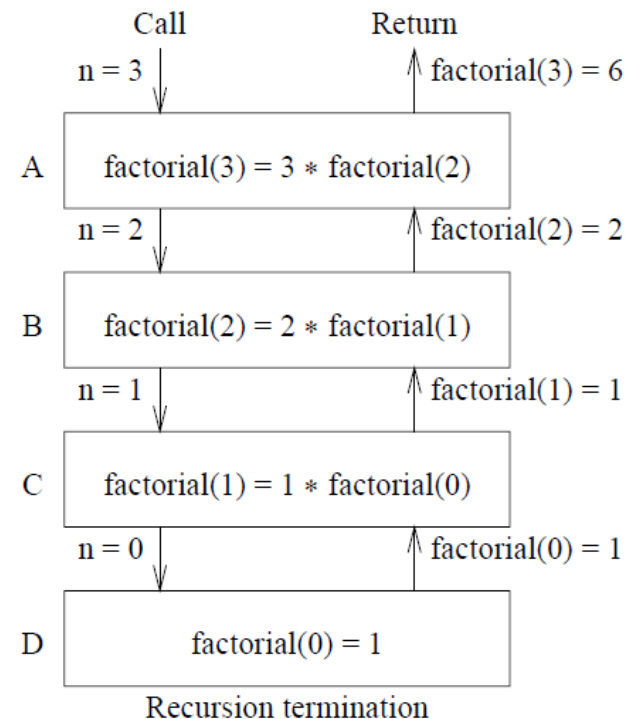
$n! = 1$ if $n=1$

$n! = n * (n-1)!$ if $n > 1$



Recursive function to compute factorial of n

```
factorial(n)
{
    int f;
    if(n==0)
        return 1;
    f=n*factorial(n-1);
    return f;
}
```



(a)

Activation record for A
Activation record for B
Activation record for C
Activation record for D

(b)

Recursive function to find the product of $a*b$

$a*b = a$ if $b=1$;

$a*b = a*(b-1)+a$ if $b > 1$;

To evaluate $6 * 3$

$6*3 = 6*2 + 6 = 6*1 + 6 + 6 = 6 + 6 + 6 = 18$

```
multiply(int a, int b)
```

```
{
```

```
    int p;
```

```
    if (b==1)
```

```
        return a
```

```
    p= multiply(a,b-1) + a;
```

```
    return p;
```

```
}
```

Fibonacci Sequence

The fibonacci sequence is the sequence of integers
1, 1, 2, 3, 5, 8, 13, 21, 34...

Each element is the sum of two preceding elements

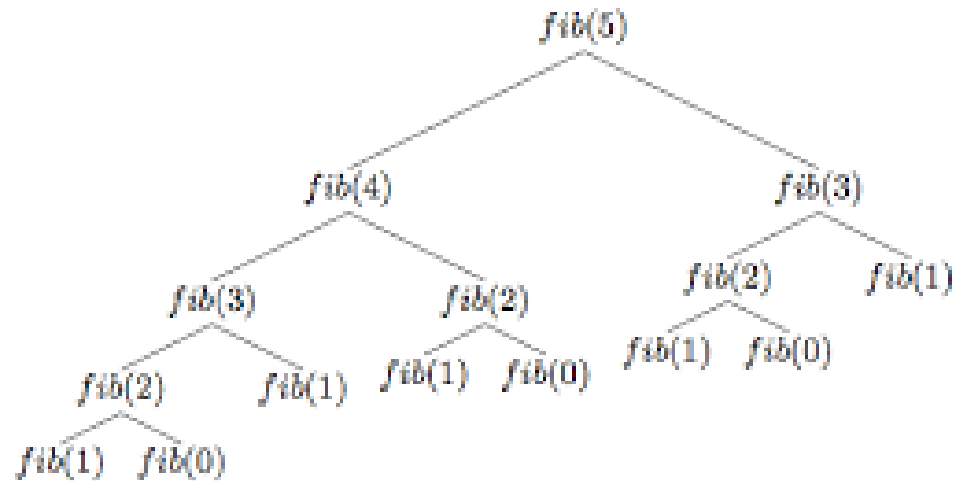
If we consider $\text{fib}(0) = 1$ and $\text{fib}(1)=1$
then recursive definition to compute the n th element in the sequence is

$\text{fib}(n) = n$ if $n=0$ or $n=1$

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ for $n \geq 2$

Recursive function to compute the nth element in the Fibonacci Sequence

```
fib(int n)
{
    int x,y;
    if ( (n==0) || (n==1) )
        return n;
    x= fib(n-1) ;
    y=fib(n-2);
    return x+y;
}
```



Recursive function to find the sum of elements of an array

```
int sum(int *a, int n)
```

```
//a is pointer to the array, n is the index of the last element of the array
```

```
{
```

```
    int s;
```

```
    if(n==0) // base condition
```

```
        return a[0];
```

```
    s= sum(a,n-1) + a[n]; // compute sum of n-1 elements and add the nth element
```

```
    return s;
```

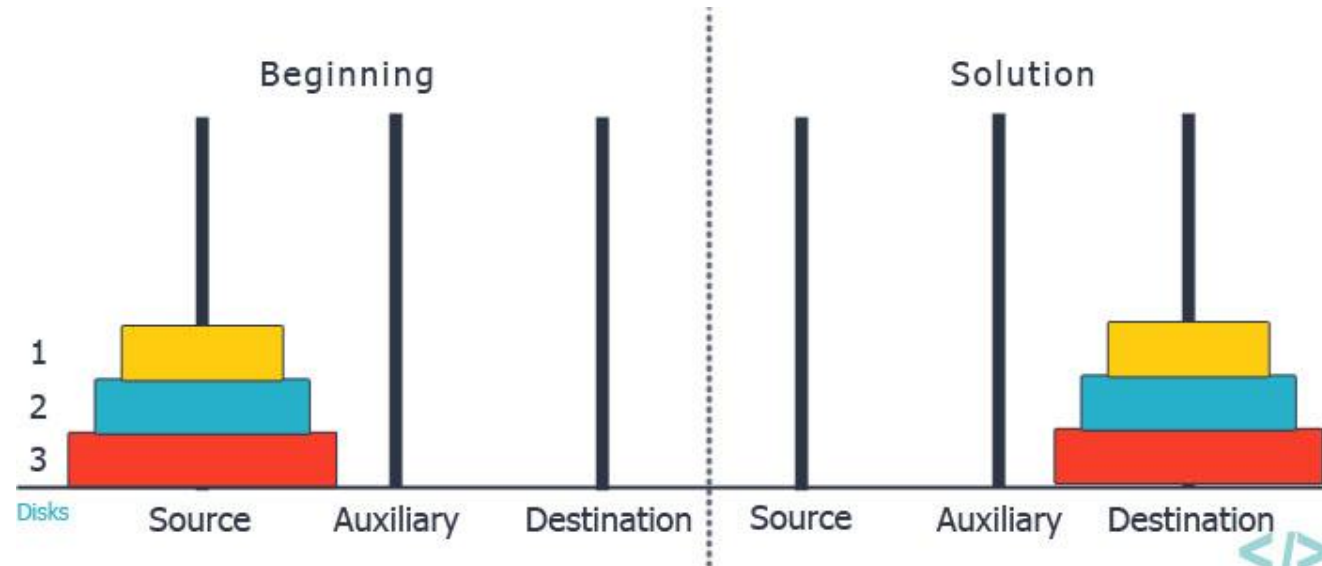
```
}
```

Recursive function to display the elements of the linked list in the reverse order

```
int display(struct node *p)
{
    if(p->next!=NULL)
        display(p->next)
    printf("%d ",p->data);
}
```


Tower of Hanoi

Three Pegs A, B and C exists. N disks of differing diameters are placed on peg A. The Larger disk is always below a smaller disk. The objective is to move the N disks from Peg A to Peg C using Peg B as the auxillary peg



Tower of Hanoi – recursive solution

If a solution to $n-1$ disks is found, then the problem would be solved. Because in the trivial case of one disk, the solution would be to move the single disk from Peg A to Peg C.

To move n disks from A to C , the recursive solution would be as follows

1. If $n=1$ move the single disk from A to C and stop
2. Move the top $n-1$ disks from A to B using C as auxillary
3. Move the remaining disk from A to C
4. Move $n-1$ disks from B to C using A as the auxillary

Recursive function for Tower of Hanoi

```
void tower(int n,char src,char tmp,char dst)
{
    if(n==1)
    {
        printf("\nMove disk %d from %c to %c",n,src,dst);
        return;
    }
    tower(n-1,src,dst,tmp);
    printf("\nMove disk %d form %c to %c",n,src,dst);
    tower(n-1,tmp,src,dst);
    return;
}
```

Recursive function for Tower of Hanoi

Solution for 4 disks

Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 3 from peg A to peg B
Move disk 1 from peg C to peg A
Move disk 2 from peg C to peg B
Move disk 1 from peg A to peg B
Move disk 4 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 2 from peg B to peg A
Move disk 1 from peg C to peg A
Move disk 3 from peg B to peg C
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C



Solution for moving 3 disks from A to B

Move 4th disk from A to C

Solution for moving 3 disks from B to C

1. **Why is a stack used to manage function calls in most programming languages?**
 - a) To allow random access to function calls.
 - b) To store the order of calls and return addresses (LIFO).
 - c) Because stacks consume less memory than arrays.
 - d) To store all function calls in the heap.

Data Structures and its Applications

Multiple-Choice-Questions(MCQ's)



1. **Why is a stack used to manage function calls in most programming languages?**
 - a) To allow random access to function calls.
 - b) To store the order of calls and return addresses (LIFO).
 - c) Because stacks consume less memory than arrays.
 - d) To store all function calls in the heap.

2. When a recursive function is called, what gets stored in the stack for each function call?

- a) Only the function name.
- b) Return address, local variables, and parameters.
- c) Only local variables.
- d) Only parameters.

2. When a recursive function is called, what gets stored in the stack for each function call?

- a) Only the function name.
- b) Return address, local variables, and parameters.
- c) Only local variables.
- d) Only parameters.

3. Which of the following is true about nested function calls?

- a) The most recent function call is executed first.
- b) The first function call executes last due to the LIFO property of the stack.
- c) Function calls are stored in a queue.
- d) Nested calls do not require a stack.

3. Which of the following is true about nested function calls?

- a) The most recent function call is executed first.
- b) The first function call executes last due to the LIFO property of the stack.
- c) Function calls are stored in a queue.
- d) Nested calls do not require a stack.

4. What happens if recursion is not terminated properly?

- a) The stack overflows.
- b) The program compiles successfully but returns 0.
- c) It creates an infinite loop without using stack.
- d) Nothing happens, the function continues normally.

4. What happens if recursion is not terminated properly?

- a) The stack overflows.
- b) The program compiles successfully but returns 0.
- c) It creates an infinite loop without using stack.
- d) Nothing happens, the function continues normally.

5. How does the stack help in recursion?

- a) By storing the base case condition.
- b) By storing function frames to return to the previous state.
- c) By allowing random jumps in memory.
- d) By precomputing recursive calls.

5. How does the stack help in recursion?

- a) By storing the base case condition.
- b) By storing function frames to return to the previous state.
- c) By allowing random jumps in memory.
- d) By precomputing recursive calls.



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Infix to Postfix and Prefix Expressions – Implementation

Dinesh Singh

Department of Computer Science & Engineering

- Consider the sum of A and B expressed as $A + B$
- This representation is called infix.
- There are two alternate notations for expressing sum of A and B using the symbols A , B and + , these are
 - Prefix : $+ A B$
 - Postfix : $A B +$
- The prefixes “Pre” , “post” and “in” refers to the relative position of the operator with respect to the two operands.
- In prefix, operators precedes the two operands
- In postfix, the operator follows the two operands
- In infix, the operator is in between the two operands

Conversion of Infix to Postfix

- For Example : consider the expression $A + B * C$
- The evaluation of the above expression requires the knowledge of precedence of operators
- $A + B * C$ can be expressed as $A + (B * C)$ as multiplication takes precedence over addition
- Applying the rules of precedence the above infix expression can be converted to postfix as follows

$$\begin{aligned}A + B * C &= A + (B * C) \\&= A + (BC *) \quad \text{convert the multiplication} \\&= A (B C *) + \quad \text{convert the addition} \\&= A B C * +\end{aligned}$$

Conversion of Infix to Prefix

- For Example : consider the expression $A + B * C$
- Applying the rules of precedence the above infix expression can be converted to prefix as follows

$$\begin{aligned}A + B * C &= A + (B * C) \\&= A + (* B C) \quad \text{convert the multiplication} \\&= + A (* B C) + \quad \text{convert the addition} \\&= + A * B C\end{aligned}$$

Applying the rules of precedence the table shows the conversion of Infix to Postfix and Prefix Expression

Infix	Postfix	Prefix
$A + B * C + D$	$A B C * + D +$	$++ A * B C D$
$(A + B) * (C + D)$	$A B + C D + *$	$* + A B + C D$
$A * B + C * D$	$A B * C D * +$	$+ * A B * C D$
$A + B + C + D$	$A B + C + D +$	$+++ A B C D$
$A \$ B * C - D + E / F / (G + H)$	$A B \$ C * D - E F / G H + / +$	$+ - * \$ A B C D / / E F + G H$
$((A + B) * C - (D - E)) \$ (F + G)$	$A B + C * D E - - F G + \$$	$\$ - * + A B C - D E + F G$
$A - B / (C * D \$ E)$	$A B C D E \$ * / -$	$- A / B * C \$ D E$

$\$$ is the exponentiation operator and its precedence is from right to left

For example $A \$ B \$ C = A \$ (B \$ C)$

opstk is the empty stack

while(not end of input)

{

 symb = next input character

 // if the input symbol is an operand , add it to the postfix string

 if(symb is an operand)

 add symb to postfix string

else

{

 // pop the contents of the stack while the precedence of

 // top of the stack is greater than the precedence of the symbol scanned

 //prcd function returns true in this case

 while(!empty(opstk) and (prcd(stacktop(opstk),symb))

 {

 topsymb=pop(opstk)

 add topsymb to postfix string

 }

```
/* if prcd function returns false, then
if the stack is empty and symbol is not ')', push symb on to the stack*/
    if( empty(opstk) || symb!=')')
        push(opstk,symb)
    else
        topsymb=pop(opstk); // if symb is ')', pop '(' from the stack
}
while(!empty(opstk)
{
    topsymb=pop(opstk)
    add topsymb to postfix string
}
}
```

Infix to postfix conversion - algorithm

- `prcd(OP1,OP2)` is a function that compares the precedence of the top of the stack(`OP1`) and the input symbol (`OP2`) and returns `TRUE` if the precedence is greater else returns `FALSE`.
- Some of the return values of `prcd` function
 - `prcd(' * ', ' + ')` returns `TRUE` : `prcd(' * ', ' / ')` returns `TRUE`
 - `prcd(' + ', ' * ')` returns `FALSE` : `prcd(' / ', ' * ')` returns `TRUE`
 - `prcd(' + ', ' + ')` returns `TRUE` :
 - `prcd(' + ', ' - ')` returns `TRUE`
 - `prcd(' - ', ' - ')` returns `TRUE`
 - `prcd(' $ ', '$')` returns `FALSE` : exponentiation operator, associatively from right to left
 - `prcd(' ( ', op)` , `prcd(op ' (')` returns `FALSE` for any operator
 - `prcd (op , ' )')` returns `TRUE` for any operator
 - `Prcd( ' ( ' , ' ) ')` returns `FALSE`

- The motivation behind the conversion algorithm is the desire to output the operators in the order in which they are to be executed.
- If an incoming operator is of greater precedence than the one on top of the stack, this new operator is pushed on to the stack.
- If on the other hand , the precedence of the new operator is less than the one on the top of the stack, the operator on the top of the stack should be executed first.
- Therefore the top of the stack is popped out and added to the postfix and the incoming symbol is compared with the new top and so on.
- Parenthesis in the input string override the order of operations
- When a left parenthesis is scanned, it is pushed on to the stack
- When its associated right parenthesis is found, all the operators between the two parenthesis are placed on the output string. Because they are to be executed before any operators appearing after the parenthesis

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $A + B * C$

symb	postfix string	opstk	Remarks
A	A	Empty	symb is operand , add 'A'on to the postfix string
+	A	+	symb is operator, and stack empty, push + on to the stack
B	AB	+	symb is operand , Add 'B' to the postfix string
*	AB	+*	symb is operator, precedence of + (stack top) is < than precedence of *, therefore push * on to the stack

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $A + B * C$

symb	postfix string	opstk	Remarks
C	ABC	+ *	symb is operand , add C on to the postfix string
End of input	ABC*	+	Pop * from the stack and add to the postfix string
-	ABC*+	Empty	Pop + from the stack and add to the postfix string

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $(A + B) * C$

symb	postfix string	opstk	Remarks
(	-	Empty	Stack empty, push '(' on to the stack
A	A	(	Symb is an operand, add 'A' to the postfix string
+	A	(+	Precedence of top of the stack '(' is less than '+', push '+' on to the stack
B	AB	(+	Symb is an operand, add B to the postfix string
)	AB+	(	Precedence of stack top '+' is > than ')' Pop '+' from the stack and add to the postfix string
	AB+	empty	Precedence of stack top stack top '(' is not greater than ') and symb is ')', therefore pop '(' from the stack

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $(A + B) * C$

symb	postfix string	opstk	Remarks
*	AB+	*	Stack empty, push '*' on to the stack
C	AB+C	*	Symb is an operand, add 'C' to the postfix string
End of string	AB+C*	Empty	End of input, pop * from the stack and add to the postfix.

Infix to postfix conversion – another algorithm

```
void convert_postfix(char *infix,char*postfix)
{
    i=0;
    char ch;
    j=0;
    push(s,&top,'#');
    while(infix[i]!='\0')
    {
        ch=infix[i];
        //while the precedence of top of stack is greater than the
        //precedence of the input symbol, pop and add to the postfix
        while(stack_prec(peep(s,top))>input_prec(ch))
            postfix[j++]=pop(s,&top);
```

```
if(input_prec(ch)!=stack_prec(peep(s,top)))
    push(s,&top,ch);
else
    pop(s,&top);
i++;
}
while(peep(s,top)!='#')
    postfix[j++]=pop(s,&top);
postfix[j]='\0';
}
```

Infix to postfix conversion – another algorithm

Precedence table

Operator	Input Precedence	Stack Precedence
+, -	1	2
*, /	3	4
\$	6	5
Operands	7	8
)	0	- : Never pushed on to stack
(	9	0
#	-	-1

Example 1:

Input : $A * B + C / D$

Output : $+ * A B / C D$

Example 2 :

Input : $(A - B / C) * (D / K - L)$

Output : $* - A / B C - / D K L$

1: Reverse the infix expression i.e $A+B*C$ will become $C*B+A$.

Note while reversing each '(' will become ')' and each ')' becomes '('.

2: Obtain the postfix expression of the modified expression i.e $CB*A+$.

3: Reverse the postfix expression. Hence in our example prefix is $+A*BC$.

- 1. When converting an infix expression to a postfix expression using a stack, which of the following operations is performed when a right parenthesis) is encountered?**
- a) Push) onto the stack.
 - b) Pop operators from the stack until a (is encountered.
 - c) Discard all operators and continue scanning.
 - d) Push all remaining operators into output.

1. When converting an infix expression to a postfix expression using a stack, which of the following operations is performed when a right parenthesis) is encountered?
- a) Push) onto the stack.
 - b) Pop operators from the stack until a (is encountered.
 - c) Discard all operators and continue scanning.
 - d) Push all remaining operators into output.

2. What is the postfix form of the expression $(A + B) * (C - D)$?

- a) $ABCD+-*$
- b) $AB+CD-*$
- c) $ABCD+*-$
- d) $AB+C-D^*$

2. What is the postfix form of the expression $(A + B) * (C - D)$?

a) ABCD+-*

b) AB+CD-*

c) ABCD+*-

d) AB+C-D*

3. Which of the following statements about operator precedence during conversion is true?

- a) Operators with higher precedence are pushed to the output queue before the stack.
- b) When a new operator is read, operators of higher or equal precedence are popped from the stack to the output.
- c) Precedence of operators is ignored when parentheses are used.
- d) None of the above.

3. Which of the following statements about operator precedence during conversion is true?

- a) Operators with higher precedence are pushed to the output queue before the stack.
- b) When a new operator is read, operators of higher or equal precedence are popped from the stack to the output.
- c) Precedence of operators is ignored when parentheses are used.
- d) None of the above.

4. While converting infix to prefix expression using a stack, what is the most efficient approach?

- a) Convert infix to postfix, then reverse it.
- b) Reverse the infix, swap parentheses, convert to postfix, and then reverse the output.
- c) Use two stacks for operators and operands.
- d) Directly scan the infix expression from left to right.

4. While converting infix to prefix expression using a stack, what is the most efficient approach?

- a) Convert infix to postfix, then reverse it.
- b) Reverse the infix, swap parentheses, convert to postfix, and then reverse the output.
- c) Use two stacks for operators and operands.
- d) Directly scan the infix expression from left to right.

5. Consider the infix expression $A + B * C - D ^ E ^ F$. What will be the correct postfix form (considering standard precedence and associativity)?

a) $ABC^*+DEF^^-$

b) $AB+C*DEF^^-$

c) $AB+C*D^E^F-$

d) $ABC*DE^F^+_-$

5. Consider the infix expression $A + B * C - D ^ E ^ F$. What will be the correct postfix form (considering standard precedence and associativity)?

a) $ABC^*+DEF^^-$

b) $AB+C*DEF^^-$

c) $AB+C*D^E^F-$

d) $ABC*DE^F^+_-$



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Evaluation of Postfix expression and Parenthesis matching

Dinesh Singh

Department of Computer Science & Engineering

- Each operator in a postfix string refers to the previous two operands.
- Each time an operand is read , it is pushed on to the stack
- When an operator is reached, its operands will be the top two elements on to the stack.
- The two elements are popped out, the indicated operation is performed on them and result is pushed on the stack so that it will be available for use as an operand of the next operator.

Evaluation of Postfix Expression - Algorithm

```
opndstk is the empty stack
while(not end of the input) // scan the input string
{
    symb=next input character;
    if (symb is an operand)
        push(opndstk,symb)
    else
    {
        opnd2=pop(opndstk);
        opnd1=pop(opndstk);
        value = result of applying symb to opnd1 and opn2;
        push(opndstk, value);
    }
    return(pop(opndstk));
}
```


Evaluation of Postfix Expression – Trace of the Algorithm

Infix : $3 + 5 * 4$

Postfix expression : $3\ 5\ 4\ *\ +$

Symb	Opnd1	Opnd2	Value	opndstk
3	-			3
5				3, 5
4				3, 5, 4
*	5	4	20	3, 20
+	3	20	23	23

```
int postfix_eval(char* postfix)
{
    int i,top,r;
    int s[10]; //stack
    top=-1;
    i=0;

    while(postfix[i]!='\0')
    {
        char ch=postfix[i];
        if(isoper(ch))
        {
            int op1=pop(s,&top);
            int op2=pop(s,&top);
```

```
switch(ch)
{
    case '+':r=op1+op2;
        push(s,&top,r);
        break;
    case '-':r=op2-op1;
        push(s,&top,r);
        break;
    case '*':r=op1*op2;
        push(s,&top,r);
        break;
    case '/':r=op2/op1;
        push(s,&top,r);
        break;
} //end switch
} //end if
```

```
else
    push(s,&top,ch-'0');//convert charcter to
integer and push
    i++;
} //end while
return(pop(s,&top));
}
```

Parenthesis Matching – overview of the Algorithm

Examples

1. `(( ))` : Valid Expression
2. `(( ( ))` : Invalid Expression (Extra opening parenthesis)
3. `(( )) )` : Invalid Expression (Extra closing parenthesis)
4. `(( } )` : Invalid Expression (Parenthesis mismatch)
5. `(( ) ]` : Invalid Expression (Parenthesis mismatch)

Parenthesis Matching – overview of the Algorithm

1. Read the input symbol from the input expression
2. If the input symbol is one of the open parenthesis (' (' , ' { ' or ' ['), it is pushed on to the stack
3. If the input symbol is of closing parenthesis, stack is popped and the popped parenthesis is compared with the input symbol, if there is a mismatch in the type of the parenthesis, return 0 (Mismatch of parenthesis)
4. If there is a match in the parenthesis , the next input symbol is read.
5. If during this process, if the stack becomes empty, return 0 (Extra closing parenthesis)
6. If at the end of the expression, if the stack is not empty, return 0 (Extra opening parenthesis)
7. If at the end of the input expression, if the stack is empty, return 1. (Parenthesis are matching)

Data Structures and its Applications

Parenthesis Matching - Implementation

```
int match(char *expr)
{
    int i,top;
    char s[10],ch,x;//stack
    i=0;
    top=-1;

    while(expr[i]!='\0')
    {
        ch=expr[i];
        switch(ch)
        {
            case '(':
            case '{':
            case '[':push(s,&top,ch);
                    break;
```

```
case ')':if(!isempty(top))
{
    x=pop(s,&top);
    if(x=='(')
        break;
    else
        return 0;//mismatch of parenthesis
}
else
    return 0;//extra closing parenthesis
```



```
case '}':if(!isempty(top))
{
    x=pop(s,&top);
    if(x=='{')
        break;
    else
        return 0;//mismatch of parenthesis
}
else
    return 0;//extra closing parenthesis
```

```
case ']':if(!isempty(top))
    {
        x=pop(s,&top);
        if(x=='[')
            break;
        else
            return 0;//mismatch of parenthesis
    }
else
    return 0;//extra closing parenthesis

} //end switch
i++;
} //end while
if(isempty(top))
    return 1;
return 0;//extra opening parenthesis
}
```

1. Which of the following correctly represents the steps for evaluating a postfix expression using a stack?
 - a) Scan expression from left to right, push operands, pop two operands on operator, evaluate, and push result.
 - b) Push operators, pop operands, evaluate, and push operator back.
 - c) Scan from right to left, push operators, pop operands when needed.
 - d) Push all symbols first, then evaluate.

Data Structures and its Applications

Multiple-Choice-Questions (MCQ's)



1. Which of the following correctly represents the steps for evaluating a postfix expression using a stack?
 - a) Scan expression from left to right, push operands, pop two operands on operator, evaluate, and push result.
 - b) Push operators, pop operands, evaluate, and push operator back.
 - c) Scan from right to left, push operators, pop operands when needed.
 - d) Push all symbols first, then evaluate.

2. Evaluate the postfix expression $6\ 3\ 2\ *\ +\ 4\ -$. What is the result?

- a) 12
- b) 10
- c) 8
- d) 9

2. Evaluate the postfix expression $6\ 3\ 2\ *\ +\ 4\ -$. What is the result?

a) 12

b) 10

c) 8

d) 9

3. Which of the following is true about the stack used for parenthesis matching?

- a) Only opening brackets are pushed onto the stack.
- b) Both opening and closing brackets are pushed.
- c) Closing brackets are compared with the top of stack; if not matched, it is invalid.
- d) Both a and c.

3. Which of the following is true about the stack used for parenthesis matching?

- a) Only opening brackets are pushed onto the stack.
- b) Both opening and closing brackets are pushed.
- c) Closing brackets are compared with the top of stack; if not matched, it is invalid.
- d) Both a and c.

4. In parenthesis matching, what condition must be satisfied for the string to be valid?

- a) The number of opening and closing parentheses must be equal.
- b) Parentheses must not cross each other (proper nesting).
- c) Every closing parenthesis must match the most recent unmatched opening parenthesis.
- d) All of the above.

4. In parenthesis matching, what condition must be satisfied for the string to be valid?

- a) The number of opening and closing parentheses must be equal.
- b) Parentheses must not cross each other (proper nesting).
- c) Every closing parenthesis must match the most recent unmatched opening parenthesis.
- d) All of the above.

Data Structures and its Applications

Multiple-Choice-Questions (MCQ's)



5. Evaluate the postfix expression $5\ 6\ 2\ +\ *\ 12\ 4\ /\ -$. What is the result?

a) 46

b) 50

c) 40

d) 37

Data Structures and its Applications

Multiple-Choice-Questions (MCQ's)



5. Evaluate the postfix expression $5\ 6\ 2\ +\ *\ 12\ 4\ /\ -$. What is the result?

a) 46

b) 50

c) 40

d) 37



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering